Qubetics Chain Abstraction - audit

Security Assessment

CertiK Assessed on Jan 22nd, 2026





CertiK Assessed on Jan 22nd, 2026

Qubetics Chain Abstraction - audit

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
SDK	Qubetics	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Rust	Preliminary comments published on 12/19/2025
	Final report published on 01/22/2026

Vulnerability Summary



122	100	5	17	0
Total Findings	Resolved	Partially Resolved	Acknowledged	Declined

0	Centralization		Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
3	Critical	1 Resolved, 2 Partially Resolved	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
30	Major	24 Resolved, 1 Partially Resolved, 5 Acknowledged	Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
38	Medium	31 Resolved, 1 Partially Resolved, 6 Acknowledged	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
25	Minor	22 Resolved, 1 Partially Resolved, 2 Acknowledged	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
26	Informational	22 Resolved, 4 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | QUBETICS CHAIN ABSTRACTION - AUDIT

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Overview

Main Flow

Research On DKG

Findings

QCA-01 : Broadcast Of Full Share Vectors Completely Breaks The Threshold Security Guarantees Of The DKG Scheme

QCA-62 : DKG Share Wipe Locks Network Funds

QCA-95 : Using A Fixed Nonce K Leads To Complete Private-Key Compromise (ECDSA Key Recovery)

QCA-03 : Incorrect Nonce Management Leads To Transaction Failures

QCA-100 : Concurrent Calling `set_private_key_from_scalar` Can Lead To Incorrect Private Key Settings

QCA-101 : Lack Of Confirming The Success Of The User's Transaction In `handle_user_mpc_deposit` Can Lead To Fund Losses

QCA-102 : All Registered Users Can Not Be Updated After DKG Is Reset

QCA-103 : Parallel Execution Of Signature Generation And Request Leads To Missing Signature Data And Transaction Failure

QCA-104 : Persistent Error State In Transaction Type Judgment Leads To Incorrect Routing And Potential Fund Loss

QCA-105 : Vault-To-Network Adds 10% Uplift Will Drain Vault Funds During Rebalancing

QCA-106 : Threshold Change Without Atomic DKG Transaction Leads To Signature Failure And Inconsistent Participation

QCA-15 : The `reconstruct_signed_bitcoin_transaction` Function Incorrectly Retrieves BTC Transactions From `pending_transactions`

QCA-17 : `select_mpc_nodes` Function Fabricates VRF Submissions

QCA-18 : Missing Signature/Proof Verification On VrfSubmission Messages

QCA-34 : The Logic Of `get_ethereum_address` To Parse An Ethereum Address From A Public Key Is Incorrect

QCA-35 : Nodes Can't Receive `ConsensusMessage::FinalNodeResult` Message Because This Message Lacks Of `GossipsubMessage::Consensus` Wrapping

QCA-36 : DKG Share Index Mismatch

QCA-37 : Concurrent Aggregation Race Leads To Duplicate Aggregation And Boardcast

QCA-51 : Local Round Assignment Cannot Handle P2P Message Delay, Causing Transaction–Signature Mismatch

QCA-53 : Unused Direct Messaging Function Reduces Protocol Robustness And Should Be Integrated Into Final Signing Flow

QCA-54 : PeerId Inconsistency Between NodeManager And NetworkLayer

QCA-55 : All Nodes Don't Agree On Consistent (`total` , `threshold`) Parameters Before Starting DKG

QCA-63 : Incorrect EVM Reward Distribution Using BTC Units Can Cause Rewards Loss

QCA-64 : Gossip Deposit Intents Bypass Authorization

QCA-73 : Single Compromised Client Can Hijack Entire DKG Network Through `handle_submit_transaction` Function

QCA-74 : Silent Fallback To 300k Gas On Estimation Failure Enables Gas-Draining Reverting Calls

QCA-75 : Attacker-Controlled `chainId` Injected Into Ethereum Signing Preimage

QCA-76 : Unconditional Insert Allows Forged `FinalNodeResult` To Overwrite Any Round

QCA-77 : Consensus-Threshold Bypass Via Spoofed `selector_node_id` In VRF Broadcasts

QCA-96 : The DKG Protocol Lacks Of Round Check

QCA-97 : Lack Of EVM Balance Validation In `create_chain_transaction_from_deposit_intent_internal` Can Lead To DoS Attack

QCA-98 : Concurrent Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Incorrect Group Public Key Settings

QCA-99 : Simultaneous Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Race Conditions When Setting Chain Handlers

QCA-04 : The `check_completion()` Counting All Validation Entries Without Deduplication May Lead To Premature Key Generation

QCA-05 : Using Hardcoded Testnet Network To Generate Bitcoin Address May Cause Fund Loss

QCA-06 : Secret Shares And Keys Logged In Info! Statements

QCA-08 : The `reset()` Method In `DKGNode` Does Not Properly Clean Up Vault-Specific Secrets

QCA-107 : Susceptible To Signature Malleability

QCA-108 : Potential Replay Attack In `handle_user_mpc_deposit`

QCA-109 : The `handle_deposit_intent` Function Lacks Of Price Slippage Protection

QCA-110 : Consensus Readiness Miscounts Broadcasts Instead Of Selected Nodes Allowing Under Or Over-Trigger Of Consensus

QCA-111 : Publicly Broadcast Signature Shares Let Any Node Assemble The Final Signature And Broadcast

QCA-112 : Invalid Or Wrong-Message Signature Shares Can Block Aggregation (DoS)

QCA-19 : The `broadcast_transaction` May Return Incorrect `txid`

QCA-20 : Hardcoded Infura URL With Placeholder `YOUR_PROJECT_ID` Will Cause Runtime Failures

QCA-21 : The `create_script_sig_for_input` Does Not Return Valid Redeem Script For P2SH-Wrapped SegWit

QCA-22 : The `create_witness_stack_for_input` Function Incorrectly Assumes All `SegWit` Inputs Are `P2WPKH`

QCA-23 : VRF Input Order Manipulation Allows Selection Bias

QCA-24 : Deadlock Risk

QCA-38 : The `verify_aggregated_signature` Does Not Fully Verify The ECDSA Signature

QCA-39 : Distributed K-Generation Introduces Single-Point-Of-Failure (SPoF) Due To Strict Full-Node Availability Requirement

QCA-40 : Using Default DKG Index 1 For Peer Will Generate Invalid Aggregated Signatures

QCA-41 : VRF Input Domain Not Locally Reconstructed

QCA-42 : Unauthenticated First-Node Timestamp & Restart Race Break 5-Minute Join Guard

QCA-43 : Missing Challenge And Verification In VRF Implementation Leads To Forged Or Unverified Node Selections

QCA-52 : The DKG Protocol Will Be Unnecessarily And Repeatedly Triggered During Each Peer Connection Events

QCA-56 : Inconsistent Threshold Calculation In DKG Start Methods

QCA-57 : Node Count Increase May Cause DKG Generation Failure

QCA-65 : Exposure Of HMAC Constant In User Info Could Allow An Attacker To Reverse-Engineer The Original `dkg_secret_share`

QCA-66 : The Solver Reward Distribution Logic Completely Omits BTC Reward

QCA-67 : Premature Loop Termination Leads To Incomplete Address Derivation Waiting

QCA-68 : Unsupported Chain Fallback Leads To Incorrect Chain Handler Usage

QCA-69 : Using Non-Reduced HMAC As Scalar Leads To Panic

QCA-70 : Hardcoded Decimal Conversion Logic Leads To Incorrect Transaction Amounts

QCA-78 : Potential Infinite Transaction Processing Loop

QCA-79 : Silent Contract Creation In EIP-155 Transaction Encoding

QCA-80 : Unauthenticated DKG Trigger (`StartADKG` / `ScheduleADKG`) Lets Any Peer Reset DKG And Override Thresholds

QCA-81 : Unauthenticated `TransactionIdBroadcast` Triggers Contract Call Execution

QCA-82 : Wrong `scriptCode` Used For Non-P2WPKH SegWit Inputs (`P2WSH` / `P2SH-P2WPKH`) In BIP143 Preimage

QCA-83 : Leader Election Biased By Lexicographic PeerId Tie-Breaker

QCA-84 : VRF Committee Size Hard-Coded To One Signer Bypassing Threshold Security

QCA-09 : Silently Defaulting To `index=1` Will Undermine DKG Security

QCA-113 : Incorrect Ethereum Address Validation

QCA-114 : Missing Authentication In HTTP Server

QCA-115 : Initializing Kademlia WIth Empty Bootstrap Nodes

QCA-116 : Hardcoded Bitcoin Testnet In `create_bitcoin_handler`

QCA-117 : Hardcoded Testnet Restriction In `create_bitcoin_script_pubkey`

QCA-118 : Potential Inconsistency In The Supported EVM Chains.

QCA-119 : Solver Attribution Missing At Initial Tx-Id Store

QCA-13 : Potential Discrepancy In `derive_peer_id_from_string`

QCA-25 : Missing Public Key Hash Validation In Legacy P2PKH Handling

QCA-26 : Unsafe Nonce Handling

QCA-27 : VRF Secret Key Exposure

QCA-44 : Potential Concurrency Issues In `calculate_reward_per_solver`

QCA-45 : Duplicate Processing Of Consensus Messages

QCA-46 : No Guard Against `share_index == 0`

QCA-58 : The Network Does Not Remove Nodes Upon Unsubscription And ConnectionClosed

QCA-59 : Hardcoded Chain Code Undermines Key Derivation Entropy

QCA-71 : No Proof Of Address Ownership Is Required For Registration

QCA-85 : Transactions With Unknown Transaction Type Should Not Be Handled

QCA-86 : Incorrect Implementation Of `handle_get_status`

QCA-87 : The `select_utxos_for_amount()` Function Fails To Enforce The Dust Limit (546 Satoshis) On The Recipient Output

QCA-88 : The `create_chain_transaction_from_deposit_intent_internal` Function Unconditionally Unwraps `group_public_key` Before Transaction Type Evaluation

QCA-89 : Ignored `manual_max_fee` In `derive_fees` Fallback Path

QCA-90 : Signature Verification Bypass In Ethereum And Bitcoin Handlers

QCA-91 : Missing Final-Node Authorization In `handle_btc_signature_for_round`

QCA-02 : The `generate_group_key` Function May Inadvertently Store Error Messages As Valid Bitcoin Addresses

QCA-07 : Lack Of Scalar Zero-Value Validation

QCA-10 : Using `unwrap` Will Panic

QCA-11 : Unused Variable `vrf_round_requested`

QCA-12 : Use Module-Level Constants Instead Of String Literals

QCA-120 : Lack Of Proper Error Handling In `handle_register_user ` Function

QCA-121 : Duplicated Error Handling

QCA-122 : Meaningless Function `is_bootstrap_node`

QCA-123 : Redundant Check For `aggregated_rounds` Leads To Unnecessary Code Execution

QCA-124 : Redundant Check For `signatures_vec.len() > self.threshold` Leads To Unnecessary Code Execution

QCA-125 : Missing Threshold Validation Leads To Potential Panic When Threshold Is Zero

QCA-126 : Redundant Signing Key Retrieval Logic

QCA-14 : Missing Cache Initialization For `aggregated_rounds`

QCA-28 : Hardcoded `is_testnet`

QCA-29 : Unused Functions

QCA-32 : Plaintext Storage Of Private Keys In RocksDB

QCA-33 : Serial K-Generation And Signing Workflow Causes High Latency; Parallelization Improves Throughput

QCA-47 : Inconsistent Calculation Of `v`

QCA-48 : The `get_available_nodes_from_manager` Function May Panic

QCA-49 : Hardcoded Node Selection Count

QCA-50 : Incorrect Documentation (Comment Does Not Match Actual Behavior)

QCA-60 : Optimizing Threshold Updates In PeerList Handling

QCA-61 : Missing Timer Cancellation Mechanism May Cause Resource Leaks

QCA-72 : Unused Variable In DKG Flow Leads To Dead Code And Potential Logic Omission

QCA-93 : Inconsistent Logic In `store_transaction_error_for_round `

QCA-94 : Discussion On Lack Of Commitment Consistency Check During The DKG Process

Appendix

Disclaimer

APPROACH & METHODS

QUBETICS CHAIN ABSTRACTION - AUDIT

This audit was conducted for Qubetics to evaluate the security and correctness of the smart contracts associated with the Qubetics Chain Abstraction - audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | QUBETICS CHAIN ABSTRACTION - AUDIT

Overview

Qubetics Chain Abstraction is a decentralized, MPC-based cross-chain interoperability protocol that enables secure asset transfers across heterogeneous blockchains. By combining **Distributed Key Generation (DKG)**, **threshold signatures (t-of-n)**, and **VRF-based node selection**, it eliminates single points of failure while maintaining cryptographic security. The architecture consists of a **P2P network layer (libp2p)**, a signing coordination service, and chain-agnostic transaction builders, supporting both **Bitcoin (P2PKH/P2WPKH)** and **Ethereum-style** chains. Key innovations include deterministic key derivation for users/vaults, non-interactive signature aggregation, and optimized broadcast protocols for finality. Designed for DeFi and institutional use cases, Qubetics abstracts away chain-specific complexities, allowing seamless cross-chain operations with decentralized custody.

RPC Module

The RPC module serves as the gateway for external interactions, providing HTTP/JSON-RPC endpoints to submit transactions, query balances, and manage cross-chain operations. It validates requests, forwards them to the P2P network, and returns signed transaction result. Designed for low-latency processing, it integrates with the Signing module for secure transaction construction and supports multi-chain compatibility through a unified API.

Network Module

The Network module handles decentralized peer-to-peer communication using libp2p, featuring Gossipsub for message broadcasting and Kademlia DHT for node discovery. It ensures reliable message propagation for DKG rounds, signature requests, and transaction broadcasts while managing peer connections dynamically.

Signing Module

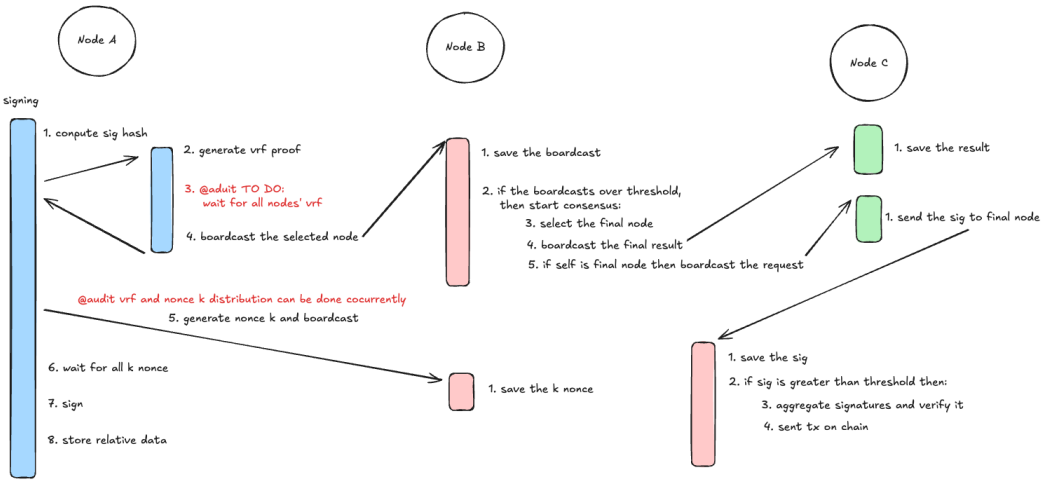
The Signing module coordinates threshold ECDSA signatures for transactions across chains (Bitcoin, Ethereum, etc.). It leverages VRF-based node selection to choose signers, collects partial signatures, and aggregates them into a final valid signature using Lagrange interpolation. It supports key derivation for user-specific and vault keys, handles Bitcoin P2WPKH/P2PKH sighashes, and ensures non-repudiation via recoverable signatures.

DKG Module

The DKG (Distributed Key Generation) module enables secure t-of-n threshold key sharing among nodes using verifiable secret sharing (VSS). Each node generates polynomial commitments, distributes secret shares, and validates contributions via Pedersen commitments. Upon completion, it derives group public keys and user-specific tweaked keys while preventing single-point compromises. The module triggers automatically when the network reaches a quorum.

Main Flow

This diagram effectively illustrates the process from initial signature to transaction finalization on the blockchain. This workflow focuses solely on the business logic, starting from a single node initiating the signature. For simplicity, the node communication segments are depicted as each communication reaching a new node to complete the business process. This representation is for illustrative purposes only; in actual P2P networks, each communication is broadcast.



Here is the key flow in concise bullet points:

- Node A initiates the signing request by hashing the message and generating a VRF proof for leader election.
- All nodes produce VRF outputs, and once collected, Node A broadcasts the selected leader node.
- In parallel, Node A generates a nonce (k) and broadcasts it, then waits to receive nonce values from other nodes before producing its partial signature.
- Node B receives the VRF broadcasts, stores them, and when the number of broadcasts exceeds the required threshold, it triggers consensus to finalize the leader node.
- After consensus, Node B broadcasts the final leader result and, if designated as the leader, broadcasts the signing request to all nodes.
- Node B collects nonce values and later partial signatures from peers; once partial signatures exceed the threshold, it aggregates and verifies the final signature.
- Node C participates by saving the final leader result and sending its partial signature to the designated leader.
- The leader node aggregates valid partial signatures, verifies the result, and submits the transaction on-chain.

Research On DKG

Distributed Key Generation (DKG) is a cryptographic protocol that allows a group of n participants to collaboratively generate a shared secret key without any single party ever learning the complete key. This is fundamental in threshold cryptography, where operations (e.g., signing) require a minimum of t participants (where $t \leq n$) to cooperate.

Core Objectives

DKG enables mutually distrustful parties to collaboratively generate cryptographic keys while ensuring:

- Threshold Security: Any subset of ≥t nodes can reconstruct the key, while ≤t-1 learn nothing
- Decentralized Trust: Eliminates single points of trust (vs. trusted dealer models)
- Verifiability: All participants can verify others' compliance with the protocol

DKG Flow

1. Secret Sharing

- Each participant i generates a random polynomial of degree t:

  $$f_i(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_t x^t \pmod{q}$$

- The constant term a_0 is the secret contribution.
- Participant j receives $s_{i,j} = f_i(j)$
- The secret is reconstructed via Lagrange interpolation:

  $$a_0 = \sum_{i \in S} s_i \cdot \prod_{\substack{j \in S \\ j \neq i}} \frac{x-j}{i-j}$$

  where S is a set of t shares.

2. Distributed Generation

In DKG, no single dealer exists. Instead, each participant P_i acts as a dealer:

- Commitment Phase: P_i generates a random polynomial $f_i(x)$ and broadcasts commitments
- Share Distribution: P_i sends $s_{i,j} = f_i(j)$ to P_j
- Verification: Each P_j verifies consistency of received shares against commitments
- Key Aggregation: The global secret x is the sum of all $a_{i,0}$ and the public key is $Y = g^x = \prod_{i=1}^{n} C_{i,0} \pmod{p}$

FINDINGS | QUBETICS CHAIN ABSTRACTION - AUDIT



122	3	0	30	38	25	26
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Qubetics to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 122 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
QCA-01	Broadcast Of Full Share Vectors Completely Breaks The Threshold Security Guarantees Of The DKG Scheme	Logical Issue	Critical	Resolved
QCA-62	DKG Share Wipe Locks Network Funds	Design Issue	Critical	Partially Resolved
QCA-95	Using A Fixed Nonce K Leads To Complete Private-Key Compromise (ECDSA Key Recovery)	Design Issue	Critical	Partially Resolved
QCA-03	Incorrect Nonce Management Leads To Transaction Failures	Denial of Service	Major	Resolved
QCA-100	Concurrent Calling `set_private_key_from_scalar` Can Lead To Incorrect Private Key Settings	Concurrency	Major	Resolved
QCA-101	Lack Of Confirming The Success Of The User's Transaction In `handle_user_mpc_deposit` Can Lead To Fund Losses	Logical Issue	Major	Acknowledged
QCA-102	All Registered Users Can Not Be Updated After DKG Is Reset	Logical Issue	Major	Resolved
QCA-103	Parallel Execution Of Signature Generation And Request Leads To Missing Signature Data And Transaction Failure	Design Issue	Major	Resolved

ID	Title	Category	Severity	Status
QCA-104	Persistent Error State In Transaction Type Judgment Leads To Incorrect Routing And Potential Fund Loss	Design Issue	Major	Acknowledged
QCA-105	Vault-To-Network Adds 10% Uplift Will Drain Vault Funds During Rebalancing	Logical Issue	Major	Acknowledged
QCA-106	Threshold Change Without Atomic DKG Transaction Leads To Signature Failure And Inconsistent Participation	Design Issue	Major	Resolved
QCA-15	The `reconstruct_signed_bitcoin_transaction` Function Incorrectly Retrieves BTC Transactions From `pending_transactions`	Logical Issue	Major	Resolved
QCA-17	`select_mpc_nodes` Function Fabricates VRF Submissions	Volatile Code	Major	Resolved
QCA-18	Missing Signature/Proof Verification On VrfSubmission Messages	Volatile Code	Major	Resolved
QCA-34	The Logic Of `get_ethereum_address` To Parse An Ethereum Address From A Public Key Is Incorrect	Logical Issue	Major	Resolved
QCA-35	Nodes Can't Receive `ConsensusMessage::FinalNodeResult` Message Because This Message Lacks Of `GossipsubMessage::Consensus` Wrapping	Logical Issue	Major	Resolved
QCA-36	DKG Share Index Mismatch	Logical Issue	Major	Resolved
QCA-37	Concurrent Aggregation Race Leads To Duplicate Aggregation And Boardcast	Design Issue	Major	Resolved
QCA-51	Local Round Assignment Cannot Handle P2P Message Delay, Causing Transaction–Signature Mismatch	Design Issue	Major	Acknowledged

ID	Title	Category	Severity	Status
QCA-53	Unused Direct Messaging Function Reduces Protocol Robustness And Should Be Integrated Into Final Signing Flow	Design Issue	Major	Resolved
QCA-54	PeerId Inconsistency Between NodeManager And NetworkLayer	Inconsistency	Major	Resolved
QCA-55	All Nodes Don't Agree On Consistent (`total` , `threshold`) Parameters Before Starting DKG	Logical Issue	Major	Partially Resolved
QCA-63	Incorrect EVM Reward Distribution Using BTC Units Can Cause Rewards Loss	Logical Issue	Major	Resolved
QCA-64	Gossip Deposit Intents Bypass Authorization	Access Control	Major	Resolved
QCA-73	Single Compromised Client Can Hijack Entire DKG Network Through `handle_submit_transaction` Function	Logical Issue	Major	Resolved
QCA-74	Silent Fallback To 300k Gas On Estimation Failure Enables Gas-Draining Reverting Calls	Logical Issue	Major	Resolved
QCA-75	Attacker-Controlled `chainId` Injected Into Ethereum Signing Preimage	Logical Issue	Major	Resolved
QCA-76	Unconditional Insert Allows Forged `FinalNodeResult` To Overwrite Any Round	Volatile Code	Major	Resolved
QCA-77	Consensus-Threshold Bypass Via Spoofed `selector_node_id` In VRF Broadcasts	Design Issue	Major	Resolved
QCA-96	The DKG Protocol Lacks Of Round Check	Logical Issue	Major	Resolved
QCA-97	Lack Of EVM Balance Validation In `create_chain_transaction_from_deposit_intent_internal` Can Lead To DoS Attack	Denial of Service	Major	Acknowledged

ID	Title	Category	Severity	Status
QCA-98	Concurrent Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Incorrect Group Public Key Settings	Concurrency	Major	Resolved
QCA-99	Simultaneous Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Race Conditions When Setting Chain Handlers	Logical Issue	Major	Resolved
QCA-04	The `check_completion()` Counting All Validation Entries Without Deduplication May Lead To Premature Key Generation	Logical Issue	Medium	Resolved
QCA-05	Using Hardcoded Testnet Network To Generate Bitcoin Address May Cause Fund Loss	Logical Issue	Medium	Resolved
QCA-06	Secret Shares And Keys Logged In Info! Statements	Volatile Code	Medium	Resolved
QCA-08	The `reset()` Method In `DKGNode` Does Not Properly Clean Up Vault-Specific Secrets	Logical Issue	Medium	Resolved
QCA-107	Susceptible To Signature Malleability	Logical Issue	Medium	Resolved
QCA-108	Potential Replay Attack In `handle_user_mpc_deposit`	Logical Issue	Medium	Acknowledged
QCA-109	The `handle_deposit_intent` Function Lacks Of Price Slippage Protection	Financial Manipulation	Medium	Acknowledged
QCA-110	Consensus Readiness Miscounts Broadcasts Instead Of Selected Nodes Allowing Under Or Over-Trigger Of Consensus	Volatile Code	Medium	Resolved
QCA-111	Publicly Broadcast Signature Shares Let Any Node Assemble The Final Signature And Broadcast	Design Issue	Medium	Resolved

ID	Title	Category	Severity	Status
QCA-112	Invalid Or Wrong-Message Signature Shares Can Block Aggregation (DoS)	Denial of Service	Medium	Resolved
QCA-19	The `broadcast_transaction` May Return Incorrect `txid`	Logical Issue	Medium	Resolved
QCA-20	Hardcoded Infura URL With Placeholder `YOUR_PROJECT_ID` Will Cause Runtime Failures	Logical Issue	Medium	Resolved
QCA-21	The `create_script_sig_for_input` Does Not Return Valid Redeem Script For P2SH-Wrapped SegWit	Logical Issue	Medium	Resolved
QCA-22	The `create_witness_stack_for_input` Function Incorrectly Assumes All `SegWit` Inputs Are `P2WPKH`	Inconsistency	Medium	Resolved
QCA-23	VRF Input Order Manipulation Allows Selection Bias	Volatile Code	Medium	Resolved
QCA-24	Deadlock Risk	Logical Issue	Medium	Resolved
QCA-38	The `verify_aggregated_signature` Does Not Fully Verify The ECDSA Signature	Logical Issue	Medium	Resolved
QCA-39	Distributed K-Generation Introduces Single-Point-Of-Failure (SPoF) Due To Strict Full-Node Availability Requirement	Design Issue	Medium	Resolved
QCA-40	Using Default DKG Index 1 For Peer Will Generate Invalid Aggregated Signatures	Logical Issue	Medium	Resolved
QCA-41	VRF Input Domain Not Locally Reconstructed	Volatile Code	Medium	Resolved
QCA-42	Unauthenticated First-Node Timestamp & Restart Race Break 5-Minute Join Guard	Volatile Code	Medium	Resolved
QCA-43	Missing Challenge And Verification In VRF Implementation Leads To Forged Or Unverified Node Selections	Design Issue	Medium	Resolved

ID	Title	Category	Severity	Status
QCA-52	The DKG Protocol Will Be Unnecessarily And Repeatedly Triggered During Each Peer Connection Events	Logical Issue	Medium	Acknowledged
QCA-56	Inconsistent Threshold Calculation In DKG Start Methods	Inconsistency	Medium	Resolved
QCA-57	Node Count Increase May Cause DKG Generation Failure	Logical Issue	Medium	Resolved
QCA-65	Exposure Of HMAC Constant In User Info Could Allow An Attacker To Reverse-Engineer The Original `dkg_secret_share`	Logical Issue	Medium	Resolved
QCA-66	The Solver Reward Distribution Logic Completely Omits BTC Reward	Logical Issue	Medium	Resolved
QCA-67	Premature Loop Termination Leads To Incomplete Address Derivation Waiting	Logical Issue	Medium	Resolved
QCA-68	Unsupported Chain Fallback Leads To Incorrect Chain Handler Usage	Logical Issue	Medium	Resolved
QCA-69	Using Non-Reduced HMAC As Scalar Leads To Panic	Design Issue	Medium	Resolved
QCA-70	Hardcoded Decimal Conversion Logic Leads To Incorrect Transaction Amounts	Volatile Code	Medium	Resolved
QCA-78	Potential Infinite Transaction Processing Loop	Logical Issue	Medium	Resolved
QCA-79	Silent Contract Creation In EIP-155 Transaction Encoding	Logical Issue	Medium	Resolved
QCA-80	Unauthenticated DKG Trigger (`StartADKG` / `ScheduleADKG`) Lets Any Peer Reset DKG And Override Thresholds	Access Control	Medium	Acknowledged
QCA-81	Unauthenticated `TransactionIdBroadcast` Triggers Contract Call Execution	Access Control	Medium	Partially Resolved

ID	Title	Category	Severity	Status
QCA-82	Wrong `scriptCode` Used For Non-P2WPKH SegWit Inputs (`P2WSH` / `P2SH-P2WPKH`) In BIP143 Preimage	Logical Issue	Medium	Acknowledged
QCA-83	Leader Election Biased By Lexicographic PeerId Tie-Breaker	Volatile Code	Medium	Resolved
QCA-84	VRF Committee Size Hard-Coded To One Signer Bypassing Threshold Security	Volatile Code	Medium	Acknowledged
QCA-09	Silently Defaulting To `index=1` Will Undermine DKG Security	Logical Issue	Minor	Resolved
QCA-113	Incorrect Ethereum Address Validation	Logical Issue	Minor	Resolved
QCA-114	Missing Authentication In HTTP Server	Access Control	Minor	Partially Resolved
QCA-115	Initializing Kademlia WIth Empty Bootstrap Nodes	Logical Issue	Minor	Resolved
QCA-116	Hardcoded Bitcoin Testnet In `create_bitcoin_handler`	Volatile Code	Minor	Resolved
QCA-117	Hardcoded Testnet Restriction In `create_bitcoin_script_pubkey`	Logical Issue	Minor	Resolved
QCA-118	Potential Inconsistency In The Supported EVM Chains.	Inconsistency	Minor	Resolved
QCA-119	Solver Attribution Missing At Initial Tx-Id Store	Volatile Code	Minor	Resolved
QCA-13	Potential Discrepancy In `derive_peer_id_from_string`	Logical Issue	Minor	Resolved
QCA-25	Missing Public Key Hash Validation In Legacy P2PKH Handling	Logical Issue	Minor	Resolved
QCA-26	Unsafe Nonce Handling	Logical Issue	Minor	Resolved

ID	Title	Category	Severity	Status
QCA-27	VRF Secret Key Exposure	Volatile Code	Minor	Resolved
QCA-44	Potential Concurrency Issues In `calculate_reward_per_solver`	Concurrency	Minor	Resolved
QCA-45	Duplicate Processing Of Consensus Messages	Logical Issue	Minor	Acknowledged
QCA-46	No Guard Against `share_index == 0`	Volatile Code	Minor	Resolved
QCA-58	The Network Does Not Remove Nodes Upon Unsubscription And ConnectionClosed	Logical Issue	Minor	Resolved
QCA-59	Hardcoded Chain Code Undermines Key Derivation Entropy	Volatile Code	Minor	Resolved
QCA-71	No Proof Of Address Ownership Is Required For Registration	Volatile Code	Minor	Resolved
QCA-85	Transactions With Unknown Transaction Type Should Not Be Handled	Logical Issue	Minor	Resolved
QCA-86	Incorrect Implementation Of `handle_get_status`	Volatile Code	Minor	Resolved
QCA-87	The `select_utxos_for_amount()` Function Fails To Enforce The Dust Limit (546 Satoshis) On The Recipient Output	Logical Issue	Minor	Resolved
QCA-88	The `create_chain_transaction_from_deposit_intent_internal` Function Unconditionally Unwraps `group_public_key` Before Transaction Type Evaluation	Logical Issue	Minor	Resolved
QCA-89	Ignored `manual_max_fee` In `derive_fees` Fallback Path	Logical Issue	Minor	Resolved
QCA-90	Signature Verification Bypass In Ethereum And Bitcoin Handlers	Logical Issue	Minor	Acknowledged

ID	Title	Category	Severity	Status
QCA-91	Missing Final-Node Authorization In `handle_btc_signature_for_round`	Volatile Code	Minor	Resolved
QCA-02	The `generate_group_key` Function May Inadvertently Store Error Messages As Valid Bitcoin Addresses	Logical Issue	Informational	Resolved
QCA-07	Lack Of Scalar Zero-Value Validation	Volatile Code	Informational	Resolved
QCA-10	Using `unwrap` Will Panic	Volatile Code	Informational	Resolved
QCA-11	Unused Variable `vrf_round_requested`	Coding Issue	Informational	Resolved
QCA-12	Use Module-Level Constants Instead Of String Literals	Coding Issue	Informational	Resolved
QCA-120	Lack Of Proper Error Handling In `handle_register_user` Function	Coding Issue	Informational	Resolved
QCA-121	Duplicated Error Handling	Volatile Code	Informational	Resolved
QCA-122	Meaningless Function `is_bootstrap_node`	Logical Issue	Informational	Resolved
QCA-123	Redundant Check For `aggregated_rounds` Leads To Unnecessary Code Execution	Code Optimization	Informational	Resolved
QCA-124	Redundant Check For `signatures_vec.len() > self.threshold` Leads To Unnecessary Code Execution	Code Optimization	Informational	Resolved
QCA-125	Missing Threshold Validation Leads To Potential Panic When Threshold Is Zero	Design Issue, Code Optimization	Informational	Resolved
QCA-126	Redundant Signing Key Retrieval Logic	Coding Issue	Informational	Resolved

ID	Title	Category	Severity	Status
QCA-14	Missing Cache Initialization For `aggregated_rounds`	Coding Issue	Informational	Acknowledged
QCA-28	Hardcoded `is_testnet`	Code Optimization	Informational	Resolved
QCA-29	Unused Functions	Coding Issue	Informational	Resolved
QCA-32	Plaintext Storage Of Private Keys In RocksDB	Volatile Code	Informational	Resolved
QCA-33	Serial K-Generation And Signing Workflow Causes High Latency; Parallelization Improves Throughput	Code Optimization	Informational	Acknowledged
QCA-47	Inconsistent Calculation Of `v`	Coding Issue	Informational	Resolved
QCA-48	The `get_available_nodes_from_manager` Function May Panic	Coding Issue	Informational	Resolved
QCA-49	Hardcoded Node Selection Count	Code Optimization	Informational	Resolved
QCA-50	Incorrect Documentation (Comment Does Not Match Actual Behavior)	Code Optimization	Informational	Resolved
QCA-60	Optimizing Threshold Updates In PeerList Handling	Logical Issue	Informational	Resolved
QCA-61	Missing Timer Cancellation Mechanism May Cause Resource Leaks	Coding Issue	Informational	Acknowledged
QCA-72	Unused Variable In DKG Flow Leads To Dead Code And Potential Logic Omission	Coding Style	Informational	Resolved
QCA-93	Inconsistent Logic In `store_transaction_error_for_round`	Inconsistency	Informational	Resolved
QCA-94	Discussion On Lack Of Commitment Consistency Check During The DKG Process	Design Issue	Informational	Acknowledged

QCA-01 Broadcast Of Full Share Vectors Completely Breaks The Threshold Security Guarantees Of The DKG Scheme

Category	Severity	Location	Status
Logical Issue	Critical	src/adkg_secp256k1/mod.rs (chain-abstraction): 302~319	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The current implementation broadcasts all secret shares (`shares: Vec<KeyShare>`) generated by each dealer node (`DKGNode`) to the entire gossip network. This allows any participant (or eavesdropper) to collect all evaluations (N shares) of each dealer's polynomial, enabling them to:

1. Interpolate each dealer's secret polynomial (since N ≥ threshold).
2. Extract every dealer's constant term (private key contribution).
3. Sum these terms to reconstruct the global secret key, effectively reducing the DKG protocol to single-party key generation.

This flaw completely breaks the threshold security guarantees of the DKG scheme, as no collaboration is needed—any single malicious actor can recover the full secret key by passively observing the network.

The vulnerability stems from broadcasting all shares publicly:

`src/adkg_secp256k1/mod.rs`

```
async fn broadcast_shares(
        &self,
        shares: Vec<KeyShare>,
        commitments: Vec<SerializablePoint>,
    ) -> Result<()> {
        let dkg_msg = DKGMessage::ShareDistribution {
            from: self.id.clone(),
            shares,
            commitments,
        };

        let gossip_wrap = GossipsubMessage::DKG(dkg_msg);

        let msg_bytes = serde_json::to_vec(&gossip_wrap)?;
        info!("📡 Broadcasting DKG message: {} bytes", msg_bytes.len());
        self.broadcast(MSG_TOPIC_DKG, &msg_bytes).await?;
        Ok(())
    }
```

Each `DKGNode` generates N shares (`KeyShare`) for N participants. Instead of securely sending each share only to its intended recipient, the protocol broadcasts all shares (`shares: Vec<KeyShare>`) via `MSG_TOPIC_DKG` . This allows adversaries to collect enough points (≥ t) to reconstruct the polynomial f(x) via Lagrange interpolation.

To reconstruct the global private key using all shares collected from the vulnerable DKG implementation, an attacker can leverage Lagrange interpolation to recover each node's polynomial and then sum their constant terms. Below is a step-by-step example:

Example Scenario

```
Threshold (t): 3
Total Nodes (n): 5
Attacker: Controls Node 1 (malicious) and passively collects all broadcasted shares.
```

Step 1: Collect All Shares

Assume the attacker intercepts shares broadcasted by Nodes 2–5 during broadcast_shares():

```
Node2 = f₂(1), f₂(2), f₂(3), f₂(4), f₂(5)
Node3 = f₃(1), f₃(2), f₃(3), f₃(4), f₃(5)
...
...
```

The attacker now has:

```
For Node 2: {(i, f₂(i))} for i ∈ [1,5]
For Node 3: {(i, f₃(i))} for i ∈ [1,5]
... (and similarly for Nodes 4–5).
```

Step 2: Reconstruct Each Polynomial

Using Lagrange interpolation, the attacker can recover any polynomial $f_j(x)$ of degree $t-1 = 2$ from any 3 shares.

For Node 2's polynomial $f_2(x)$:

1. Pick 3 points: $(1, f_2(1)), (2, f_2(2)), (3, f_2(3))$.
2. Compute $f_2(x)$ using Lagrange basis polynomials:

$$f_2(x) = f_2(1) \cdot L_1(x) + f_2(2) \cdot L_2(x) + f_2(3) \cdot L_3(x)$$

Where:

$$L_1(x) = (x-2)(x-3)/((1-2)(1-3))$$
$$L_2(x) = (x-1)(x-3)/((2-1)(2-3))$$
$$L_3(x) = (x-1)(x-2)/((3-1)(3-2))$$

3. Evaluate $f_2(0)$ to get Node 2's secret contribution:

$$f_2(0) = f_2(1)\cdot(6) + f_2(2)\cdot(-3) + f_2(3)\cdot(2)$$

Repeat for Nodes 3–5 to get $f_3(0)$, $f_4(0)$, $f_5(0)$.

Step 3: Sum All Constant Terms

The global private key S is the sum of all nodes' constant terms:

$$S = f_1(0) + f_2(0) + f_3(0) + f_4(0) + f_5(0)$$

(The attacker already knows $f_1(0)$ as Node 1's owner.)

Step 4: Derive the Global Public Key

Compute the corresponding public key:

$$P = S \cdot G$$

Where G is the elliptic curve generator.

Secure Design Reference: FROST-DKG

The FROST-DKG protocol explicitly avoids broadcast:

```
Here, f_i(0) = a_i[0] acts as the secret scalar to be shared. Participant i computes
a VSS share shares[j] = f_i(j+1) for every participant j (including j = i), which is
supposed to be sent to participant j in private.
```

Recommendation

Consider not broadcasting all shares, instead, securely sending each share only to its intended recipient (e.g., via encrypted P2P messages).

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by sending shares directly to nodes in commit 3d23cf23ecd9b1175fd2485fdda3537eef8448db.

QCA-62 | DKG Share Wipe Locks Network Funds

Category	Severity	Location	Status
Design Issue	Critical	src/adkg_secp256k1/mod.rs (chain-abstraction): 82~102	Partially Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `adkg_secp256k1/mod.rs`
- `src/network/mod.rs`

Every DKG start begins with `DKGNode::reset` wiping all locally stored shares, including the persisted final secret/public key that the node later uses for network-level signing.

```
//adkg_secp256k1/mod.rs
  pub async fn reset(&mut self) {
      if let Err(e) = self.storage.clear_shares().await {
          warn!("Failed to clear shares: {}", e);
      }
      if let Err(e) = self.storage.clear_commitments().await {
          warn!("Failed to clear commitments: {}", e);
      }
      if let Err(e) = self.storage.clear_validations().await {
          warn!("Failed to clear validations: {}", e);
      }
      if let Err(e) = self.storage.delete_final_secret().await {
          warn!("Failed to delete final secret: {}", e);
      }
      if let Err(e) = self.storage.delete_final_public().await {
          warn!("Failed to delete final public key: {}", e);
      }
      self.status = DKGStatus::New;
      self.round += 1;
  }
```

This reset runs on every manual or scheduled DKG trigger.

```
        // 779:807:src/network/mod.rs
                            RPCRequest::StartADKG => {
                                match self.dkg_node.as_mut() {
                                    Some(dkg) => {
                                        info!("🚀 Received DKG start request from
peer {}", peer);
                                        let total =
self.node_manager.get_total_nodes();
                                        let threshold =
self.node_manager.get_threshold();
                                        dkg.update_node_params(total, threshold);
                                        dkg.reset().await;
                                        if let Some(signing_node) = &mut
self.signing_node {

signing_node.set_threshold(self.node_manager.get_total_nodes() - 1);
                                        }
                                        self.node_manager.set_threshold(total - 1);
                                        // ...
```

```
  //src/network/mod.rs
            ChannelMessage::ScheduledDKGStart => {
                let total = self.node_manager.get_total_nodes();
                let threshold = self.node_manager.get_threshold();
                if let Some(dkg) = &mut self.dkg_node {
                    dkg.update_node_params(total, threshold);
                    dkg.reset().await;
                    if let Some(signing_node) = &mut self.signing_node {
                        signing_node.set_threshold(total/2 + 1);
                    }
                    self.node_manager.set_threshold(total/2 + 1);
                    if let Err(e) = dkg.start_dkg(self.node_manager.clone()).await {
                        warn!("❌ Scheduled DKG start failed: {:?}", e);
                    }
                } else {
                    warn!("❌ DKG node not initialized at scheduled start!");
                }
            }
```

The deleted `final_secret` is the node's signing share for network-controlled addresses (e.g., withdrawals or vault operations). Once wiped, the node cannot service `NetworkToTarget` flows until a fresh DKG completes, and any assets remaining on addresses derived from the previous group key become permanently inaccessible.

```
  //src/network/mod.rs
                let signing_key: Option<k256::Scalar> = match transaction_type {
                    crate::types::TransactionType::UserToVault => {

self.get_signing_key_for_user(user_eth_address.as_ref()).await
                    }
                    crate::types::TransactionType::NetworkToTarget => {
                        if let Some(dkg_node) = &self.dkg_node {
                            dkg_node.get_final_secret_share().await
                        } else {
                            None
                        }
                    }
                    crate::types::TransactionType::VaultToNetwork => {
                        if let Some(dkg_node) = &self.dkg_node {
                            dkg_node.get_vault_tweaked_secret_share().await
                        } else {
                            None
                        }
                    }
                };
```

Because the reset happens before the new round succeeds, any failure or delay during the fresh DKG leaves the network keyless, and even a successful rerun rotates the key, so historical funds guarded by the prior share can never be unlocked.

Recommendation

Gate key deletion on confirmed completion of the replacement DKG (or on an explicit migration workflow), store multiple key epochs, and block scheduled restarts while assets remain under the previous key.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and partially resolved the issue in commit a71556df0537417f1c3c8d4f72858200c4c4ed87.

> we have set the initial nodes on the network to 4 and after the forth node joins the network the DKG triggers but it doesn't mean that DKG will only triggers once , it will be triggered each time the node 5th , 6th up to n node will joins the network with refreshed share where the network private key remains the same using proactive resharing.

[CertiK, 12/29/2025]: The auditor raised a concern where the initial-DKG completion flag is not set if the run executes with `total_nodes != 4`, leaving the system unguarded against re-triggered DKG in the new commit.

> An introduced issue needs to be handled.
>
> The initial-DKG completion flag is not set if the run executes with `total_nodes != 4`, leaving the system >unguarded against re-triggered DKG. The `ScheduledDKGStart` seems designed to start the initial DKG with 4 nodes, however, it does not panic if total nodes are not 4, only emits a warning message.

> An exploit may arise such as when the initial DKG starts with 5 nodes, as the DKG completion flag is not set, >which allows a new DKG ceremony to be retriggered.

QCA-95 Using A Fixed Nonce K Leads To Complete Private-Key Compromise (ECDSA Key Recovery)

Category	Severity	Location	Status
Design Issue	Critical	src/signing/mod.rs (chain-abstraction): 1610~1677, 1972~2039, 2041~2108	Partially Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

Three code snippets were mentioned in the location, all pertaining to the same issue. For illustrative purposes, function `create_ecdsa_signature_with_key` is used as an example here.

1. The function `create_ecdsa_signature_with_key` is intended to produce a recoverable ECDSA signature for a given `message` using the provided `signing_key`. It manually constructs the signature by choosing a nonce `k`, computing `R = k·G`, deriving `r = R.x mod n`, computing `s = k⁻¹·(z + r·x) mod n`, and assembling a `RecoverableSignature`.

```
1610      pub fn create_ecdsa_signature_with_signing_key(
1611          &self,
1612          message: &Message,
1613          signing_key: &SecretKey,
1614      ) -> Result<RecoverableSignature> {
1615
        // Your exact proven custom signature logic (copied from
create_ecdsa_signature)
1616          // 1) Use provided signing key and context
1617          let sk = signing_key;
1618          let secp = Secp256k1::new();
1619
1620          // 2) Fixed nonce k = 0x42…42
1621          let k_bytes = [0x42u8; 32];
1622          let sec_nonce =
1623
           SecretKey::from_slice(&k_bytes).map_err(|e| anyhow!("invalid nonce
slice: {}", e))?;
1624
1625          // 3) R = k·G
1626          let r = PublicKey::from_secret_key(&secp, &sec_nonce);
1627          let rser = r.serialize_uncompressed(); // [0x04 || X(32) || Y(32)]
1628          info!("[SIGNING] R serialized: {:?}", rser);
1629          if BigUint::from_bytes_be(&rser) == BigUint::from(0u32) {
1630              return Err(anyhow!("r is zero, need new nonce"));
1631          }
1632
1633          // 4) r = X mod n
1634          let mut r_bytes: [u8; 32] = [0u8; 32];
1635          r_bytes.copy_from_slice(&rser[1..33]);
1636          let r = BigUint::from_bytes_be(&r_bytes);
1637
1638          // 5) z = message hash as BigUint
1639          let z = BigUint::from_bytes_be(message.as_ref());
1640
1641          // 6) x = private key as BigUint
1642          let sk_bytes = sk.secret_bytes();
1643          let x = BigUint::from_bytes_be(&sk_bytes);
1644
1645          // 7) Curve order n
1646          let n = BigUint::parse_bytes(
1647
           b"FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141",
1648              16,
1649          )
1650          .unwrap();
1651
1652          // 8) k⁻¹ mod n
1653          let k_inv = BigUint::from_bytes_be(&k_bytes)
1654              .modinv(&n)
1655              .ok_or_else(|| anyhow!("nonce not invertible"))?;
1656
```

```
1657          // 9) s = k⁻¹·(z + r·x) mod n
1658          let s = (&k_inv * (&z + (&r * &x) % &n)) % &n;
1659
1660          // 10) Serialize r‖s into 64-byte array
1661          let mut sig_bytes = [0u8; 64];
1662          let r_be: Vec<u8> = r.to_bytes_be();
1663          let s_be = s.to_bytes_be();
1664          sig_bytes[32 - r_be.len()..32].copy_from_slice(&r_be);
1665          sig_bytes[64 - s_be.len()..64].copy_from_slice(&s_be);
1666
1667          // 11) Compute RecoveryId from R_y parity
1668          let y_parity = (rser[64] & 1) as i32;
1669          let rid: RecoveryId =
1670
            RecoveryId::from_i32(y_parity).map_err(|e| anyhow!("invalid recovery ID:
{}", e))?;
1671
1672          // 12) Build the recoverable signature
1673          let rec_sig = RecoverableSignature::from_compact(&sig_bytes, rid)
1674              .map_err(|e| anyhow!("invalid compact signature: {}", e))?;
1675
1676          Ok(rec_sig)
1677      }
```

2. The implementation uses a **constant, fixed nonce** (`k_bytes = [0x42u8; 32]`) for every signature instead of generating a fresh, unpredictable nonce per signing operation. Reusing the same `k` across different messages makes the ECDSA algebra solvable for the private key: an attacker who obtains two signatures `(r, s1)` and `(r, s2)` made with the same `k` can compute the private key `d` directly. The code also logs `R` (`info!("[SIGNING] R serialized: {:?}"...)`), which leaks more signature-internal data.

In ECDSA, the nonce `k` must never be reused across different messages or transactions. The signing formula is:

```
s = k⁻¹ * (z + r * d) mod n
```

where:

- `d` = private key
- `z` = message hash
- `r` = derived from `k * G`
- `n` = curve order

If the same `k` is reused for two different messages (z_1 and z_2), both signatures share the same `r` :

$$s_1 = k^{-1} * (z_1 + r * d) \bmod n$$
$$s_2 = k^{-1} * (z_2 + r * d) \bmod n$$

An attacker can eliminate `k` and solve directly for the private key `d` :

$$d = (s_1 * z_2 - s_2 * z_1) * (r * (s_2 - s_1))^{-1} \bmod n$$

Once `d` is recovered, the entire private key is compromised.

3. The impact is catastrophic: reuse of `k` leaks the ECDSA private key, enabling an attacker to forge signatures for any message and fully take over the key. This is not a theoretical weakness — it has been exploited in real incidents.

Scenario

1. Alice runs the signer that uses `create_ecdsa_signature_with_key` and (by mistake or because of a deterministic build) the nonce `k` is the constant `0x4242...42`.
2. Mallory (attacker) observes two signed transactions/messages produced by Alice: signatures `(r, s1)` for `z1` and `(r, s2)` for `z2`. Both signatures share the same `r` because `k` was reused.
3. Mallory computes the private key `d` using the relation derived from the two signatures (simple algebra shown in the conversation). With `d` recovered, Mallory can sign arbitrary message as Alice.
4. Alice loses control of the node. The trust in the entire system is under threat.

Proof of Concept

```
#[test]
fn test_fixed_k_leaks_private_key() {
    // secp256k1 context
    let secp = Secp256k1::new();

    let private_key = SecretKey::from_slice(&[0x01; 32]).expect("32-byte private
key");
    let message1 = Message::from_slice(&[0xab; 32]).expect("32-byte message");
    let message2 = Message::from_slice(&[0xcd; 32]).expect("32-byte message");

    let signing_node = SigningNode {
    id: "test-node".to_string(),
    threshold: 1,
    private_key: Some(private_key),
    public_key: Some(PublicKey::from_secret_key(&secp, &private_key)),
    public_keys: Arc::new(RwLock::new(HashMap::new())),
    signatures: Arc::new(RwLock::new(HashMap::new())),
    signatures_by_round: Arc::new(RwLock::new(HashMap::new())),
    final_nodes_by_round: Arc::new(RwLock::new(HashMap::new())),
    aggregated_rounds: Arc::new(RwLock::new(HashSet::new())),
    message_tx: tokio::sync::mpsc::channel(1).0,
    vrf_service: None,
    chain_handler:
Arc::from(crate::chain::ChainHandlerFactory::create_ethereum_handler(9029)),
    contract_chain_handler:
Arc::from(crate::chain::ChainHandlerFactory::create_ethereum_handler(9029)),
    consensus_node: None,
    available_nodes: None,
    pending_transactions: Arc::new(RwLock::new(HashMap::new())),
    pending_transactions_btc: Arc::new(RwLock::new(HashMap::new())),
    btc_sigs_by_round: Arc::new(RwLock::new(HashMap::new())),
    btc_agg_by_round: Arc::new(RwLock::new(HashMap::new())),
    btc_done_inputs_by_round: Arc::new(RwLock::new(HashMap::new())),
    group_public_key: None,
    vault_group_key: None,
    user_registry: None,
    database: None,
    broadcast_config: BroadcastConfig::default(),
    };

    let signature1 = signing_node
        .create_ecdsa_signature_with_signing_key(&message1, &private_key)
        .expect("Signature creation failed");
    let signature2 = signing_node
        .create_ecdsa_signature_with_signing_key(&message2, &private_key)
        .expect("Signature creation failed");

    // r and s
    let (recovery_id1, sig_bytes1) = signature1.serialize_compact();
```

```
    let (recovery_id2, sig_bytes2) = signature2.serialize_compact();
    let r = BigUint::from_bytes_be(&sig_bytes1[0..32]);
    let s1 = BigUint::from_bytes_be(&sig_bytes1[32..64]);
    let s2 = BigUint::from_bytes_be(&sig_bytes2[32..64]);

    // message hashes
    let z1 = BigUint::from_bytes_be(message1.as_ref());
    let z2 = BigUint::from_bytes_be(message2.as_ref());

    // mod n
    let n = BigUint::parse_bytes(
        b"FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141",
        16,
    )
    .unwrap();

    // d = (s1 * z2 - s2 * z1) / (r * (s2 - s1)) mod n
    let numerator = (&s1 * &z2 + &n - &s2 * &z1).mod_floor(&n);
    let denominator = (&r * (&s2 + &n - &s1)).mod_floor(&n);
    let denominator_inv = denominator
        .modinv(&n)
        .expect("Denominator not invertible");
    let recovered_sk = (&numerator * &denominator_inv).mod_floor(&n);
    println!("Recovered private key: {:?}", recovered_sk);

    // verify
    let original_sk = BigUint::from_bytes_be(&private_key.secret_bytes());
    println!("Original private key: {:?}", original_sk);
    assert_ne!(
        recovered_sk, original_sk,
        "Private key was successfully recovered due to repeated k!"
    );
}
```

}

Output results:

```
running 1 test
test signing::tests::test_fixed_k_leaks_private_key ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 25 filtered out;
finished in 0.00s
```

Recommendation

Never reuse `k`.Generate a fresh, unpredictable nonce for every signature. Prefer one of the following safe options:

- Use RFC6979 deterministic nonces derived from the message hash and the private key (widely recommended because it avoids RNG dependence while still producing a unique `k` per message). Most crypto libraries provide RFC6979 signing primitives.
- Use a cryptographically secure RNG (CSPRNG) seeded from high-entropy sources for each signature (ensure the RNG cannot be influenced by attacker-controlled inputs).

If deterministic nonces are used, document and test them. If you adopt RFC6979, document which hash function and procedure you use and include cross-tests against known test vectors.

Alleviation

[Qubetics, 11/05/2025]: The team heeded the advice and partially resolved the issue by generating a random k value through all nodes in commit `8a0c09991843732c5f9ab35d366cdf834d57e177` .

[Certik, 01/12/2026]: The commit `8a0c09991843732c5f9ab35d366cdf834d57e177` you provided have resolved the fixed k issue but introduced another issue:

1. The current logic (`compute_aggregated_k` / surrounding K-GEN flow) requires collecting `k_i` from **all** nodes returned by `get_available_nodes_from_manager()` and sums them to produce `k_aggregated` . This enforces an `n-of-n` participation model.
2. The problem is that `n-of-n` aggregation creates a fragile dependency: any single unavailable node prevents completion of the round, producing a system-wide signing stall. The existing code already contains primitives for node selection/leadership, but the K-GEN path does not leverage them.
3. The likely impact is reduced fault tolerance and operational outages: routine node restarts, transient network partitions, or a single faulty node can block signing; recovery requires manual intervention or waiting for timeouts, harming availability and UX.

QCA-03 | Incorrect Nonce Management Leads To Transaction Failures

Category	Severity	Location	Status
Denial of Service	Major	src/signing/mod.rs (chain-abstraction): 1030~1136	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/signing/mod.rs

The `handle_sign_request` function within the signing module is responsible for processing and signing Ethereum transactions. The function correctly implements logic to determine the appropriate sender address based on the transaction type (`UserToVault` , `NetworkToTarget` , `VaultToNetwork`). It then proceeds to fetch the correct, current nonce for that sender address from the Ethereum blockchain via the `fetch_ethereum_nonce` helper. This freshly retrieved nonce is stored in the `dynamic_nonce` variable. The vulnerability lies in the fact that this `dynamic_nonce` is never used. When the final transaction is constructed and signed via the `create_signed_transaction` utility, the function instead uses `original_tx.nonce` , which is the nonce provided in the initial request payload. The up-to-date nonce fetched from the chain is discarded, defeating the purpose of the preceding logic.

```
  //    src/signing/mod.rs

          pub async fn reconstruct_signed_ethereum_transaction() ...

               let (nonce_address, dynamic_nonce) = match transaction_type {
                    // ...
                };
                // ...
                let signed_tx =
crate::utils::transaction::create_signed_transaction(
                    &crate::rpc_server::DummyTransaction {
                        to: original_tx.to.clone(),
                        value: original_tx.value.clone(),
                        nonce: original_tx.nonce,
```

Recommendation

To remediate this vulnerability, the call to `create_signed_transaction` must be updated to use the `dynamic_nonce` variable. This ensures that every transaction is signed with the correct, up-to-date nonce fetched directly from the blockchain, making the signing process robust and reliable.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue by replacing original nonce with `dynamic_nonce` in commit b2016c77dd7e2605cb37f17857ce4c5ccd12137c

QCA-100 Concurrent Calling `set_private_key_from_scalar` Can Lead To Incorrect Private Key Settings

Category	Severity	Location	Status
Concurrency	Major	src/network/mod.rs (chain-abstraction): 1619~1630, 1840~1844	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The identified thread safety issue arises from concurrent modifications and accesses to the shared private key state in `SigningNode` when processing different message types (e.g., `GossipsubMessage::SolverReward`, `GossipsubMessage::DepositIntent`, `ChannelMessage::DepositIntent` and `ChannelMessage::SolverReward`). The current implementation risks key corruption and incorrect signatures due to unsynchronized `set_private_key_from_scalar` calls across asynchronous tasks.

The logic in `GossipsubMessage::SolverReward`:

`src/network/mod.rs`

```
1619  if let Some(dkg_node) = &self.dkg_node {
1620      if let Some(secret_share) = dkg_node.get_final_secret_share().await {
1621          info!("🔐 [REWARD] Got secret share from DKG: {:?}", secret_share);
1622          signing_node.set_private_key_from_scalar(secret_share);
1623
        info!("🔐 [REWARD] Starting solver reward signing process: 0x{}",
hex::encode(tx_bytes.clone()));
1624
        signing_node.sign_message(tx_bytes, &dummy_tx, Some(secret_share), None,
None, None).await?;
1625          info!("✅ [REWARD] Solver reward signing completed");
1626      } else {
1627
        info!("⏳ [REWARD] DKG not completed yet, no secret share available");
1628          return Ok(());
1629      }
1630  }
```

The logic in `GossipsubMessage::DepositIntent`:

`src/network/mod.rs`

```
1840  // Use the signing key we got earlier
1841      if let Some(key) = signing_key {
1842          info!("🔐 [GOSSIPSUB] Got signing key for deposit intent");
1843          signing_node.set_private_key_from_scalar(key);
```

The `SigningNode` maintains mutable private key state (`set_private_key_from_scalar`) that is shared across:

1. SolverReward: Requires the network's DKG key
2. DepositIntent: May require either user-specific keys (for `UserToVault`) or network keys (for other transaction types)

There is no synchronization exists between concurrent message processing tasks.

Race conditions during key swaps may lead to:

1. Reward transactions signed with user keys → Invalid signatures
2. Deposit transactions signed with network keys → Security violations

Recommendation

Consider passing private keys explicitly as function arguments instead of maintaining mutable state in the signing node.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue by passing private keys explicitly as function arguments in commit c1e76806b71aef3190d719479fb2c556ca986ce4.

QCA-101 Lack Of Confirming The Success Of The User's Transaction In `handle_user_mpc_deposit` Can Lead To Fund Losses

Category	Severity	Location	Status
Logical Issue	Major	src/rpc_server/mod.rs (chain-abstraction): 1455~1476	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The current implementation in `handle_user_mpc_deposit` contains a critical flaw in cross-chain transaction handling where the system may process a `NetworkToTarget` transfer before confirming the success of the prerequisite `UserToVault` transaction. This breaks atomicity guarantees and could lead to fund losses when the initial deposit fails but the subsequent transfer succeeds.

The current implementation in `handle_user_mpc_deposit`:

`src/rpc_server/mod.rs`

```
1423  if let Some(tx_ids) = user_registry
1424          .get_intent_transaction_ids(&intent_hash_hex)
1425          .await
1426      {
1427          // Only store transaction IDs if there's no error
1428          if tx_ids.error_message.is_none() {
1429
           if let Some(ref user_to_network_tx) = tx_ids.user_to_network_tx_id {
1430                  transaction_ids.push(user_to_network_tx.clone());
1431              }
1432
           if let Some(ref network_to_target_tx) = tx_ids.network_to_target_tx_id {
1433                  transaction_ids.push(network_to_target_tx.clone());
1434              }
1435          }
1436      }
1437
1438      let transaction_type = if transaction_ids.len() == 1 {
1439          crate::types::TransactionType::NetworkToTarget
1440      } else if transaction_ids.len() == 0 {
1441          crate::types::TransactionType::UserToVault
1442      } else {
1443          // More than 2 transactions indicates a duplicate intent
1444          return Ok(warp::reply::json(&UserMpcDepositVerifyResponse {
1445              status: "error".to_string(),
1446              signer_address: "".to_string(),
1447              is_registered: false,
1448              amount: 0,
1449              user_to_network_tx_id: tx_mapping
1450                  .as_ref()
1451                  .and_then(|tx| tx.user_to_network_tx_id.clone()),
1452              network_to_target_tx_id: tx_mapping
1453                  .as_ref()
1454                  .and_then(|tx| tx.network_to_target_tx_id.clone()),
1455              vault_to_network_tx_id: None,
1456              final_solver_id: None,
1457
           error_message: Some("Duplicate intent detected - this intent has already
been processed".to_string()),
1458          }));
1459      };
```

It only checks transaction existence via `transaction_ids.len()` . There is no verification of on-chain confirmation status.

Impact

1. Direct fund depletion when:

- `UserToVault` fails (e.g., due to insufficient gas or slippage)
- `NetworkToTarget` succeeds (transfers out vault funds)

2. Loss magnitude scales with transaction values

3. Irreversible once blockchain confirms the outgoing transfer

Scenario

1. Faulty Deposit Initiation

Attacker submits a `UserToVault` transaction with intentionally insufficient gas, causing it to fail in the mempool while still generating a transaction ID.

2. Immediate Withdrawal Request

Before the failed deposit is detected, attacker submits a `NetworkToTarget` request referencing the unconfirmed deposit TX ID.

3. System Validation Bypass

Bridge system only checks for TX ID existence (not confirmation status) and approves the withdrawal request.

4. Malicious Transfer Execution

System processes valid outbound transfer to attacker's target chain address, pulling funds from vault reserves.

5. Funds Acquisition

Attacker receives the cross-chain funds while the original deposit remains unconfirmed and ultimately fails.

6. Vault Balance Discrepancy

Protocol vault shows reduced balance despite never receiving the corresponding deposit, creating irreversible loss.

7. Repeat Exploitation

Attacker repeats process to drain remaining vault funds until the vulnerability is patched.

Recommendation

Consider following fix:

1. Validate Transaction Success

Strictly verify on-chain execution status (e.g., status=1 in EVM chains) before processing dependent operations, rejecting failed/reverted transactions.

2. Enforce Block Confirmations

Require sufficient block depth (e.g., 6+ blocks on Ethereum) to mitigate chain reorganizations (reorgs), ensuring transaction finality.

Alleviation

[Qubetics, 12/16/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> the hit for networktotarget transaction will come from our backend service that will first check the confirmation of that txn

[CertiK, 12/14/2025]: We still recommend implementing on-chain transaction verification rather than relying on backend checks, as this ensures atomicity and eliminates irreversible fund loss risks while maintaining compliance with blockchain security principles.

QCA-102 | All Registered Users Can Not Be Updated After DKG Is Reset

Category	Severity	Location	Status
Logical Issue	Major	src/rpc_server/mod.rs (chain-abstraction): 963~965	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The current implementation of `handle_register_user` function has a critical design flaw where resetting the DKG (Distributed Key Generation) private keys invalidates all user-associated keys without providing a mechanism to update them. This creates a cryptographic inconsistency that can break system functionality.

In DKG, the `reset()` clears all DKG key materials and generates new keys:

```
82  pub async fn reset(&mut self) {
83          if let Err(e) = self.storage.clear_shares().await {
84              warn!("Failed to clear shares: {}", e);
85          }
86          if let Err(e) = self.storage.clear_commitments().await {
87              warn!("Failed to clear commitments: {}", e);
88          }
89          if let Err(e) = self.storage.clear_validations().await {
90              warn!("Failed to clear validations: {}", e);
91          }
92          if let Err(e) = self.storage.delete_final_secret().await {
93              warn!("Failed to delete final secret: {}", e);
94          }
95          if let Err(e) = self.storage.delete_final_public().await {
96              warn!("Failed to delete final public key: {}", e);
97          }
98
99          self.status = DKGStatus::New;
100          self.round += 1;
101      }
```

In `handle_register_user` function, it will call `user_registry.register_user()`:

`src/rpc_server/mod.rs`

```
963 match user_registry
964         .register_user(&addr_cs, dkg_secret_share.as_ref(), &node_id)
965         .await
966     {
```

In `register_user` function, it returns `Err` when the user exists:

```
161         // 2) Dedup on canonical key
162         {
163             let users = self.users.read().await;
164             if users.contains_key(&addr_cs) {
165                 return Err("User already registered".to_string());
166             }
167         }
```

That means the user's `tweaked_secret_share` becomes immutable after registration. There is no API exists to refresh user keys after DKG reset.

Impact:

1. Cryptographic operations fail for existing users after DKG reset
2. System enters inconsistent state where:

- New users get valid keys
- Existing users have stale keys

3. No recovery path except manual database intervention

Recommendation

Consider providing a mechanism to update registered users.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by implementing a refresh mechanism in commit 94b97730ac62b858126080cb850159a6cffab67f.

QCA-103 Parallel Execution Of Signature Generation And Request Leads To Missing Signature Data And Transaction Failure

Category	Severity	Location	Status
Design Issue	Major	src/network/mod.rs (chain-abstraction); src/signing/mod.rs (chain-abstraction): 3099~3111, 4376~4443	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`
- `src/signing/mod.rs`

```
4376      pub async fn send_btc_signatures_to_final_node(
4377          &mut self,
4378          final_node_id: &str,
4379          round: u64,
4380      ) -> Result<()> {
4381          info!(
4382
          "📤 [SIGNING] Sending BTC per-input signatures for round {} to final
node {}",
4383              round, final_node_id
4384          );
4385
4386          // Fetch our stored per-input signatures for this round
4387          let maybe_sigs = {
4388              let map = self.btc_sigs_by_round.read().await;
4389              if let Some(rmap) = map.get(&round) {
4390                  let mut all_sigs = Vec::new();
4391                  for (input_idx, imap) in rmap {
4392                      for (peer_id, ecsig) in imap {
4393                          all_sigs.push(BtcIndexedSignature {
4394                              input_index: *input_idx,
4395                              signature: ecsig.clone(),
4396                          });
4397                      }
4398                  }
4399                  Some(all_sigs)
4400              } else {
4401                  None
4402              }
4403          };
4404
4405          if let Some(signatures) = maybe_sigs {
4406              let signature_msg = SigningMessage::BtcSignaturesMessage {
4407                  from: self.id.clone(),
4408                  signatures,
4409                  round,
4410
               timestamp:
SystemTime::now().duration_since(UNIX_EPOCH)?.as_millis(),
4411                  user_eth_address: None,
4412              };
4413
4414              // Wrap in GossipsubMessage::Signing for proper message handling
4415
          let gossipsub_msg =
crate::types::GossipsubMessage::Signing(signature_msg);
4416              let signature_data = serde_json::to_vec(&gossipsub_msg)?;
4417
4418
          // Broadcast to a specific topic that the final node subscribes to
4419              let res: std::result::Result<(), anyhow::Error> = self
4420                  .broadcast("signatures-to-final-node", &signature_data)
```

```
4421                 .await;
4422             match res {
4423                 Ok(_) => info!(
4424 
                  "📤 [SIGNING] Successfully sent BTC signatures for round {} to
final node {}",
4425                     round, final_node_id
4426                 ),
4427                 Err(e) => {
4428                     error!(
4429 
                      "❌ [SIGNING] Failed to send BTC signatures for round {} to
final node {}: {}",
4430                         round, final_node_id, e
4431                     );
4432                     return Err(e);
4433                 }
4434             }
4435         } else {
4436             info!(
4437 
               "⚠️ [SIGNING] No BTC per-input signatures found for round {} for
this node",
4438                 round
4439             );
4440         }
4441 
4442         Ok(())
4443     }
```

In the `sign_bitcoin_transaction` workflow, the signature process involves several sequential steps: (1) computing the sighash, (2) generating a VRF proof, (3) waiting for all nodes' VRF proofs, (4) selecting and broadcasting the final node, (5) generating and broadcasting the nonce k, (6) waiting for all k nonces, (7) signing, and (8) storing signature data in `btc_sigs_by_round`. The core issue is that after step 4, two processes run in parallel:

- The current node (possibly the final node) continues with signature generation and data storage.
- All nodes, upon receiving the final node election broadcast, store the result and the final node immediately requests signatures from others.

The final node's request for signatures depends on the data in `btc_sigs_by_round`, which is only available after the signature generation process completes. Since these two processes are parallel, the request for signatures may occur before the signature data is ready, leading to missing data and failed signature submission.

Impact

In the function `trigger_aggregate_signature_for_final_node`, it may have misjudged the transaction type. Because the signature here may not be due to a type mismatch, but rather because the signing process has not yet been completed.

```
3099          let signature_request = if is_btc_round {
3100              SigningMessage::RequestBtcSignatures {
3101                  round,
3102                  final_node_id: self.id.clone(),
3103                  message: "Aggregate BTC signatures request".to_string(),
3104              }
3105          } else {
3106              SigningMessage::RequestSignatures {
3107                  round,
3108                  final_node_id: self.id.clone(),
3109                  message: "Aggregate signature request".to_string(),
3110              }
3111          };
```

If the final node requests signatures before other nodes have finished generating and storing their signatures in `btc_sigs_by_round`, those nodes will fail to find the required data and cannot respond. This results in the final node being unable to collect enough signatures to reach the threshold, causing the transaction aggregation to fail. The transaction may never be broadcast to the network, leading to permanent transaction loss and a degraded user experience.

Scenario

1. Alice, Bob, and Carol participate in a signing round.
2. After VRF consensus, Alice is selected as the final node and broadcasts this result (step 4).
3. Alice continues with signature generation and stores data in `btc_sigs_by_round` (steps 5-8).
4. Simultaneously, Alice requests signatures from Bob and Carol.
5. Bob and Carol receive the request but have not yet completed their own signature generation and data storage.
6. Bob and Carol fail to find their signatures in `btc_sigs_by_round` and cannot respond.
7. Alice cannot collect enough signatures, and the transaction fails to aggregate and broadcast.

Recommendation

Introduce synchronization between the signature generation process and the signature request/response process. Ensure that nodes only respond to signature requests after their signature data is fully generated and stored in `btc_sigs_by_round`. This can be achieved by adding a retry mechanism or awaiting the completion of signature generation before responding to requests. Alternatively, design the protocol so that nodes proactively send their signatures to the final node as soon as they are ready, decoupling the request/response dependency and eliminating the race condition.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue in commit 0ff7ca611c8728ea8d63536c8b3977423e23c006.

QCA-104 Persistent Error State In Transaction Type Judgment Leads To Incorrect Routing And Potential Fund Loss

Category	Severity	Location	Status
Design Issue	Major	src/rpc_server/mod.rs (chain-abstraction): 1426~1467	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

```
1426      {
1427          // Only store transaction IDs if there's no error
1428          if tx_ids.error_message.is_none() {
1429
           if let Some(ref user_to_network_tx) = tx_ids.user_to_network_tx_id {
1430                  transaction_ids.push(user_to_network_tx.clone());
1431              }
1432
           if let Some(ref network_to_target_tx) = tx_ids.network_to_target_tx_id {
1433                  transaction_ids.push(network_to_target_tx.clone());
1434              }
1435          }
1436      }
1437      // Process the deposit intent
1438
    // If there are any transaction IDs, req.tx_id must be provided, else return
error
1439      // if transaction_ids.len() >= 1 && req.tx_id.is_none() {
1440      //     return Ok(warp::reply::json(&UserMpcDepositVerifyResponse {
1441      //         status: "error".to_string(),
1442      //         signer_address: "".to_string(),
1443      //         is_registered: false,
1444      //         amount: 0,
1445      //         user_to_network_tx_id: None,
1446      //         network_to_target_tx_id: None,
1447      //         vault_to_network_tx_id: None,
1448      //         error_message: Some(
1449
    //             "Transaction ID must be provided when transaction(s) already
exist for this intent"
1450      //                 .to_string(),
1451      //         ),
1452      //     }));
1453      // }
1454
1455      let transaction_type = if transaction_ids.len() == 1 {
1456          crate::types::TransactionType::NetworkToTarget
1457      } else if transaction_ids.len() == 0 {
1458          crate::types::TransactionType::UserToVault
1459      } else {
1460          // More than 2 transactions indicates a duplicate intent
1461          return Ok(warp::reply::json(&UserMpcDepositVerifyResponse {
1462              status: "error".to_string(),
1463              signer_address: "".to_string(),
1464              is_registered: false,
1465              amount: 0,
1466              user_to_network_tx_id: tx_mapping
1467                  .as_ref()
```

The code determines the transaction type based on the number of transaction IDs: the first call is treated as "UserToVault", and the second as "NetworkToTarget". This is the intended flow—users first deposit funds to the network vault, then the network transfers to the target chain. However, the logic includes a check: if `tx_ids.error_message.is_none()`,

transaction IDs are added. The `error_message` is only set when a transaction fails on-chain and there is no mechanism to clear this error. As a result, if the first transaction fails, the error is stored. On subsequent calls, this check still triggers the "UserToVault" type, even if the next transaction succeeds. Since the error is never cleared, all future calls will continue to treat the transaction as the first type, regardless of actual progress. This affects address selection and routing logic, potentially causing funds to become stuck or lost.

Impact

If the error state is not cleared after a failed transaction, the system will misclassify subsequent transactions, always treating them as the initial "UserToVault" type. This can lead to incorrect address selection, repeated or invalid transaction attempts, and ultimately, funds being locked or lost due to misrouted transactions. The user may be unable to complete the intended cross-chain transfer, and recovery may be impossible without manual intervention.

Scenario

1. Alice initiates a deposit (UserToVault), but the transaction fails and an error is stored in `tx_ids.error_message`.
2. Alice retries the deposit, and this time the transaction succeeds, but the error is still present.
3. On the next call, the system still treats the transaction as "UserToVault" due to the persistent error, instead of progressing to "NetworkToTarget".
4. The address selection and routing logic are based on the wrong transaction type, causing funds to be sent to the wrong address or become unrecoverable.

Recommendation

Implement a mechanism to clear or update the `error_message` field once a transaction succeeds or is retried. Ensure that the transaction type judgment accurately reflects the current state, not just the presence of a past error. Consider using explicit transaction status tracking (e.g., confirmed, failed, retried) rather than relying solely on error messages. This will prevent persistent error states from causing incorrect routing and potential fund loss.

Alleviation

[Qubetics, 12/20/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> if usertovault txn fails , without having a success usertovault txn , backend will never initiate networktotarget txn. networktotarget txn are initiated by backend.

QCA-105 Vault-To-Network Adds 10% Uplift Will Drain Vault Funds During Rebalancing

Category	Severity	Location	Status
Logical Issue	Major	src/rpc_server/mod.rs (chain-abstraction): 772~775	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The `handle_deposit_intent` function applies a 10% uplift to all non-`NetworkToTarget` flows, which includes `VaultToNetwork`. That path is typically a vault rebalance/top-up, but it still sends `intent.amount * 1.1` out of the vault. Solver rewards are only calculated on `UserToVault`, so this uplift is not tied to fresh rewards and simply over-withdraws vault assets.

```
//728:776:src/rpc_server/mod.rs
if matches!(transaction_type, crate::types::TransactionType::NetworkToTarget) {
    …
} else {
    // For VaultToNetwork and other types, use the intent amount plus 10%
    let base_amount = intent.amount + (intent.amount / 10);
    required_amount_u128 = base_amount;
}
```

Rewards are computed as 10% of the original intent amount and are triggered only for `UserToVault`, not for `VaultToNetwork`, so the extra 10% on `VaultToNetwork` isn't even aligned with reward attribution.

```
4247:4356:src/signing/mod.rs
let intent_amount = EthereumU256::from(intent.amount);
let base_reward = intent_amount / EthereumU256::from(10u8); // 10% of intent amount
…
if let Some(signing_node) = &self.signing_node {
    // reward calculation is only triggered for UserToVault
    if let Err(e) =
signing_node.update_solver_amounts_for_user_to_vault(&intent_hash).await { … }
    else if let Err(e) =
signing_node.calculate_reward_per_solver(&intent_hash).await { … }
}
```

Recommendation

Remove the 10% uplift for `VaultToNetwork` (apply it only to `UserToVault` where users fund fees), or explicitly split fee vs. principal so rebalance/send-back flows transmit only the owed principal. If `VaultToNetwork` must ever fund rewards, add an explicit fee input and align reward calculation to that leg; otherwise keep vault rebalancing zero-fee to avoid vault erosion.

Alleviation

[Qubetics, 12/19/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> Issue acknowledged. I won't make any changes for the current version.
>
> Its the part of platform fees kind of thing and will be mentioned on our platform UI

QCA-106 Threshold Change Without Atomic DKG Transaction Leads To Signature Failure And Inconsistent Participation

Category	Severity	Location	Status
Design Issue	Major	src/network/mod.rs (chain-abstraction)	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

DKG Protocol and Threshold Management

In a threshold cryptography system, the Distributed Key Generation (DKG) protocol is responsible for securely generating shared keys and setting the threshold for signature aggregation. Proper DKG execution is critical whenever the set of participating nodes changes (node join/leave), as both the key shares and the threshold must be updated atomically to maintain security and correctness.

Problems in Current Implementation

1. **Non-Atomic Threshold Update**
   The code updates the threshold value immediately after a node joins, before DKG is executed. If DKG subsequently fails, the threshold remains increased, but the key shares are not updated. This leads to a mismatch between the threshold and the actual cryptographic state, causing signature aggregation to fail and potentially halting the network's ability to process transactions.

2. **Premature Participation of New Nodes**
   After a node joins and the threshold is updated, there is a window before DKG completes where the new node should not participate in signing. However, the code does not prevent the new node from being included in signature rounds during this period, risking invalid or incomplete signatures.

3. **No Handling for Node Departure**
   When a node leaves, the threshold and DKG are not updated. This can result in the network attempting to aggregate signatures from nodes that are no longer present, causing signature rounds to stall or fail.

Impact

- If DKG fails after a threshold increase, the network is left in an inconsistent state, unable to reach the required threshold for valid signatures.
- Transactions initiated during the DKG window may include nodes that do not have valid key shares, leading to failed or insecure signatures.
- Node departures are not handled, so the network may wait indefinitely for signatures from missing nodes, causing transaction delays or permanent failures.

Scenario

1. Bob joins the network, triggering a threshold increase and DKG start.
2. The threshold is updated before DKG completes. If DKG fails, the threshold remains higher, but key shares are not updated.
3. During the DKG window, a transaction is initiated. The new node (Bob) is included in the signing set, but does not have valid key shares yet.
4. The signature round fails due to missing or invalid shares.
5. Later, Carol leaves the network, but the threshold and DKG are not updated. The network continues to expect Carol's participation, causing future signature rounds to stall.

Recommendation

- **Atomic Threshold and DKG Update:** Only update the threshold after DKG completes successfully. Roll back any threshold changes if DKG fails.
- **Participation Control:** Prevent new nodes from participating in signature rounds until DKG has completed and key shares are distributed.
- **Handle Node Departure:** On node leave, immediately trigger a new DKG and threshold update to exclude the departed node and maintain a consistent signing set.
- **Transactional Approach:** Treat threshold and DKG changes as a single atomic transaction to ensure network consistency and prevent signature failures.

This will ensure that the threshold and key shares are always in sync, and only valid nodes participate in signing, maintaining the security and reliability of the network.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143.

QCA-15 The `reconstruct_signed_bitcoin_transaction` Function Incorrectly Retrieves BTC Transactions From `pending_transactions`

Category	Severity	Location	Status
Logical Issue	Major	src/signing/mod.rs (chain-abstraction): 908~910	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `sign_bitcoin_transaction` function stores Bitcoin transactions in `pending_transactions_btc`, but `reconstruct_signed_bitcoin_transaction` incorrectly retrieves them from `pending_transactions`. This inconsistency can lead to missing transaction data (e.g., BtcRoundExtras) and failed transaction reconstruction, potentially causing invalid Bitcoin transactions.

The `sign_bitcoin_transaction` stores Bitcoin transactions in `pending_transactions_btc`:

`src/signing/mod.rs`

```
533 let mut pending = self.pending_transactions_btc.write().await;
534         pending.insert(
535             current_round,
536             (
537                 ChainTransaction::Bitcoin(original_tx.clone()),
538                 tx_bytes.clone(),
539                 intent_hash.clone(),
540                 transaction_type.clone(),
541                 Some(extras), // <-- critical for aggregation/finalization
542             ),
543         );
```

However, the `reconstruct_signed_bitcoin_transaction` retrieves them from `pending_transactions`:

`src/signing/mod.rs`

```
902  pub async fn reconstruct_signed_bitcoin_transaction(
903          &self,
904          round: u64,
905          aggregated_signature: &[u8],
906          _recovery_id: u8,
907      ) -> Result<Option<String>> {
908          let pending_transactions = self.pending_transactions.read().await;
909
910
        if let Some((chain_tx, _tx_bytes, _, transaction_type)) =
pending_transactions.get(&round) {
911              if let ChainTransaction::Bitcoin(original_tx) = chain_tx {
912              ...
913              ...
```

Here the `chain_tx` will never be `ChainTransaction::Bitcoin`. As a result, the reconstruction will always fail.

Recommendation

Consider following fix:

```
let pending= self.pending_transactions_btc.read().await;

if let Some((chain_tx, _tx_bytes, _, transaction_type)) = pending.get(&round) {
...
...
```

Alleviation

[Qubetics, 12/08/2025]: The team heeded the advice and resolved the issue in commit 282ab70e8ff7cdff5143da8b0cfc80fc0cfca092.

QCA-17 | `select_mpc_nodes` Function Fabricates VRF Submissions

Category	Severity	Location	Status
Volatile Code	Major	src/vrf/mod.rs (chain-abstraction): 311~323	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/vrf/mod.rs

`select_mpc_nodes` fabricates VRF submissions for every peer instead of using (or verifying) proofs actually signed by those peers, so node selection no longer reflects cryptographic randomness or authenticity. `generate_selection_proof` uses the local node's secret key and the same input each time, so the loop inserts identical, locally-signed VRF outputs for every peer. The resulting NodeSelectionResult is entirely controlled by the node executing this code.

```
        // Simulate collecting VRF submissions from all nodes
        let mut all_submissions = HashMap::new();
        for node in available_nodes {
            // In a real implementation, you would receive actual VRF proofs from
other nodes
            // For now, we'll simulate by generating proofs for all nodes
            let simulated_proof = self.generate_selection_proof(round,
available_nodes);
            all_submissions.insert(SerializablePeerId(*node), simulated_proof);
        }
```

Any node running this path can unilaterally decide (or later forge) which peers are "selected", defeating Sybil resistance and fairness guarantees promised by the VRF-based MPC selection flow. Peers receiving the broadcast cannot verify the proofs because they were never produced by the claimed key holders.

Recommendation

Remove the simulation stub and use the local cached submissions

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by using local cached submissions in commit 3aab22bf177592ac47fe1996ab7b91b370092569.

QCA-18 | Missing Signature/Proof Verification On VrfSubmission Messages

Category	Severity	Location	Status
Volatile Code	Major	src/vrf/mod.rs (chain-abstraction): 396~411	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/vrf/mod.rs

The VrfService is responsible for handling VRF-related messages for node selection. One of the message types is `VrfSelectionMessage::VrfSubmission`. These messages are handled by the `VrfNodeSelector::handle_message` function, which dispatches to `handle_vrf_submission`.

```
    async fn handle_vrf_submission(
        &mut self,
        node_id: PeerId,
        vrf_output: VrfOutput,
        round: u64,
    ) -> Result<()> {
        debug!("Received VRF submission from node {} for round {}", node_id, round);

        self.submissions
            .entry(round)
            .or_insert_with(HashMap::new)
            .insert(node_id, vrf_output);

        Ok(())
    }
```

As shown above, the `handle_vrf_submission` function directly inserts the received `vrf_output` into the submissions map, keyed by `round` and `node_id`. The `vrf_output` contains a cryptographic proof (`VrfOutput.proof`) that is intended to prove the message's authenticity and correctness. However, this proof is never verified.

The module contains a function `verify_vrf_proof` for this purpose, but it is not called when processing `VrfSubmission` messages.

The same problem may also affect `handle_selection_result` function.

Recommendation

Verify the authenticity and correctness of every VrfSubmission message before it is processed and stored.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by verifying submission in `handle_vrf_submission` in commit 60a12d70dc87ee166907e7cd3833dbd3447c8155.

QCA-34 The Logic Of `get_ethereum_address` To Parse An Ethereum Address From A Public Key Is Incorrect

Category	Severity	Location	Status
Logical Issue	Major	src/signing/mod.rs (chain-abstraction): 4028~4051	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `get_ethereum_address` function attempts to derive an Ethereum address from a public key, but it contains a critical flaw: it directly extracts the first 20 bytes of the public key (`x` coordinate) instead of computing the correct Keccak-256 hash and taking the last 20 bytes.

The current implementation:

`src/signing/mod.rs`

```
4034            if public_key_bytes.len() >= 21 {
4035                let address_bytes = &public_key_bytes[1..21];
4036
4037                // Convert to Ethereum address format (0x + hex)
4038                let address = format!("0x{}", hex::encode(address_bytes));
4039
4040                info!(
4041                    "🏦 [SIGNING] Node {} Ethereum address: {}",
4042                    self.id, address
4043                );
4044                Some(address)
4045            } else {
4046                None
4047            }
```

Ethereum addresses are not derived from raw public key bytes. The correct method involves:

1. Hashing the x and y coordinates (excluding the 0x04 prefix) with Keccak-256.

2. Taking the last 20 bytes of the hash.

The current implementation is incorrect and will return wrong ethereum addrees.

Recommendation

Consider following fix:

```
use tiny_keccak::{Keccak, Hasher};

pub fn get_ethereum_address(&self) -> Option<String> {
    if let Some(public_key) = &self.public_key {
        // 1. Get uncompressed public key (65 bytes: 0x04 || x || y)
        let public_key_bytes = public_key.serialize_uncompressed();

        // 2. Verify length (must be 65 bytes)
        if public_key_bytes.len() == 65 {
            // 3. Compute Keccak-256(x || y) (skip 0x04 prefix)
            let mut hasher = Keccak::v256();
            hasher.update(&public_key_bytes[1..]); // Hash 64 bytes (x || y)
            let mut hash = [0u8; 32];
            hasher.finalize(&mut hash);

            // 4. Take last 20 bytes as the address
            let address_bytes = &hash[hash.len()-20..];

            // 5. Format as 0x + hex string
            let address = format!("0x{}", hex::encode(address_bytes));

            info!("🏦 [SIGNING] Node {} Ethereum address: {}", self.id, address);
            Some(address)
        } else {
            None // Invalid key length
        }
    } else {
        None // No public key
    }
}
```

Alleviation

[Qubetics, 11/26/2025]: The team heeded the advice and resolved the issue in commit 72df95abc066e86c89b9c09b4188f02cd38a97a7.

QCA-35 Nodes Can't Receive `ConsensusMessage::FinalNodeResult` Message Because This Message Lacks Of `GossipsubMessage::Consensus` Wrapping

Category	Severity	Location	Status
Logical Issue	Major	src/network/mod.rs (chain-abstraction): 1244~1256; src/signing/mod.rs (chain-abstraction): 2992~3004	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`
- `src/network/mod.rs`

The `start_consensus_for_round` function in the signing module incorrectly broadcasts a `ConsensusMessage::FinalNodeResult` without wrapping it in the expected `GossipsubMessage::Consensus` envelope. This mismatch prevents receiving nodes from properly parsing the message, as they expect a two-layer message structure (`GossipsubMessage` containing `ConsensusMessage`).

In `start_consensus_for_round`, the function constructs a `ConsensusMessage::FinalNodeResult` and directly serializes it for broadcast:

`src/signing/mod.rs`

```
2992                    // Broadcast the consensus result
2993                    let consensus_msg = ConsensusMessage::FinalNodeResult {
2994                        result: result.clone(),
2995                        round,
2996                    };
2997
2998                    let broadcast_data = serde_json::to_vec(&consensus_msg)?;
2999
3000                    // Store the result for later use
3001
                let is_final_node = consensus_node.is_final_node(round,
&own_node_id);
3002
3003                    // Broadcast outside of the borrow
3004
                self.broadcast("consensus-result", &broadcast_data).await?;
```

The message is then broadcast without the outer `GossipsubMessage::Consensus` wrapper.

In `handle_gossipsub_message` function in the network module, it first attempts to deserialize the message as a `GossipsubMessage::Consensus`:

src/network/mod.rs

```
1244  GossipsubMessage::Consensus(consensus_msg) => {
1245
            info!("📥 [CONSENSUS] Received consensus message: {:?}",
consensus_msg);
1246
1247                // Handle consensus message in signing node
1248                if let Some(signing_node) = &mut self.signing_node {
1249
signing_node.handle_consensus_message(consensus_msg.clone()).await?;
1250                }
1251
1252                // Also handle in standalone consensus node if available
1253                if let Some(consensus_node) = &mut self.consensus_node {
1254                    consensus_node.handle_message(consensus_msg).await?;
1255                }
1256            }
```

Only after this outer layer is parsed does it extract the inner `ConsensusMessage`. Since the sender does not wrap the `ConsensusMessage` in `GossipsubMessage::Consensus`, the receiver fails to deserialize the message correctly, leading to message processing errors.

Impact

1. Message Loss: Consensus results will not be properly received by other nodes, breaking the expected flow of the protocol.

2. Silent Failure: The error may not be immediately obvious because the receiving node simply skips the message (if deserialization fails) or logs an error without taking corrective action.
3. Consensus Disruption: If nodes cannot reliably exchange final node results, the consensus mechanism may fail to reach agreement, affecting system reliability.

Recommendation

Modify the broadcasting logic to wrap the `ConsensusMessage` in `GossipsubMessage::Consensus` before serialization, for example:

```
let consensus_msg = ConsensusMessage::FinalNodeResult { result: result.clone(),
round };
let gossipsub_msg = GossipsubMessage::Consensus(consensus_msg); // Add outer wrapper
let broadcast_data = serde_json::to_vec(&gossipsub_msg)?; // Serialize properly
```

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by wrapping in `GossipsubMessage::Consensus` for proper deserialization in commit c1e76806b71aef3190d719479fb2c556ca986ce4.

QCA-36 | DKG Share Index Mismatch

Category	Severity	Location	Status
Logical Issue	Major	src/network/mod.rs (chain-abstraction): 826~849	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/network/mod.rs

In `RPCRequst::SendAVSSShare()`, the fallback path hashes the `receiver_id` and clamps it to `1..=100`, which rarely matches the true evaluation point used when the dealer generated the share, causing valid shares to be rejected.

```
//src/network/mod.rs 831:841
let share_index = if let Ok(index) = receiver_id.0.to_string().parse::<usize>() {
    index
} else {
    let mut hasher = DefaultHasher::new();
    receiver_id.0.to_string().hash(&mut hasher);
    let hash = hasher.finish();
    ((hash % 100) + 1) as usize
};
```

Share generation is performed over deterministic indices `1..=total_nodes`, so any mismatch breaks verification.

```
//src/adkg_secp256k1/mod.rs 256:
for i in 1..=self.total_nodes {
    let x = Scalar::from(i as u64);
    // ...
    shares.push(KeyShare { index: i, value: SerializableScalar(y) });
}
```

node index when distributing are from `get_node_index`:

```
// src/adkg_secp256k1/mod.rs
    async fn handle_share_distribution(
        &mut self,
        from: String,
        shares: Vec<KeyShare>,
        commitments: Vec<SerializablePoint>,
        node_manager: Box<dyn NodeMembership>,
    ) -> Result<()> {
        let raw_commitments: Vec<AffinePoint> = commitments.into_iter().map(|c|
c.0).collect();

        let node_index = self.get_node_index(node_manager.clone());
// ...
```

Recommendation

- Transmit the intended share index explicitly alongside the share (e.g., extend `SendAVSSShare` to include `share_index`) and trust that value during verification.
- Alternatively, derive the index deterministically the same way as the dealer (e.g., reuse the sorted-node approach from `get_node_index`) instead of hashing the peer ID.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue in commit f0f05f8de0c632b00a80f9b670e70475a44c61e3

QCA-37 Concurrent Aggregation Race Leads To Duplicate Aggregation And Boardcast

Category	Severity	Location	Status
Design Issue	Major	src/signing/mod.rs (chain-abstraction): 3641~3920	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

1. The function `handle_btc_signature_for_round` is intended to ingest per-input signatures, store them under `(round → input_index → peer)`, aggregate any input that has reached `threshold`, record the aggregated signature in `btc_agg_by_round`, mark the input done in `btc_done_inputs_by_round`, and finalize & broadcast the transaction once all inputs are aggregated.

2. The implementation performs multiple non-atomic read/modify/write steps across shared maps (`btc_sigs_by_round`, `btc_done_inputs_by_round`, `btc_agg_by_round`, `aggregated_rounds`) using separate short locks. There is no per-input test-and-set or processing claim: two concurrent tasks can both observe that an input has reached the `threshold` and then both proceed to aggregate the same input concurrently. The code first reads `peer_map.len()` under a read lock, then later performs aggregation and writes `btc_agg_by_round` and `btc_done_inputs_by_round` under separate write locks, leaving a race window where multiple tasks may perform the same aggregation work or interleave writes.

3. The impact includes duplicate aggregation work, conflicting writes to `btc_agg_by_round` / `btc_done_inputs_by_round`, possible wasted CPU/network resources, and nondeterministic timing for finalization. In some interleavings this can also lead to missed finalization triggers or inconsistent state if aggregation writes and done-marks interleave poorly, causing operational instability and hard-to-reproduce signing failures.

Scenario

One of possible scenarios:

1. time1: Two worker tasks (A and B) concurrently enter `handle_btc_signature_for_round` for the same `round = R` and `input_index = i`. Both tasks read `peer_map` from `btc_sigs_by_round` and observe `peer_map.len() >= self.threshold`. Neither task yet holds any processing claim for input `i`.

2. time2: Both A and B proceed to select the same deterministic subset, build `sigs`, call `aggregate_signatures(...)` and perform the POST-AGGREGATION verification.
3. time3: Both A and B write the aggregated signature into `btc_agg_by_round` and insert `i` into `btc_done_inputs_by_round`. Because writes are not serialized per-input, the two writes race and may overwrite each other or create transient inconsistencies. If both then attempt to finalize the round, this may trigger duplicate broadcast attempts or conflicting storage updates.

Recommendation

1. **Per-Input Atomic Insert-and-Trigger:** When inserting a signature into `btc_sigs_by_round`, acquire the write lock for that `round` (or a per-input lock), update the `peer_map`, and immediately check `peer_map.len()` while still holding the lock. If the threshold is reached and the input is not already in `btc_done_inputs_by_round`, atomically mark it done and either perform aggregation under the same lock or enqueue a work item for a single aggregator to process. This removes the race window between detection and aggregation.

2. **Introduce Per-Input Processing Claim / Mutex:** Maintain a `processing` map or per-`(round,input)` `Mutex` so only one task can claim processing for a given input. Implement a CAS-style test-and-set: if a claim is successful the task proceeds to aggregate; otherwise it backs off. This ensures only one aggregator runs per input.

Alleviation

[Qubetics, 12/29/2025]: The team heeded the advice and resolved the issue by making signing without waiting for nonce k consensus in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143

QCA-51 Local Round Assignment Cannot Handle P2P Message Delay, Causing Transaction–Signature Mismatch

Category	Severity	Location	Status
Design Issue	Major	src/signing/mod.rs (chain-abstraction): 494~508, 532~549, 3086~3124; src/vrf/mod.rs (chain-abstraction): 170~175	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/signing/mod.rs`
- `src/vrf/mod.rs`

1. The signing flow computes a per-signing `round` using `get_next_round()` (a local, self-incrementing counter) and then stores signatures and the unsigned transaction under that `round` key (e.g. `pending_transactions_btc.insert(current_round, (...))`). The code also writes per-input signatures into `btc_sigs_by_round[current_round][input_index][peer] = signature`.

```
170     pub async fn get_next_round(&self) -> u64 {
171         let mut current = self.current_round.write().await;
172         let round = *current;
173         *current += 1;
174         round
175     }
```

2. This is how node store the sigs and tx:

```
    {
        let mut by_round = self.btc_sigs_by_round.write().await;
        let rmap = by_round.entry(current_round).or_insert_with(HashMap::new);
        for btc_sig in &btc_sigs {
            let imap = rmap
                .entry(btc_sig.input_index)
                .or_insert_with(HashMap::new);
            imap.insert(self.id.clone(), btc_sig.signature.clone());
        }
        info!(
            "💾 [SIGNING] Stored {} Bitcoin input signatures for round {}",
            btc_sigs.len(),
            current_round
        );
    }
```

```
    if let Some(original_tx) = &original_tx {
        let mut pending = self.pending_transactions_btc.write().await;
        pending.insert(
            current_round,
            (
                ChainTransaction::Bitcoin(original_tx.clone()),
                tx_bytes.clone(),
                intent_hash.clone(),
                transaction_type.clone(),
                Some(extras), // <-- critical for aggregation/finalization
            ),
        );
        info!(

      "💾 [SIGNING] Stored original BTC tx + extras for round {} (inputs={})",
            current_round,
            original_tx.inputs.len()
        );
    }
```

The problem is that `round` is assigned locally and depends on the order each node processes incoming transactions. In a P2P network messages arrive out-of-order and at different times on different nodes, so there is no global guarantee that `round` ↔ `transaction` mapping is consistent across the cluster. Using a local auto-increment as the primary storage key breaks the canonical link between signatures and the transaction they sign.

3. This is how nodes do sig request:

```
3086      /// Trigger aggregate signature for the final selected node
3087
    async fn trigger_aggregate_signature_for_final_node(&self, round: u64) ->
Result<()> {
3088          info!(
3089
            "🚀 [SIGNING] Triggering aggregate signature for final node in round
{}",
3090              round
3091          );
3092
3093
        // If we have BTC per-input signatures tracked for this round, request BTC
signatures
3094          let is_btc_round = {
3095              let map = self.btc_sigs_by_round.read().await;
3096              map.contains_key(&round)
3097          };
3098
3099          let signature_request = if is_btc_round {
3100              SigningMessage::RequestBtcSignatures {
3101                  round,
3102                  final_node_id: self.id.clone(),
3103                  message: "Aggregate BTC signatures request".to_string(),
3104              }
3105          } else {
3106              SigningMessage::RequestSignatures {
3107                  round,
3108                  final_node_id: self.id.clone(),
3109                  message: "Aggregate signature request".to_string(),
3110              }
3111          };
3112
3113          // Wrap in GossipsubMessage::Signing for proper message handling
3114
        let gossipsub_msg =
crate::types::GossipsubMessage::Signing(signature_request);
3115          let broadcast_data = serde_json::to_vec(&gossipsub_msg)?;
3116          self.broadcast("signing", &broadcast_data).await?;
3117
3118          info!(
3119
            "📤 [SIGNING] Broadcasted {} signature request for round {} to all
nodes",
3120              if is_btc_round { "BTC" } else { "ECDSA" }, round
3121          );
3122
3123          Ok(())
3124      }
```

This can cause signatures to be stored against the wrong transaction or to be impossible to aggregate: finalizers that expect signatures under a given `round` may not find the corresponding signatures from other nodes, signatures may be orphaned

or collide, aggregation fails, rounds may be replayed or mismatched, and the signing protocol can deadlock or accept incorrect/unauthorized signatures—creating availability, correctness, and security risks.

Impact

1. The system becomes incapable of signing the affected transaction AND all later transactions

When a mismatch occurs:

- Node A stores TX1 signatures under round 5
- Node B stores TX1 signatures under round 8
- Finalizer cannot gather enough signatures for either round
- TX1 never completes
- TX2, TX3… cannot progress because round numbering is already diverged

Result: the signing pipeline deadlocks permanently.

2. The system cannot self-recover — it remains permanently broken

This issue is not self-healing because:

- Nodes will continue incrementing rounds independently
- The round → transaction mapping remains inconsistent
- No protocol exists to reconcile or resynchronize rounds
- Nodes keep producing signatures under different round keys

Only manual administrator intervention can reset state. Without an admin override, the system is permanently bricked after the first mismatch.

3. This failure is likely to happen — not a corner case This is not theoretical. The conditions occur frequently in any decentralized P2P system:

- Gossip message delay is common
- Network jitter happens regularly
- Nodes experience different load conditions
- Users send multiple signing requests concurrently

As the number of signing requests increases, the probability of different nodes receiving them in different orders approaches certainty. The system design assumes a synchronized ordering that P2P networks cannot guarantee, making this failure mode extremely realistic.

Scenario

One of possible scenario:

1. Alice submits `DepositIntentA` and Bob submits `DepositIntentB` at nearly the same time.

2. Node-1 receives `A` then `B` and assigns `round = 7` to `A` and `round = 8` to `B`. Node-2 receives `B` then `A` and assigns `round = 7` to `B` and `round = 8` to `A`.
3. Node-1 stores signatures for `A` under round `7`; Node-2 stores signatures for `A` under round `8`. The finalizer requests signatures for round `7` expecting `A`, but some nodes have `B` under `7` (or no entry for `A`), so aggregation fails and the transaction cannot be finalized or is finalized with the wrong inputs.

Recommendation

Use a **transaction-unique identifier** as the primary key for storing and aggregating signatures rather than a local auto-increment counter. Concrete fixes (pick one or combine):

- **Key by `intent_hash` / `tx_hash`**: store signatures and extras under `intent_hash` (or canonical `tx_hash`) and index by input index:
  - `btc_sigs_by_intent[intent_hash][input_index][peer] = signature`
  - `pending_transactions[intent_hash] = (tx, tx_bytes, extras, ... )` This guarantees the mapping is independent of local arrival order.
- **Make `round` deterministic / authoritative**: if you must use rounds, ensure the round is **assigned deterministically** (e.g., derived from `intent_hash` via VRF selection or assigned by a single authoritative coordinator and gossiped alongside the tx) so all nodes agree which `round` corresponds to which transaction.

Alleviation

[Qubetics, 01/05/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

QCA-53 Unused Direct Messaging Function Reduces Protocol Robustness And Should Be Integrated Into Final Signing Flow

Category	Severity	Location	Status
Design Issue	Major	src/network/mod.rs (chain-abstraction): 2964~2972; src/signing/mod.rs (chain-abstraction): 3155~3170, 4418~4440	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`
- `src/signing/mod.rs`

The function:

```
 2964
   pub async fn send_direct_message(&self, peer_id: &str, data: &[u8]) ->
Result<()> {
 2965         self.message_tx
 2966             .send(ChannelMessage::Unicast {
 2967                 peer_id: peer_id.to_string(),
 2968                 data: data.to_vec(),
 2969             })
 2970             .await?;
 2971         Ok(())
 2972     }
```

is implemented but never used. Instead, the protocol sends the per-input Bitcoin signatures to the final node using a **topic broadcast**:

```
3155                 let res: std::result::Result<(), anyhow::Error> = self
3156                     .broadcast("signatures-to-final-node", &signature_data)
3157                     .await;
3158                 match res {
3159                     Ok(_) => info!(
3160 
                      "📤 [SIGNING] Successfully sent signature for round {} to
final node {}",
3161                         round, final_node_id
3162                     ),
3163                     Err(e) => {
3164                         error!(
3165 
                       "❌ [SIGNING] Failed to send signature for round {} to
final node {}: {}",
3166                             round, final_node_id, e
3167                         );
3168                         return Err(e);
3169                     }
3170                 }
```

Using a **broadcast** instead of a **direct message to the selected final node** fundamentally breaks the expected signing workflow.

1. Broadcast Allows *Every* Node to Receive All Signatures

When signatures are broadcast to the `"signatures-to-final-node"` topic:

- Any node subscribed to the topic will receive all signatures.
- This means **every node** (not just the designated final node) can collect:
  - all per-input signatures,
  - all peer contributions for the same round,
  - and thus reconstruct the **final transaction**.

This violates the expected architecture where **only the elected final node** should aggregate and broadcast the completed Bitcoin transaction.

2. Multiple Nodes Can Attempt to Finalize and Submit the Same Transaction

Because all nodes have the full signature set:

- Several nodes may independently attempt to reconstruct and broadcast the final BTC transaction.
- This leads to:
  - duplicate submissions,
  - conflicting transaction assembly,
  - inconsistent fee/UTXO selection,

- potential double-broadcasts,
- or malformed transactions caused by slight ordering differences.

In a multisignature workflow, **only one node should ever assemble and submit the transaction** — otherwise the protocol loses determinism and may exhibit unpredictable behavior.

3. The System's "Final Node" Concept Becomes Meaningless

If every node receives the same signing material, the distinction between:

- signer nodes
- aggregator (final) node

collapses entirely.

This destroys trust boundaries and undermines any election, stake-weighted selection, reputation, or BFT logic used to choose the finalizer.

Recommendation

Use `send_direct_message()` when sending BTC signatures to the final node.

Specifically:

- After serializing the `SigningMessage::BtcSignaturesMessage`,
- Attempt **direct unicast to the chosen final node** instead of (or in addition to) topic broadcast.

This dramatically improves robustness during network delays, high-load bursts, or when certain nodes temporarily disconnect.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 94b97730ac62b858126080cb850159a6cffab67f.

QCA-54 PeerId Inconsistency Between NodeManager And NetworkLayer

Category	Severity	Location	Status
Inconsistency	Major	src/main.rs (chain-abstraction): 69~102; src/network/mod.rs (chain-abstraction): 200~234	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/main.rs`
- `src/network/mod.rs`

In `main.rs`, the `node_manager` is initialized with a `PeerId` generated from a newly created ephemeral keypair (`Keypair::generate_ed25519()`), while the `NetworkLayer` loads or creates its own persistent keypair from the database or a newly created ephemeral keypair (`Keypair::generate_ed25519()`). This can result in the `PeerId` used by `node_manager` being different from the one used by `NetworkLayer`.

```
    // Generate node identity
    let local_key = Keypair::generate_ed25519();
    let local_peer_id = libp2p::PeerId::from(local_key.public());

    // Create VRF-based node manager
    let threshold_calculator = Box::new(DefaultThresholdCalculator);
    let node_manager = Box::new(VrfNodeManager::new(
        local_peer_id,
        threshold_calculator,
        10, // selection interval
    ));

    let total = node_manager.get_total_nodes();
    let threshold = node_manager.get_threshold();

    info!("Initial node count: {}, threshold: {}", total, threshold);

    // Initialize database
    let db_config = DatabaseConfig {
        path: format!(
            "./data/rocksdb-{}",
            local_peer_id
                .to_string()
                .chars()
                .take(8)
                .collect::<String>()
        ),
        create_if_missing: true,
        max_open_files: 1000,
    };
    let database = Arc::new(Database::new(db_config)?);
     info!("🗄️ [MAIN] Database initialized successfully");

     let mut network = NetworkLayer::new(node_manager).await?;
```

```
200      async fn load_or_create_keypair() -> Result<(Keypair, PeerId)> {
201          use crate::database::{Database, DatabaseConfig, keys};
202
203          // Create database connection
204          let db_config = DatabaseConfig::default();
205          let database = Database::new(db_config)?;
206
207          // Try to load existing keypair
208
        if let Some(keypair_bytes) = database.get_keypair(keys::PEER_KEYPAIR)? {
209              match Keypair::from_protobuf_encoding(&keypair_bytes) {
210                  Ok(keypair) => {
211                      let peer_id = PeerId::from(keypair.public());
212                      info!("🔄 Loaded existing keypair - Peer ID: {}", peer_id);
213                      return Ok((keypair, peer_id));
214                  }
215                  Err(e) => {
216
                    warn!("⚠️ Failed to decode existing keypair: {}. Generating new
one.", e);
217                  }
218              }
219          } else {
220              info!("🆕 No existing keypair found. Generating new one.");
221          }
222
223          // Generate new keypair
224          let keypair = Keypair::generate_ed25519();
225          let peer_id = PeerId::from(keypair.public());
226
227          // Save keypair to database
228          let keypair_bytes = keypair.to_protobuf_encoding()
229              .map_err(|e| anyhow::anyhow!("Failed to encode keypair: {}", e))?;
230          database.put_keypair(keys::PEER_KEYPAIR, &keypair_bytes)?;
231
232          info!("✅ Generated and saved new keypair - Peer ID: {}", peer_id);
233          Ok((keypair, peer_id))
234      }
```

Impact:

- Peer identity inconsistency may cause issues in node discovery, threshold calculation, and network coordination.
- The node may be recognized as different identities across restarts, breaking persistent membership and routing.

Recommendation

Refactor initialization so that `node_manager` uses the same `PeerId` as `NetworkLayer` (i.e., `network.get_local_peer_id()`), ensuring consistent node identity throughout the system.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue by using the same ID for initialization in commit 3aab22bf177592ac47fe1996ab7b91b370092569.

QCA-55 All Nodes Don't Agree On Consistent (`total` , `threshold`) Parameters Before Starting DKG

Category	Severity	Location	Status
Logical Issue	Major	src/network/mod.rs (chain-abstraction): 665~692	Partially Resolved

Description

The code in `SwarmEvent::ConnectionEstablished` triggers local Distributed Key Generation (DKG) by calling `schedule_dkg_at` function and send `RPCRequest::ScheduleADKG` message to active nodes to start remote DKG. But due to P2P network asynchrony, different nodes may compute different (`total` , `threshold`) values, leading to DKG failures. This happens because:

1. Network delays cause nodes to observe peer connections at different times.
2. Kademlia routing updates are eventually consistent, so some nodes may have stale views of the network.
3. Threshold calculations may differ if nodes use dynamic thresholds (threshold = total/2 + 1) inconsistently.

The current implementation:

```
let total = self.node_manager.get_total_nodes();
let threshold = self.node_manager.get_threshold();

if total >= threshold + 1 {
    info!(
        "🎯 Sufficient nodes connected ({} >= {} + 1), triggering DKG!",
        total, threshold
    );

    let now_secs = SystemTime::now()
        .duration_since(UNIX_EPOCH).unwrap()
        .as_secs();
    let start_time = now_secs + 10;
    info!("🕒 Scheduling DKG start at unix time {}", start_time);

    for peer in self.node_manager.get_active_nodes() {
        if peer != *self.get_local_peer_id() {
            swarm.behaviour_mut()
                .request_response
                .send_request(
                    &peer,
                    RPCRequest::ScheduleADKG { start_time },
                );
        }
    }
    info!("📅 Scheduled DKG start for all peers at {}", start_time);
    self.schedule_dkg_at(start_time);
}
```

It will send `RPCRequest::ScheduleADKG` message to each active node and finally calls `schedule_dkg_at`:

```
550 fn schedule_dkg_at(&mut self, start_time_unix: u64) {
551         if let Some(prev) = self.scheduled_dkg_task.take() {
552             info!("⏰ Cancelling previous scheduled DKG task");
553             prev.abort();
554         }
555 
556         let now = SystemTime::now()
557             .duration_since(UNIX_EPOCH)
558             .unwrap()
559             .as_secs();
560         let delay = start_time_unix.saturating_sub(now);
561 
562         // clone only the sender
563         let tx = self.message_tx.clone();
564         let handle = tokio::spawn(async move {
565             tokio::time::sleep(Duration::from_secs(delay)).await;
566             // send the one "start now" message back into your main loop
567             let _ = tx.send(ChannelMessage::ScheduledDKGStart).await;
568         });
569 
570         self.scheduled_dkg_task = Some(handle);
571         info!("✅ Scheduled DKG to start at {}", start_time_unix);
572     }
```

The `schedule_dkg_at` function will send `ChannelMessage::ScheduledDKGStart` back into main loop:

```
2926  ChannelMessage::ScheduledDKGStart => {
2927                  let current_time = SystemTime::now()
2928                      .duration_since(UNIX_EPOCH)
2929                      .unwrap()
2930                      .as_secs();
2931
2932                  let total = self.node_manager.get_total_nodes();
2933                  let threshold = self.node_manager.get_threshold();
2934                  if let Some(dkg) = &mut self.dkg_node {
2935                      dkg.update_node_params(total, threshold);
2936                      dkg.reset().await;
2937                      if let Some(signing_node) = &mut self.signing_node {
2938                          signing_node.set_threshold(total/2 + 1);
2939                      }
2940                      self.node_manager
2941                          .set_threshold(total/2 + 1);
2942
                  info!("🔄 DKG parameters updated: total_nodes={}, threshold={}",
total, threshold);
2943
                  if let Err(e) = dkg.start_dkg(self.node_manager.clone()).await {
2944                          warn!("❌ Scheduled DKG start failed: {:?}", e);
2945                      }
2946                  } else {
2947                      warn!("❌ DKG node not initialized at scheduled start!");
2948                  }
2949              }
```

It will update `total` and `threshold` into current dkg node and start DKG. After the remote node receives `RPCRequest::ScheduleADKG` message, it will also call `schedule_dkg_at`:

```
850  RPCRequest::ScheduleADKG { start_time } => {
851      info!("🕒 Got DKGStartAt({})", start_time);
852      self.schedule_dkg_at(start_time);
853      RPCResponse::Ack
854  }
```

The issue here is that all nodes don't agree on consistent (`total`, `threshold`) parameters before starting DKG. If DKG runs with mismatched parameters, signature aggregation will fail.

Scenario

1. Node A sees total = 6, computes threshold = 4, and triggers DKG.
2. Node B sees total = 6, computes threshold = 4, and triggers DKG.
3. Node C sees total = 9, computes threshold = 5, and triggers DKG.
4. Since Node C use different `total` and `threshold`, signature aggregation will fail.

Recommendation

Consider implementing mechanism to agree on (`total` , `threshold`) before DKG.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and partially resolved the issue in commit 75b3ac44e94d7e6a4fa01ed1fd6ff97d8d7182c7.

[CertiK, 12/19/2025]: Thank you for sharing your current approach to DKG initiation. While triggering DKG at a fixed node count (e.g., 4 nodes) simplifies coordination, we've identified a fundamental challenge: asynchronous network conditions may still cause nodes to observe inconsistent network states (total, threshold) when DKG begins, leading to failures during signature aggregation.

To ensure all participants agree on consistent parameters, we recommend introducing a consensus round before scheduling DKG:

1. Voting Phase:

When a node detects sufficient connections (e.g., total >= threshold + 1), it broadcasts a VoteDKGParams{total, threshold} request to peers. Each recipient validates the proposed (total, threshold) against its local view and responds with agreement/rejection.

2. Quorum Verification:

The initiator collects responses until a majority (e.g., > threshold nodes) confirm matching parameters. If consensus is reached, the initiator broadcasts a confirmed ScheduleADKG{start_time, total, threshold} with locked-in parameters.

3. Graceful Fallback:

Nodes that fail to achieve consensus within a timeout retry or defer to a leader-based fallback.

QCA-63 Incorrect EVM Reward Distribution Using BTC Units Can Cause Rewards Loss

Category	Severity	Location	Status
Logical Issue	Major	src/rpc_server/mod.rs (chain-abstraction): 2201	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`
- `src/network/mod.rs`

A severe financial miscalculation occurs when BTC-denominated rewards (in satoshis) are incorrectly treated as EVM/native token values (in wei) during reward distribution. This can result in rewards loss.

The current logic:

`src/rpc_server/mod.rs`

```
2135  async fn handle_solver_reward(
2136      req: SolverRewardRequest,
2137      user_registry: std::sync::Arc<DatabaseUserRegistry>,
2138      dkg_storage: std::sync::Arc<DkgStorage>,
2139
    message_tx: std::sync::Arc<tokio::sync::Mutex<mpsc::Sender<ChannelMessage>>>,
2140  ) -> Result<impl warp::Reply, warp::Rejection> {
2141       ....
2142       ....
2143      // Use user registry's reward storage instead of direct reward storage
2144      let reward_opt = user_registry.get_solver_reward_btc(&signer).await;
2145      let reward_str = reward_opt.as_ref().map(|r| r.to_string());
```

The `handle_solver_reward` function calls `get_solver_reward_btc` to get solver reward. This reward is in satoshis as U256:

```
/// Get BTC reward for a specific solver (in satoshis as U256)
    pub async fn get_solver_reward_btc(&self, solver_address: &str) ->
Result<Option<U256>> {
        let key = key_utils::solver_reward_btc_key(solver_address);
        if let Some(value) = self.db.get_string::<String>(&key)? {
            let reward = U256::from_dec_str(&value)
                .map_err(|e| anyhow::anyhow!("Failed to parse btc reward: {}", e))?;
            Ok(Some(reward))
        } else {
            Ok(None)
        }
    }
```

When handling `GossipsubMessage::SolverReward`, the reward is treated as an evm chain reward:

`src/network/mod.rs`

```
 1560  GossipsubMessage::SolverReward { solver_address, reward } => {
 1561
               info!("💰 [REWARD] Received solver reward for {}: {}",
solver_address, reward);
 1562                   // Create dummy transaction paying the solver
 1563
               // Fetch chain_id on the basis of tics (assuming tics is the chain
name or identifier)
 1564
               let chain_to_use = "qubetics"; // Replace this with the actual tics
variable if available
 1565
               let chain_id =
crate::utils::transaction::get_chain_id_from_name(chain_to_use)
 1566
                   .ok_or_else(|| anyhow::anyhow!("Unknown EVM chain: {}",
chain_to_use))?;
 1567
 1568
               // You may need to determine from_address based on your logic, e.g.
from DKG node/group key
 1569                   let from_address = if let Some(dkg_node) = &self.dkg_node {
 1570
                   if let Some(group_key) = dkg_node.get_final_public_key().await {
 1571
                       crate::utils::get_eth_address_from_group_key(group_key)
 1572                   } else {
 1573                       "".to_string()
 1574                   }
 1575                   } else {
 1576                       "".to_string()
 1577                   };
 1578
 1579                   // Use fetch_ethereum_nonce from transaction utils
 1580
               let nonce =
crate::utils::transaction::fetch_ethereum_nonce(&from_address, chain_id).await?;
 1581
 1582                   let ethereum_tx = EthereumTransaction {
 1583                       to: solver_address.clone(),
 1584                       value: format!("0x{:x}", reward),
 1585                       gas_limit: 300000,
 1586                       gas_price: get_default_gas_price_for_chain(chain_to_use),
 1587                       nonce,
 1588                       data: None,
 1589                       chain_id,
 1590                   };
 1591   ...
 1592   ...
```

The `reward` is used in `EthereumTransaction`:

`src/network/mod.rs`

```
1582  let ethereum_tx = EthereumTransaction {
1583                      to: solver_address.clone(),
1584                      value: format!("0x{:x}", reward),
1585                      gas_limit: 300000,
1586                      gas_price: get_default_gas_price_for_chain(chain_to_use),
1587                      nonce,
1588                      data: None,
1589                      chain_id,
1590                  };
```

This mismatch artificially result in reward loss. For example:

1. Intended Reward: 1 BTC (100,000,000 satoshis)
2. Actual EVM TX Value: 100_000_000 wei = 0.0000001 ETH (if chain uses ETH decimals)
3. Solver receives 0.0000001 ETH instead of 1 BTC equivalent and loses most of his rewards.

Recommendation

Consider following fix:

```
- let reward = get_solver_reward_btc(solver); // Returns satoshis (❌)
+ let reward = get_solver_reward_tics(solver); // Returns TICS wei (✅)
```

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue by redesigning the logic to handle both btc and tics rewards in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-64 | Gossip Deposit Intents Bypass Authorization

Category	Severity	Location	Status
Access Control	Major	src/network/mod.rs (chain-abstraction): 1659~1700	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

When a node receives `GossipsubMessage::DepositIntent`, it never authenticates the sender, never checks whether the `intent` was previously registered/approved, and never verifies any user signature. It immediately derives the chain, loads the signing material, and launches the MPC signing flow, even for cross-chain withdrawals:

```
        //src/network/mod.rs
            GossipsubMessage::DepositIntent { intent, intent_id, user_eth_address,
transaction_type, amount } => {
                let intent_hash = crate::utils::calculate_intent_hash(&intent)?;
                let chain_to_use = match transaction_type { ... };
                let signing_key = match transaction_type { ... };   // network/vault
shares for withdrawals
                if let Some(signing_node) = &mut self.signing_node {
                    if crate::utils::transaction::is_evm_chain(chain_to_use) { ... }
                    let chain_transaction_result =

crate::utils::transaction::create_chain_transaction_from_deposit_intent(
                            &intent, group_public_key, derived_eth_addr.as_ref(),
derived_btc_addr.as_ref(),
                            transaction_type.clone(), vault_eth_addr.as_ref(),
vault_btc_addr.as_ref(), amount).await;
                    if let Ok(chain_transaction) = chain_transaction_result {
                        signing_node.set_private_key_from_scalar(key);
                        signing_node.sign_message(... intent_hash, transaction_type
...).await?;
                    }
```

For `NetworkToTarget` and `VaultToNetwork`, the helper that builds the on-chain transaction sets the `from` address to the globally shared network or vault key and the `to` field to whatever `intent.target_address` the gossip sender supplied—no whitelist, no intent-hash lookup, no balance check:

```
//   src/utils/transaction.rs
        let (from_address, to_address) = match transaction_type {
            crate::types::TransactionType::UserToVault => (derived_addr.clone(),
vault_address),
            crate::types::TransactionType::NetworkToTarget => {
                (network_address.clone(), &intent.target_address)
            }
            crate::types::TransactionType::VaultToNetwork => {
                (vault_address.clone(), &network_address)
            }
        };
```

Any peer that can join the libp2p mesh can forge a `deposit-intent` gossip message with `transaction_type = NetworkToTarget` (or `VaultToNetwork`) and point `intent.target_address` to an attacker-controlled account. Every honest node will grab the shared DKG key, build a withdrawal transaction from the cluster's hot wallet, and start MPC signing it. As soon as the threshold is met, the attacker gets a fully valid multi-party signature authorizing an arbitrary withdrawal. This is a catastrophic key-compromise-by-protocol flaw: the entire pooled liquidity can be drained by a single malicious peer without ever interacting with the authenticated HTTP API.

Recommendation

- Require deposit-intent messages to carry a user/cluster signature that is verified before any signing begins, and/or ignore gossiped intents that are not already persisted/approved in the local registry.

Alleviation

[Qubetics, 01/06/2026]: The team heeded the advice and resolved the issue in commit c3f22f7b6f0a812eaaf2da406fb3d3c9854737f70.

The team also considered "in every network to target and vault to network transaction type we are calculating the intent hash with each request and checking if the intent hash exist or not in the node database also we are checking if the previous transaction is also present or not, so if the intent is changed in any gossip message then the intent will not be fulfilled as the intent hash is changed and not found in the database of all the node. So, the issue will be handled there".

QCA-73 Single Compromised Client Can Hijack Entire DKG Network Through `handle_submit_transaction` Function

Category	Severity	Location	Status
Logical Issue	Major	src/network/mod.rs (chain-abstraction): 1288~1324, 2159~2194; src/rpc_server/mod.rs (chain-abstraction): 553~603; src/signing/mod.rs (chain-abstraction): 1690~1704	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`
- `src/rpc_server/mod.rs`
- `src/signing/mod.rs`

The current implementation of `handle_submit_transaction` allows a single compromised node to broadcast arbitrary transactions, which other nodes **blindly sign** without validation. This enables potential theft of network funds if malicious transactions (e.g., unauthorized transfers) are signed by honest nodes using their DKG private key shares.

The `handle_submit_transaction` function accepts transactions without requiring any form of authentication:

`src/rpc_server/mod.rs`

```
553 async fn handle_submit_transaction(
554     tx: DummyTransaction,
555 
    message_tx: std::sync::Arc<tokio::sync::Mutex<mpsc::Sender<ChannelMessage>>>,
556 ) -> Result<impl warp::Reply, warp::Rejection> {
557     info!("📝 HTTP: Received transaction");
558 
559     let tx_hash = format!(
560         "0x{:x}",
561         sha2::Sha256::digest(format!("{:?}", tx).as_bytes())
562     );
563 
564     // First, process the transaction locally (this node will sign it)
565     info!("🔐 [LOCAL] Processing transaction locally for signing");
566 
567     // Send a special local processing message
568     let local_msg = ChannelMessage::LocalTransaction {
569         transaction: tx.clone(),
570     };
571 
572     let tx_sender = message_tx.lock().await;
573     if let Err(e) = tx_sender.send(local_msg).await {
574         warn!("Failed to send transaction for local processing: {:?}", e);
575         return Err(warp::reject::custom(TransactionError::NetworkError));
576     }
577 
578     // Then broadcast to other nodes
579 
    let gossipsub_msg = crate::types::GossipsubMessage::Transaction(tx.clone());
580     let tx_data = serde_json::to_vec(&gossipsub_msg).map_err(|e| {
581         warn!("Failed to serialize transaction: {}", e);
582         warp::reject::custom(TransactionError::SerializationError)
583     })?;
584 
585     if let Err(e) = tx_sender
586         .send(ChannelMessage::Broadcast {
587             topic: "transactions".to_string(),
588             data: tx_data,
589         })
590         .await
591     {
592         warn!("Failed to send transaction to network: {:?}", e);
593         return Err(warp::reject::custom(TransactionError::NetworkError));
594     }
595 
596     info!("✅ HTTP: Transaction submitted to network");
597 
598     Ok(warp::reply::json(&TransactionResponse {
599         tx_hash,
600         status: "submitted".to_string(),
601         message: "Transaction submitted to P_2_P network".to_string(),
602     }))
```

```
603 }
```

It will send `ChannelMessage::LocalTransaction` and broadcast `GossipsubMessage::Transaction`.

When handling `ChannelMessage::LocalTransaction`, the `signing_node.sign_message` function is called with `user_tweaked_share=None`

`src/network/mod.rs`

```
2159 ChannelMessage::LocalTransaction { transaction } => {
2160
            info!("🔐 [LOCAL] Processing transaction locally: {:?}",
transaction);
2161
2162                 if let Some(signing_node) = &mut self.signing_node {
2163                     ...
2164                     ...
2165
                signing_node.sign_message(tx_bytes, &transaction, None, None,
None, None).await?;
2166                     info!("✅ [LOCAL] Local transaction signing completed");
2167                 } else {
2168
                warn!("❌ [LOCAL] Signing node not available for local
processing");
2169                 }
2170             }
```

Because `user_tweaked_share` is None, the `sign_message` defaults to use current node's DKG key(`self.private_key`):

`src/signing/mod.rs`

```
1679  pub async fn sign_message(
1680          &mut self,
1681          _message: Vec<u8>,
1682          transaction: &DummyTransaction,
1683          user_tweaked_share: Option<k256::Scalar>,
1684          user_eth_address: Option<String>,
1685          intent_hash: Option<String>,
1686          transaction_type: Option<crate::types::TransactionType>,
1687      ) -> Result<()> {
1688          // Determine which key to use for signing
1689          // Redacted user_tweaked_share from logs
1690          let signing_key = if let Some(tweaked_share) = user_tweaked_share {
1691              // Use user's tweaked share for user deposits
1692
           info!("🔑 [SIGNING] Using user-specific tweaked share for signing");
1693              Self::derive_secp_secret(&tweaked_share)
1694          } else {
1695              // Use network DKG share for network withdrawals
1696              info!("🔑 [SIGNING] Using network DKG share for signing");
1697              match self.private_key {
1698                  Some(key) => key,
1699                  None => {
1700
                   error!("❌ [SIGNING] Node {} cannot sign: no private key
available", self.id);
1701                      return Err(anyhow::anyhow!("No private key available"));
1702                  }
1703              }
1704          };
1705          ...
1706          ...
```

When handling `GossipsubMessage::Transaction`, the node blindly signs without validation:

src/network/mod.rs

```
1288  GossipsubMessage::Transaction(transaction) => {
1289
              info!("📝 Received transaction via gossipsub: {:?}", transaction);
1290              // Here you can add transaction processing logic
1291
              // For example, validate the transaction, add to mempool, etc.
1292              if let Some(signing_node) = &mut self.signing_node {
1293                  ...
1294                  ...
1295
1296
                   signing_node.sign_message(tx_bytes, &transaction, None, None,
None, None).await?;
1297                  }
1298              }
```

As a result, any client can submit any transactions and these transactions are automatically broadcasted to the whole network nodes. All nodes will then sign transactions without proper authentication, authorization, or transaction validation. If a client is compromised, it can submit arbitrary transactions (e.g., fund transfers) to steal funds.

Scenario

1. The attacker creates a fraudulent transfer:

```
{
  "from": "protocol_treasury",
  "to": "attacker_wallet",
  "amount": "1000000",
  "asset": "USDC"
}
```

2. The compromised client sends the transaction directly to the network without requiring signatures or permissions

```
// Then broadcast to other nodes
    let gossipsub_msg = crate::types::GossipsubMessage::Transaction(tx.clone());
    let tx_data = serde_json::to_vec(&gossipsub_msg).map_err(|e| {
        warn!("Failed to serialize transaction: {}", e);
        warp::reject::custom(TransactionError::SerializationError)
    })?;

    if let Err(e) = tx_sender
        .send(ChannelMessage::Broadcast {
            topic: "transactions".to_string(),
            data: tx_data,
        })
        .await
    {
        warn!("Failed to send transaction to network: {:?}", e);
        return Err(warp::reject::custom(TransactionError::NetworkError));
    }
```

3. Network Blindly Processes & Signs

- Nodes do not verify the sender's identity.
- The transaction is signed with the network's authority key (due to missing `user_tweaked_share`).
- The "legitimate-looking" transaction is included in the next block.

4. Funds Successfully Stolen

- The attacker receives 1,000,000 USDC in their wallet.
- Since the transaction appears valid, no alarms are triggered.

- The attack can be repeated until all funds are drained.

Recommendation

Consider verifying authorization when handling `ChannelMessage::LocalTransaction` and `GossipsubMessage::Transaction` messages.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by removing the `handle_submit_transaction` endpoint in commit 873739725fa6ae53f669b1b70d68e2a77b4b0891.

QCA-74 Silent Fallback To 300k Gas On Estimation Failure Enables Gas-Draining Reverting Calls

Category	Severity	Location	Status
Logical Issue	Major	src/utils/transaction.rs (chain-abstraction): 1112~1123	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The function `create_chain_transaction_from_contract_call` contains a critical vulnerability where it proceeds with transaction construction even when gas estimation fails, defaulting to a hardcoded gas limit of `300,000`. This behavior allows an attacker to force the system to broadcast transactions that will revert, wasting gas fees from the network-controlled account without any actual value transfer.

The code uses `unwrap_or(300000)` when `eth_estimateGas` fails:

`src/utils/transaction.rs`

```
1112  let calldata_hex = calldata
1113          .as_ref()
1114          .map(|d| format!("0x{}", hex::encode(d)))
1115          .unwrap_or_else(|| "0x".to_string());
1116      let estimated_gas = estimate_gas_for_deposit_intent(
1117          &network_address,
1118          &contract_address_env,
1119          &calldata_hex,
1120          chain_id,
1121      )
1122      .await
1123      .unwrap_or(300000);
```

If `eth_estimateGas` returns an error (which typically occurs when the call would revert), the code still builds a transaction using ~330k gas and returns it for signing/broadcast. There is no validation that the calldata is valid or that the contract call is executable. This is a pure logic flaw: an estimation failure (a strong signal the call will revert) should abort, not default to a large gas limit.

An attacker can repeatedly submit invalid transactions, forcing the system to pay gas fees for failed executions.

Recommendation

Update the logic to abort transaction creation if gas estimation fails.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue by returning error if gas estimation fails in commit 1833cc127e4440421dc34052344827bfb43983ab.

QCA-75 | Attacker-Controlled `chainId` Injected Into Ethereum Signing Preimage

Category	Severity	Location	Status
Logical Issue	Major	src/chain/mod.rs (chain-abstraction): 213, 229, 254, 270	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/chain/mod.rs`

A critical security vulnerability exists in the Ethereum transaction signing implementation where the system fails to validate that the `chain_id` specified in the input transaction matches the handler's configured chain ID. This allows attackers to obtain valid signatures for arbitrary Ethereum-compatible chains, enabling cross-chain transaction replays when the same private key is used across multiple chains.

The vulnerability stems from two key functions in `EthereumChainHandler`:

1. create_transaction_bytes()
2. create_transaction_bytes_contract()

Both functions construct signing payloads using the `chain_id` from the untrusted input transaction (`eth_tx.chain_id`) without verifying it matches the handler's configured `self.chain_id`. This uncontrolled `chain_id` value is then incorporated into the EIP-155 signing preimage through RLP encoding:

```
chain_id: eth_tx.chain_id,
```

Because no equality check is performed between `eth_tx.chain_id` and the handler's intended chain, any caller can choose the `chainId` that becomes part of the signed payload. In EIP-155, `chainId` is part of the message being signed and also determines the final `v` value; therefore, this directly produces a valid signature for the attacker-chosen chain rather than the handler's configured chain.

In multi-EVM environments where the same key/address exists across chains, being able to obtain a signature for an arbitrary `chainId` allows spending or executing transactions on that other chain. The function explicitly enables this by forwarding the caller-controlled `eth_tx.chain_id` into the preimage; the handler's `self.chain_id` is ignored here.

Scenario

1. Attacker initializes handler for Chain A (e.g., Ethereum Mainnet)
2. Crafts transaction with `chain_id` for Chain B (e.g., BSC)
3. Obtains signature valid for Chain B despite handler being configured for Chain A
4. Can execute transactions on Chain B if same key is used there

Recommendation

Enforce that the `chainId` used in transaction construction matches the handler's configured self.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 23d3e17aec12fc743b8f05cdb569ef85cc035a07.

QCA-76 | Unconditional Insert Allows Forged `FinalNodeResult` To Overwrite Any Round

Category	Severity	Location	Status
Volatile Code	Major	src/consensus/mod.rs (chain-abstraction): 167~179	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/consensus/mod.rs`

`handle_message` accepts `ConsensusMessage::FinalNodeResult` and unconditionally writes `self.consensus_results.insert(round, result)` with no validation. There is no check that:

1. a result for that `round` was actually computed by this node;
2. the round is known/expected;
3. `result.round` matches the provided `round`;
4. a result for that `round` does not already exist; or
5. the `final_node` belongs to the selected nodes for that `round`.

Because insert overwrites existing entries, any supplied message can create or replace a round's "final" consensus outcome.

Impact:

A caller can forge or replace the consensus result for any round, effectively hijacking the chosen `final_node` for that round. This directly poisons the contract's `consensus_results` state, causing consumers of that state (e.g., checks on who is the final node for a round) to act on falsified data.

```
pub async fn handle_message(&mut self, msg: ConsensusMessage) -> Result<()> {
        match msg {
            ConsensusMessage::FinalNodeResult { result, round } => {
                info!(
                    "📥 [CONSENSUS] Received final node result for round {}: {:?}",
                    round, result.final_node
                );
                self.consensus_results.insert(round, result);
            }
        }
        Ok(())
    }
```

Scenario

Precondition:

The node has a valid result stored for round R (or even if not, the exploit still works). No privileges are required.

Step 1:

Prepare a `SerializablePeerId` you control (A).

Step 2:

Construct a forged ConsensusResult `result_forge` with fields such as:

```
final_node = A
round = R (any value works; it can even mismatch the next step’s round)
node_frequencies = any plausible map (contents are not checked)
total_selections = any positive number
```

Step 3:

Call `handle_message` with `msg = ConsensusMessage::FinalNodeResult { result: result_forge, round: R }`.

Step 4:

The function logs and executes `self.consensus_results.insert(R, result_forge)`, overwriting any existing legitimate result for round R.

Step 5:

Observing state shows `consensus_results[R].final_node == A`, i.e., the round’s final node has been replaced by the attacker’s ID.

Recommendation

Implement strict validation in `handle_message` for `ConsensusMessage::FinalNodeResult` messages. Ensure that:

1. the round is valid and expected;
2. `result.round` matches the provided round parameter;
3. a consensus result for the round does not already exist;
4. the result can be verified through proper local computation by this node; and
5. the `final_node` is among the selected nodes for the given round.

Reject any messages that fail these checks before updating `consensus_results`.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue in commit 1c959b9de6872219be6d8fc1d494c17f87a71385

QCA-77 Consensus-Threshold Bypass Via Spoofed `selector_node_id` In VRF Broadcasts

Category	Severity	Location	Status
Design Issue	Major	src/vrf/mod.rs (chain-abstraction): 147~161	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/vrf/mod.rs`

`handle_vrf_broadcast` -> `handle_selected_nodes_broadcast` -> `can_start_consensus` treats `broadcast.selector_node_id` as an authenticated identity and uses the number of distinct `selector_node_id` entries received in a round to decide readiness. There is no validation that the broadcast actually came from the claimed `selector_node_id` or that the broadcast is signed. Because `received_selections[round]` is keyed solely by `broadcast.selector_node_id`, a single adversarial node can inject arbitrarily many entries by crafting broadcasts with different `selector_node_id` values.

Both `handle_selected_nodes_broadcast` and `handle_vrf_broadcast` set `min_selections = total_nodes`, so the attacker can unilaterally satisfy the threshold by sending `total_nodes` such spoofed broadcasts. This causes `selector.can_start_consensus(...)` to return `true` and the service to conclude that “consensus can be triggered,” even though only one node sent all messages.

Root Cause:

- `VrfService::handle_vrf_broadcast` stores the broadcast via `selector.handle_selected_nodes_broadcast(broadcast, total_nodes)` and then checks `can_start_consensus(round, min_selections)` with `min_selections = total_nodes`.
- `VrfNodeSelector::handle_selected_nodes_broadcast` inserts the broadcast into `self.received_selections[round]` keyed by `broadcast.selector_node_id` and repeats the same readiness check with `min_selections = total_nodes`.
- `VrfNodeSelector::can_start_consensus` computes `total_selections = (received_selections[round].len()) + (1 if own_selection else 0)`, and `returns total_selections >= min_selections`. There is no authentication or validation of `selector_node_id`, so `received_selections[round].len()` can be inflated by spoofed broadcasts.

- `selector_node_id` is entirely message-controlled and used as the deduplication key. No signature or sender verification is performed anywhere in these functions.
- Readiness is based on “how many unique selector_node_id broadcasts were received,” not on verified participants or on the aggregated selected nodes. Therefore, one node can impersonate many.

Scenario

1. Pick a target round R that the victim node will accept (e.g., their current round).
2. Generate total_nodes distinct PeerIds locally (e.g., ephemeral ed25519 keypairs) and label them P1..PN.
3. For each i in 1..N, craft a VrfSelectedNodesBroadcast with:
   - round = R
   - selector_node_id = Pi (a different one each time)
   - selected_nodes = any vector (contents are not validated in these functions)
   - timestamp = any positive value
4. Send these N broadcasts to the victim so they are processed by `handle_vrf_broadcast` in sequence. Each call inserts the message into `received_selections[R]` keyed by Pi, increasing the map’s length by 1.
5. After the Nth message, `can_start_consensus(R, min_selections=total_nodes)` returns `true` because `received_count == total_nodes` (or after N-1 if the victim already has an own_selection for R). The victim logs “consensus can be triggered,” treating the threshold as satisfied solely by spoofed identities.

```
pub async fn handle_vrf_broadcast(&self, broadcast: VrfSelectedNodesBroadcast,
total_nodes: usize) {
        let mut selector = self.selector.write().await;
        let round = broadcast.round;

        selector.handle_selected_nodes_broadcast(broadcast, total_nodes);

        // Check if consensus can be started
        let min_selections = total_nodes;
        if selector.can_start_consensus(round, min_selections) {
//////////////////
            info!(
                "🎯 [VRF] Enough selections collected for round {}, consensus can be
triggered",
                round
            );
        }
    }
```

Recommendation

Implement strict authentication and verification of `selector_node_id` values in all relevant broadcast handling functions. Ensure each broadcast is cryptographically signed by the originating node, and only accept messages where the signature and claimed `selector_node_id` correspond to a verified peer. Deduplicate received broadcasts using authenticated peer identities instead of unauthenticated, message-controlled `selector_node_id` fields to prevent spoofing and false threshold satisfaction.

Alleviation

[Qubetics, 12/26/2025]: The team resolved the issue in commit 474a73b6cdbdba8b42f50ee91b32143abeb1473b

QCA-96 | The DKG Protocol Lacks Of Round Check

Category	Severity	Location	Status
Logical Issue	Major	src/adkg_secp256k1/mod.rs (chain-abstraction): 333~339	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The current implementation of DKG protocol lacks round validation in `DKGMessage::ShareValidation` handling:

`src/adkg_secp256k1/mod.rs`

```
321
pub async fn handle_message(&mut self, msg: DKGMessage,node_manager: Box<dyn
NodeMembership>,
322  ) -> Result<()> {
323          match msg {
324              DKGMessage::ShareDistribution {
325                  from,
326                  shares,
327                  commitments,
328              } => {
329                  info!("📥 Received share distribution from {}", from);
330
                self.handle_share_distribution(from, shares,
commitments,node_manager.clone())
331                      .await?;
332              }
333              DKGMessage::ShareValidation { from, to, is_valid ,round} => {
334                  info!(
335                      "✅ Received share validation from {} for {}: {}",
336                      from, to, is_valid
337                  );
338
             >>@audit  self.handle_share_validation(from, to,
is_valid,node_manager).await?;
339              }
340          }
341          Ok(())
342      }
```

This creates two critical security vulnerabilities:

1. **Cross-round Confusion:** Validation messages from previous rounds can interfere with current round operations
2. **Replay Attacks:** Malicious actors can resend old valid validation messages to disrupt the protocol

1 Cross-round Confusion

When a node resets and starts a new DKG round, validation messages from previous rounds could be mistakenly processed as belonging to the current round.

Attack Flow

1. Normal Operation:

```
Round 1 DKG completes successfully
Nodes store validation messages from round 1
```

2. Node Reset:

```
System initiates round 2
round counter increments to 2
Storage may retain old validation data
```

3. Attack Execution:

```
Old round 1 validation messages remain in network buffers
Nodes receive these stale messages
Messages get processed without round checking
Round 2 DKG gets corrupted by round 1 validations
```

Impact

1. Invalid threshold signature aggregation
2. Potential key material leakage
3. Consensus failures among nodes

2 Replay Attacks

Replayed `true` validations from an earlier round accumulate, driving `check_completion` to satisfy the `valid_validations >= total_nodes * total_nodes` gate and forcing `generate_group_key()` even if the current round has not actually received fresh validations. An attacker who recorded the previous round can replay the entire set of validations (or even a single peer can keep re-broadcasting its old message), resulting in premature DKG completion with potentially stale share data.

Attack Flow

1. Eavesdropping:

```
Attacker monitors network during round 1
Records valid ShareValidation messages
```

2. Round Advancement:

```
System progresses to round 2
Honest nodes generate new polynomials
```

3. Attack Execution:

```
Attacker replays round 1 validation messages
Messages appear valid but belong to wrong round
Nodes accept them due to missing round check
```

Impact

1. False validation counts accumulate
2. DKG may complete with invalid shares
3. Final key becomes compromised
4. Enables rogue key generation

Recommendation

Consider validating round when handling `DKGMessage::ShareValidation`.

Alleviation

[Qubetics, 11/03/2025]: The team heeded the advice and resolved the issue in commit `86c2db00ba26176472ba6e7eea8ace322ba56e01`.

QCA-97 Lack Of EVM Balance Validation In `create_chain_transaction_from_deposit_intent_internal` Can Lead To DoS Attack

Category	Severity	Location	Status
Denial of Service	Major	src/utils/transaction.rs (chain-abstraction): 842~850	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The `create_chain_transaction_from_deposit_intent_internal` function has a critical security gap: while it properly validates Bitcoin (BTC) transaction inputs via `select_utxos_for_amount` (ensuring sufficient balance before constructing a transaction), it fails to perform equivalent balance checks for EVM-based (Ethereum and compatible chains) transactions. This oversight allows the following risks:

1. Invalid Transactions On-Chain: Transactions may be submitted even when the sender lacks sufficient funds, leading to failed executions.
2. Wasted Gas Costs: Every failed transaction still consumes gas, unnecessarily draining system funds.
3. Potential DoS Attack Vector: Malicious actors could repeatedly submit invalid transactions, forcing the system to pay for failed executions.

The current implementation:

`src/utils/transaction.rs`

```
// Get dynamic gas price for Qubetics, fallback to static for other chains
        let gas_price = get_gas_price_for_chain(chain_to_use, chain_id).await
            .unwrap_or_else(|e| {

           warn!("⚠️ Failed to get dynamic gas price: {}, using fallback", e);
                get_default_gas_price_for_chain(chain_to_use)
            });

        let ethereum_tx = EthereumTransaction {
            to: to_address.to_string(),
            value: amount_to_hex_string(amount),
            gas_limit: 300000,
            gas_price,
            nonce: fetch_ethereum_nonce(&from_address, chain_id).await?,
            data: None,
            chain_id,
        };

        Ok(ChainTransaction::Ethereum(ethereum_tx))
```

It directly constructs an `EthereumTransaction` without checking the sender's balance. As a result, a malicious user could:

1. Repeatedly trigger transactions from an underfunded address.
2. Force the system to pay gas fees for failed transactions.
3. Potentially exhaust the system's gas budget in a denial-of-service (DoS) attack.

Recommendation

Before creating an EVM transaction, verify the sender's balance covers:

1. The transfer amount (value).
2. The maximum gas cost (gas_limit * gas_price).

Alleviation

[Qubetics, 12/16/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> in case of ethereum , the txn broadcast fails when balance is not sufficient for txn to complete

[CertiK, 12/14/2025]: We still recommend the client to verify the sender's balance before creating an EVM transaction because the lack of pre-transaction balance validation for EVM chains exposes the system to unnecessary gas costs and potential DoS risks.

QCA-98 Concurrent Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Incorrect Group Public Key Settings

Category	Severity	Location	Status
Concurrency	Major	src/network/mod.rs (chain-abstraction): 1847~1871, 2444~2470	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

A critical race condition exists in the handling of `DepositIntent` messages where concurrent processing of `ChannelMessage::DepositIntent` and `GossipsubMessage::DepositIntent` can lead to incorrect group public key settings. The current implementation shares mutable signing node state across asynchronous operations, creating a window where transaction signing may use wrong cryptographic keys when processing different transaction types (`UserToVault` vs `NetworkToTarget`) simultaneously.

Both message handlers (`ChannelMessage` and `GossipsubMessage` paths) mutate the same `group_public_key` field:

`src/network/mod.rs`

```
// Logic Of GossipsubMessage::DepositIntent
...
...
// Set the appropriate group public key based on transaction type
  match transaction_type {
      crate::types::TransactionType::UserToVault => {
          // For user deposits, use user-specific group key
          if let Some(eth_address) = &user_eth_address {
              if let Some(user_registry) = &self.user_registry {

               if let Some(user) =
user_registry.get_user_by_address(eth_address).await {
                      if let Some(user_group_key) = &user.user_group_key {

signing_node.set_group_public_key(user_group_key.0.into()); // Convert AffinePoint
to ProjectivePoint

                       info!("🔑 [GOSSIPSUB] Set user-specific group public key
for: {}", eth_address);
                      } else {

                       warn!("❌ [GOSSIPSUB] User {} has no group key available",
eth_address);
                          return Ok(());
                      }
                  }
              }
      }
      crate::types::TransactionType::NetworkToTarget => {
          // For network withdrawals, use the original DKG group key
          if let Some(dkg_node) = &self.dkg_node {

           if let Some(group_key) = dkg_node.get_final_public_key().await {
                  signing_node.set_group_public_key(group_key);
                  info!("🔑 [GOSSIPSUB] Set network DKG group public key");
              }
          }
      }
```

`src/network/mod.rs`

```
// Logic of ChannelMessage::DepositIntent
...
...
// Set the appropriate group public key based on transaction type
match transaction_type {
    crate::types::TransactionType::UserToVault => {
        // For user deposits, use user-specific group key
        if let Some(eth_address) = &user_eth_address {
            if let Some(user_registry) = &self.user_registry {

             if let Some(user) =
user_registry.get_user_by_address(eth_address).await {
                    if let Some(user_group_key) = &user.user_group_key {

                    signing_node.set_group_public_key(user_group_key.0.into());
// Convert AffinePoint to ProjectivePoint

                    info!("🔑 [LOCAL] Set user-specific group public key for:
{}", eth_address);
                    } else {

                    warn!("❌ [LOCAL] User {} has no group key available",
eth_address);
                        return Ok(());
                    }
                }
            }
        }
    }
    crate::types::TransactionType::NetworkToTarget => {
        // For network withdrawals, use the original DKG group key
        if let Some(dkg_node) = &self.dkg_node {
            if let Some(group_key) = dkg_node.get_final_public_key().await {
                signing_node.set_group_public_key(group_key);
                info!("🔑 [LOCAL] Set network DKG group public key");
            }
        }
    }
```

There is a potential race condition:

```
[Thread A - NetworkToTarget]           [Thread B - UserToVault]
1. set_group_public_key(network_key)
                                  2. set_group_public_key(user_key)
3. sign() // Uses wrong key!
```

Thread B will override `network_key` with `user_key`. As a result, Thread A will use wrong key when signing transactions.

Impact:

1. User transactions may be signed with network keys (security violation)

2. Network transactions may be signed with user keys (functional failure)
3. Non-reproducible signature validation failures

Recommendation

Consider passing the group key to the signing function per transaction.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by passing the group key to the signing function in commit 6dd237f82ad5fd046ed89b36ad1521b1ee4585c9.

QCA-99 Simultaneous Processing `ChannelMessage::DepositIntent` And `GossipsubMessage::DepositIntent` Can Lead To Race Conditions When Setting Chain Handlers

Category	Severity	Location	Status
Logical Issue	Major	src/network/mod.rs (chain-abstraction): 1707, 1774, 2307, 2374	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The code exhibits a critical concurrency vulnerability where simultaneous processing of `ChannelMessage::DepositIntent` and `GossipsubMessage::DepositIntent` can lead to race conditions when setting chain handlers. This occurs because both message handlers modify the shared SigningNode's chain handler state without proper synchronization, potentially causing transactions to be processed with incorrect chain handlers.

In `GossipsubMessage::DepositIntent`, it will set chain handlers based on transaction type:

`src/network/mod.rs`

```
 1704  if crate::utils::transaction::is_evm_chain(chain_to_use) {
 1705
                        if let Some(chain_id) =
crate::utils::transaction::get_chain_id_from_name(chain_to_use) {
 1706
                            let chain_handler =
crate::chain::ChainHandlerFactory::create_ethereum_handler(chain_id);
 1707
                            signing_node.set_chain_handler(chain_handler.into());
 1708
                            info!("🔗 [GOSSIPSUB] Set Ethereum chain handler for
chain: {} (ID: {})", chain_to_use, chain_id);
 1709                           }
 1710
                    } else if chain_to_use == "bitcoin" || chain_to_use == "btc" {
 1711
                        let mut chain_handler =
crate::chain::ChainHandlerFactory::create_bitcoin_handler("testnet".to_string());
 1712
 1713                           ...
 1714                           ...
 1715                           signing_node.set_chain_handler(chain_handler.into());
 1716                           info!("🔗 [GOSSIPSUB] Set Bitcoin chain handler");
 1717                       } else {
 1718
                        warn!("❌ [GOSSIPSUB] Unsupported chain: {}, using default
Ethereum handler", chain_to_use);
 1719                       }
```

The same logic exits in `ChannelMessage::DepositIntent`:

`src/network/mod.rs`

```
 2304  if crate::utils::transaction::is_evm_chain(chain_to_use) {
 2305
                        if let Some(chain_id) =
crate::utils::transaction::get_chain_id_from_name(chain_to_use) {
 2306
                            let chain_handler =
crate::chain::ChainHandlerFactory::create_ethereum_handler(chain_id);
 2307
                            signing_node.set_chain_handler(chain_handler.into());
 2308
                            info!("🔗 [LOCAL] Set Ethereum chain handler for chain:
{} (ID: {})", chain_to_use, chain_id);
 2309                           }
 2310
                    } else if chain_to_use == "bitcoin" || chain_to_use == "btc" {
 2311
                        let mut chain_handler =
crate::chain::ChainHandlerFactory::create_bitcoin_handler("testnet".to_string());
 2312
 2313                           ...
 2314                           ...
 2315
 2316                           signing_node.set_chain_handler(chain_handler.into());
 2317                           info!("🔗 [LOCAL] Set Bitcoin chain handler");
 2318                       } else {
 2319
                        warn!("❌ [LOCAL] Unsupported chain: {}, using default
Ethereum handler", chain_to_use);
 2320                       }
```

Consider following race condition scenario:

1. Thread A processes `ChannelMessage::DepositIntent` (e.g., for an EVM transaction)
    - Sets **EVM** chain handler via `set_chain_handler()`
    - Begins transaction preparation
2. Thread B processes `GossipsubMessage::DepositIntent` (e.g., for a BTC transaction)
    - Overrides with **BTC** chain handler via `set_chain_handler()`
3. Thread A continues execution but now operates with the wrong (BTC) chain handler

Impact:

1. EVM transactions being processed as BTC transactions (or vice versa)
2. Invalid transaction serialization/deserialization
3. Failed transaction submissions

Recommendation

Consider passing chain handler to the signing function per transaction.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by passing chain handler to the signing function in commit ebef65083275f40584bbc34445965ae889d3a144.

QCA-04 The `check_completion()` Counting All Validation Entries Without Deduplication May Lead To Premature Key Generation

Category	Severity	Location	Status
Logical Issue	Medium	src/adkg_secp256k1/mod.rs (chain-abstraction): 564~608	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The Distributed Key Generation (DKG) implementation contains a critical validation counting flaw where duplicate validation messages can artificially inflate the completion criteria. The current logic in `check_completion()` counts all validation entries without deduplication, potentially leading to premature key generation.

The current storage allows duplicate validations:

`src/adkg_secp256k1/mod.rs`

```
550  async fn handle_share_validation(
551          &mut self,
552          from: String,
553          to: String,
554          is_valid: bool,
555          node_manager: Box<dyn NodeMembership>,
556      ) -> Result<()> {
557          self.storage.add_validation(&to, is_valid).await?;
558          info!("📊 Validation from {} to {}: {}", from, to, is_valid);
559          self.check_completion(node_manager).await?;
560          Ok(())
561      }
```

```
73  pub async fn add_validation(&self, to: &str, is_valid: bool) -> Result<()> {
74          let key = format!("{}{}", keys::DKG_VALIDATION, to);
75
        let mut current: Vec<bool> = self.db.get_string(&key)?.unwrap_or_default();
76          current.push(is_valid);
77          self.db.put_string(&key, &current)?;
78          Ok(())
79  }
```

Validations are stored as a `Vec<bool>` without tracking the validator identity. Multiple `ShareValidation` messages between the same (from, to) pair create duplicate entries.

The `check_completion` counts the valid validations using following code:

`src/adkg_secp256k1/mod.rs`

```
580  let valid_validations: usize = validations
581                  .values()
582                  .map(|v| v.iter().filter(|&&valid| valid).count())
583                  .sum();
```

This counts every true validation regardless of whether it's from the same source. The following condition:

```
valid_validations >= (node_manager.get_total_nodes()) *
(node_manager.get_total_nodes())
```

may evaluate as `true` even when not all nodes have submitted valid validations, due to duplicate validation messages artificially inflating the count.

As a result, the DKG protocol may generate an incorrect group key when:

1. Some nodes haven't actually validated others properly
2. The validation count appears sufficient due to duplicates
3. The resulting key won't have the proper threshold security properties

Scenario

Actual state:

- A validated B: true
- A validated C: false
- B validated A: true
- B validates C: false
- C validates A: true
- C validates B: true

But if A sends 10 extra "A validates B: true":

- System sees 13 validations (3 real + 10 fake)
- May trigger completion despite two false validations

Proof of Concept

Run following test:

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::collections::HashMap;

    #[test]
    fn test_validation_counting_with_duplicates() {
        // Setup test data - Mock storage with HashMap<String, Vec<bool>>
        let mut validations: HashMap<String, Vec<bool>> = HashMap::new();

        // Simulate 3 nodes (n=3)
        let nodes = vec!["A".to_string(), "B".to_string(), "C".to_string()];
        let n = nodes.len();

        // Case 1: Normal scenario without duplicates (should pass)
        validations.insert("A".to_string(), vec![true, true, true]);  // A,B,C
validates A
        validations.insert("B".to_string(), vec![true, true, true]);  // A,B,C
validates B
        validations.insert("C".to_string(), vec![true, true, true]);  // A,B,C
validates C

        let valid_validations = validations.values()
            .flat_map(|v| v.iter().filter(|&&valid| valid))
            .count();

        assert_eq!(valid_validations, n*n);  // 9 validations (correct)

        // Case 2: Missing some validations but with duplicates (should fail but
passes)
        validations.clear();
        validations.insert("A".to_string(), vec![true, true, true, true, true]); //
B sends 3 duplicates
        validations.insert("B".to_string(), vec![true, true]);                   //
2 validations
        validations.insert("C".to_string(), vec![true, true]);                   //
2 validations

        let valid_validations = validations.values()
            .flat_map(|v| v.iter().filter(|&&valid| valid))
            .count();

        // Should fail (only 4 unique validations needed) but passes due to
duplicates
        assert!(valid_validations >= n*n);  // 9 validations (incorrect - actually
has 9 counts)

        println!("Test should fail but pass");
```

```
    }
}
```

The result:

```
---- adkg_secp256k1::tests::test_validation_counting_with_duplicates stdout ----
Test should fail but pass
```

Recommendation

Consider implementing deduplication directly in the `add_validation` method.

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue by using HashMap in commit 7b7c6938a402c93a167fdfceca892142e8cb1689.

QCA-05 Using Hardcoded Testnet Network To Generate Bitcoin Address May Cause Fund Loss

Category	Severity	Location	Status
Logical Issue	Medium	src/utils/mod.rs (chain-abstraction): 139	Resolved

Description

The `get_btc_address_from_group_key_p2pkh` function currently hardcodes Bitcoin addresses to `Network::Testnet`, which poses risks in production environments. This design choice lacks flexibility and may lead to unintended consequences, such as generating invalid addresses for Bitcoin mainnet usage or causing confusion between test and production environments.

The current implementation:

```
139  let btc_address = bitcoin::Address::p2pkh(&compressed, Network::Testnet);
```

If deployed in production, the generated addresses will be Testnet-only, meaning they cannot receive real Bitcoin (BTC). Users or systems may accidentally send mainnet BTC to Testnet addresses, resulting in permanent fund loss.

Recommendation

Consider accepting Network as a parameter:

```
pub fn get_btc_address_from_group_key_p2pkh(
    pubkey: ProjectivePoint,
    network: Network // Accept Network as a parameter
) -> Result<String> {
...
...
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit ba7a7f695fd0b091556c8b7e0b09837f788c450b.

QCA-06 | Secret Shares And Keys Logged In Info! Statements

Category	Severity	Location	Status
Volatile Code	Medium	src/adkg_secp256k1/mod.rs (chain-abstraction): 281~290	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/adkg_secp256k1/mod.rs
- src/network/mod.rs

`DKGNode` prints the raw polynomial coefficients, per-node shares, final secret share, vault tweak scalars, and vault-derived keys at the normal `info` log level. Anyone with access to application logs (local disk, centralized logging, crash dumps) can reconstruct the node's secret share, tweaked share, and resulting group/vault keys, defeating the secrecy guarantees of the DKG protocol.

```
    //src/adkg_secp256k1/mod.rs
     info!("  📊 Final secret share: {:?}", final_secret_share);
     info!("  🔧 Tweak scalar: {:?}", tweak_scalar);
     info!("  🏦 Vault tweaked secret: {:?}", vault_tweaked_secret);
```

```
     src/adkg_secp256k1/mod.rs
     info!("🔍 [DKG] Share {}: index={}, value={:?}", i+1, share.index,
share.value.0);
     info!("🔍 [DKG] === END SECRET SHARE GENERATION ===");
```

```
     src/adkg_secp256k1/mod.rs
     info!("🔐 [DKG] Final secret share: {:?}", final_secret_share);
     info!("🏦 [DKG] Vault tweak scalar: {:?}", tweak_scalar);
     info!("🏦 [DKG] Vault ETH address: {}", vault_eth);
```

Because DKG shares must remain confidential for threshold security, logging them at all (especially at `info` rather than a gated debug path) is a critical leak. Any compromise of the logging sink or accidental log sharing exposes the material. In production this trivially enables key recovery for adversaries.

Note:

`src/main.rs` configures `tracing_subscriber` with a default directive of `LevelFilter::INFO`, then wires that filter into the registry along with both file and console layers. No conditional gating distinguishes production from other builds, so all `info!` emissions propagate to `stdout/stderr` and the rolling log file in every environment unless `RUST_LOG` is manually overridden.

Recommendation

Strip or heavily gate all logging that emits secret material (scalar values, private shares, tweak factors, vault keys). If detailed diagnostics are needed, wrap them behind compile-time debug features and ensure they never run in production.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue by stripping secrete materials in commit f32af25a1a008985d863c7f780d84c76938095d6

QCA-08 The `reset()` Method In `DKGNode` Does Not Properly Clean Up Vault-Specific Secrets

Category	Severity	Location	Status
Logical Issue	Medium	src/adkg_secp256k1/mod.rs (chain-abstraction): 82~101	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The `reset()` method in `DKGNode` does not properly clean up vault-specific secrets (`vault_tweaked_secret` and `vault_group_key`) when resetting the DKG protocol. This oversight could leave stale cryptographic material in storage, potentially causing security risks or incorrect behavior in subsequent DKG rounds.

The current reset() method clears:

`src/adkg_secp256k1/mod.rs`

```
83          if let Err(e) = self.storage.clear_shares().await {
84              warn!("Failed to clear shares: {}", e);
85          }
86          if let Err(e) = self.storage.clear_commitments().await {
87              warn!("Failed to clear commitments: {}", e);
88          }
89          if let Err(e) = self.storage.clear_validations().await {
90              warn!("Failed to clear validations: {}", e);
91          }
92          if let Err(e) = self.storage.delete_final_secret().await {
93              warn!("Failed to delete final secret: {}", e);
94          }
95          if let Err(e) = self.storage.delete_final_public().await {
96              warn!("Failed to delete final public key: {}", e);
97          }
```

However, it does not delete `vault_tweaked_secret` and `vault_group_key` . Since these vault-specific secrets are derived from the DKG results, they should also be cleared when resetting the protocol.

Impact:

1. Stale Secrets: If a new DKG round starts without clearing these values, old vault keys might persist unintentionally.
2. Incorrect Key Derivation: Future operations might mistakenly reuse outdated vault keys.
3. Potential Attack Surface: Retaining unnecessary secrets increases the risk of key leakage.

Recommendation

Consider adding following logic in `reset`:

```
if let Err(e) = self.storage.delete_vault_tweaked_secret().await {
    warn!("Failed to delete vault tweaked secret: {}", e);
}
if let Err(e) = self.storage.delete_vault_group_key().await {
    warn!("Failed to delete vault group key: {}", e);
}
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 66d15dad4728e0b03ec10a4075660b48f3abc1e5.

QCA-107 | Susceptible To Signature Malleability

Category	Severity	Location	Status
Logical Issue	Medium	src/rpc_server/mod.rs (chain-abstraction): 2006~2010	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The `recover_eth_address_secp` function currently lacks validation for the upper range of `s` values in `ECDSA` signatures, which conflicts with Ethereum's strict checks introduced in the Homestead hard fork (EIP-2). This omission could lead to:

1. Incompatible signatures: Signatures with `s > secp256k1_N/2` are rejected by Ethereum nodes but accepted locally.
2. Security risks: Potential acceptance of malleable signatures, violating Ethereum's anti-replay protections.

Ethereum enforces strict checks via ValidateSignatureValues:

```
// ValidateSignatureValues verifies whether the signature values are valid with
// the given chain rules. The v value is assumed to be either 0 or 1.
func ValidateSignatureValues(v byte, r, s *big.Int, homestead bool) bool {
    if r.Cmp(common.Big1) < 0 || s.Cmp(common.Big1) < 0 {
        return false
    }
    // reject upper range of s values (ECDSA malleability)
    // see discussion in secp256k1/libsecp256k1/include/secp256k1.h
    if homestead && s.Cmp(secp256k1halfN) > 0 {
        return false
    }
    // Frontier: allow s to be in full N range
    return r.Cmp(secp256k1N) < 0 && s.Cmp(secp256k1N) < 0 && (v == 0 || v == 1)
}
```

It rejects `s > secp256k1_N/2` to prevent signature malleability.

The current implementation:

`src/rpc_server/mod.rs`

```
2006  let secp = Secp256k1::new();
2007
   let m = Message::from_digest_slice(&hash).map_err(|e| anyhow::anyhow!("message:
{}", e))?;
2008      let pk: PublicKey = secp
2009          .recover_ecdsa(&m, &rsig)
2010          .map_err(|e| anyhow::anyhow!("recover pk: {}", e))?;
```

It accepts any `s` value, even those Ethereum would reject.

ECDSA signatures are malleable due to elliptic curve symmetry:

1. For a given `(r, s)`, another valid signature exists as `(r, -s mod n)`.
2. `recover_eth_address_secp` does not enforce `s ≤ n/2`, allowing attackers to submit modified `s` values.

This permits us to carefully adjust `s` to produce a second valid signature for the same `r`, thus breaking the assumption that a signature cannot be replayed in what is known as a replay-attack.

Recommendation

Consider adding explicit s-value validation before signature recovery.

Alleviation

[Qubetics, 11/03/2025]: The team heeded the advice and resolved the issue in commit 86c2db00ba26176472ba6e7eea8ace322ba56e01.

QCA-108 | Potential Replay Attack In `handle_user_mpc_deposit`

Category	Severity	Location	Status
Logical Issue	Medium	src/rpc_server/mod.rs (chain-abstraction): 1528~1550	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The `handle_user_mpc_deposit` function in the RPC server is vulnerable to replay attacks due to the lack of mechanisms (such as nonces) to ensure request uniqueness. An attacker could intercept and resubmit a valid `DepositIntent` request multiple times, leading to unintended repeated `UserToVault` transactions and potential loss of user funds.

The current implementation of `handle_user_mpc_deposit` does not enforce request uniqueness:

`src/rpc_server/mod.rs`

```
let user_registration = match user_registry
        .get_user_by_intent_hash(&intent_hash_hex)
        .await
    {
        Some(existing_user) => existing_user,
        None => {
            // No existing user for this intent hash
            match &req.signature {
                Some(signature) => {

                 info!("No existing user found for intent hash, but signature is present in the request.");

                 let (compact, v) = match normalize_eth_signature(signature) {
                          Ok(t) => t,
                          Err(e) => {
                              warn!("❌ Failed to normalize signature: {}", e);

                         return Ok(warp::reply::json(&UserMpcDepositVerifyResponse {
                                  status: "error".to_string(),
                                  signer_address: "".to_string(),
                                  is_registered: false,
                                  amount: 0,
                                  user_to_network_tx_id: None,
                                  network_to_target_tx_id: None,
                                  vault_to_network_tx_id: None,
                                  final_solver_id: None,

                              error_message: Some(format!("Invalid signature format: {}", e)),
                              }));
                          }
                      };

                 let (signer, _used_personal) = match recover_eth_address_secp(
                          intent_bytes,
                          &compact,
                          v,
                          true,
                      ) {
                      ...
                      ...
```

Since the signature only covers the `DepositIntent` structure (without a `nonce`), the same request can be reused indefinitely. This could allow an attacker (or even a malicious relayer) to drain user's funds by repeatedly triggering the same transaction.

Scenario

1. Legitimate User signs and submits a `DepositIntent` to move funds to the vault(UserToVault transaction).

2. Attacker intercepts the request (e.g., via MITM or logs).
3. Attacker resubmits the same request multiple times.
4. Server processes each duplicate request, leading to multiple unintended transfers.
5. The user will suffer fund loss.

Recommendation

Consider including a user-specific counter (`nonce`) to prevent reuse.

Alleviation

[Qubetics, 12/16/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> Issue acknowledged. I won't make any changes for the current version. As its the part of flow
>
> 1st attempt usertovault txn
>
> 2nd attempt networktotarget
>
> 3rd attempt vaulttonetwork
>
> and each txn will process on the basis of data stored on solver network not the request payload request payload cant define txn-type

QCA-109 The `handle_deposit_intent` Function Lacks Of Price Slippage Protection

Category	Severity	Location	Status
Financial Manipulation	Medium	src/rpc_server/mod.rs (chain-abstraction): 735~768	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The current implementation has a critical vulnerability related to price timing between the initial `UserToVault` transaction and the final `NetworkToTarget` execution. The system fetches real-time prices only during the `NetworkToTarget` phase without accounting for potential market movements since the user initiated the swap. This creates substantial financial risk for users who may receive significantly fewer tokens than expected due to price fluctuations.

The current implementation:

`src/rpc_server/mod.rs`

```
735 let prices = fetch_crypto_prices()
736             .await
737
          .map_err(|_| warp::reject::custom(TransactionError::NetworkError))?;
738       ...
739       ...
740 let target_smallest = convert_src_smallest_to_dst_smallest(
741             intent.amount,
742             &intent.source_chain,
743             &intent.target_chain,
744             &prices,
745         )?;
746 required_amount_u128 = target_smallest;
```

Prices may change between `UserToVault` transaction and `NetworkToTarget` execution. If prices move unfavorably between these two transactions, users may receive significantly fewer tokens than expected.

For example: A `UserToVault` transaction quoted at 1 BTC = 100,000 TICS. It may execute at 1 BTC = 90,000 TICS due to a 10% price drop when the final `NetworkToTarget` is executed. This will cost the user 10,000 TICS.

Recommendation

Consider implementing user-defined slippage and adding slippage validation when processing `NetworkToTarget` transaction.

Alleviation

[Qubetics, 12/16/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> right now the slippage condition is not the part the business requirement

QCA-110 Consensus Readiness Miscounts Broadcasts Instead Of Selected Nodes Allowing Under Or Over-Trigger Of Consensus

Category	Severity	Location	Status
Volatile Code	Medium	src/vrf/mod.rs (chain-abstraction): 506~508, 636	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/vrf/mod.rs`

Consensus gating uses the number of broadcaster entries (`received_selections.len()`) plus an optional local “+1” instead of the number of selected nodes or the VRF threshold. `min_selections` is set to `total_nodes` , which likely represents network size, not the expected number of broadcasts or selected nodes. This can start consensus with too few selected nodes(if current active nodes number are small) or block consensus (if `total_nodes` exceeds expected broadcasters).

```
//483:509:src/vrf/mod.rs
pub fn handle_selected_nodes_broadcast(&mut self, broadcast:
VrfSelectedNodesBroadcast, total_nodes: usize) {
    ...
    let min_selections = total_nodes; // Minimum 4 selections to start consensus
    if self.can_start_consensus(round, min_selections) {
        ...
    }
}
```

```
//617:637:src/vrf/mod.rs
pub fn can_start_consensus(&self, round: u64, min_selections: usize) -> bool {
    let own_selection = self.results.get(&round).is_some();
    let received_count = self
        .received_selections
        .get(&round)
        .map(|selections| selections.len())
        .unwrap_or(0);

    let total_selections = if own_selection { received_count + 1 } else {
received_count };
    total_selections >= min_selections
}
```

Recommendation

- Decide the true gating criterion: number of unique selected nodes meeting VRF `threshold` (preferred) or number of distinct, valid broadcasts.
- If gating on selected nodes: compute `unique_selected.len() >= threshold` (or a configured quorum) from aggregated broadcasts; do not rely on broadcaster count.
- If gating on broadcasters: set `min_selections` to the expected broadcaster count for the round, not `total_nodes`, and still validate that aggregated selected-node cardinality meets `threshold`.

Alleviation

[Qubetics, 12/24/2025]: The team resolved the issue in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143. If a node goes down, it is removed from the active peer list, ensuring that it no longer participates in the consensus process.

QCA-111 Publicly Broadcast Signature Shares Let Any Node Assemble The Final Signature And Broadcast

Category	Severity	Location	Status
Design Issue	Medium	src/signing/mod.rs (chain-abstraction): 3151~3157	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

Signature shares are gossiped on the shared topic `signatures-to-final-node`, so every peer on the mesh receives the raw shares. Any node can collect ≥threshold shares and run the same aggregation as the designated final node.

```
//3127:3176:9d3440fe169a63244ca5f3080e36096e090865d6/src/signing/mod.rs
// Broadcast to a specific topic that the final node subscribes to
let res: std::result::Result<(), anyhow::Error> = self
    .broadcast("signatures-to-final-node", &signature_data)
    .await;
```

```
//1187:1213:9d3440fe169a63244ca5f3080e36096e090865d6/src/network/mod.rs
if message.topic.to_string() == "signatures-to-final-node" {
    match signing_msg {
        SigningMessage::ECDSASignature { from, signature, round, .. } => {
            signing_node.handle_signature_to_final_node(from, signature,
round).await
        }
        ...
    }
}
```

- The only guard is a local check of “am I the final node?” before aggregating; other nodes still receive all shares and can aggregate off-path:

```
3924:3955:9d3440fe169a63244ca5f3080e36096e090865d6/src/signing/mod.rs
if is_final_node { return self.handle_signature_for_round(from, signature,
round).await; }
info!("Received signature ... but this node is not the final node");
```

Impact:

- Any participant or observer on the gossip mesh can reconstruct the recoverable signature and broadcast the transaction before (or instead of) the elected final node, enabling reward theft/credit hijack and making the final-node election moot. Multiple broadcasts can also cause race/grief scenarios.

Recommendation

- Stop gossiping signature shares to a public topic. Send them directly to the elected final node over an authenticated/encrypted channel (e.g., libp2p secured direct stream) or encrypt shares to the final node's key if broadcast transport is required.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue by sending unicast messages in commit 94b97730ac62b858126080cb850159a6cffab67f

QCA-112 Invalid Or Wrong-Message Signature Shares Can Block Aggregation (DoS)

Category	Severity	Location	Status
Denial of Service	Medium	src/signing/mod.rs (chain-abstraction): 3300~3313, 3473	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

`aggregate_signatures` in signing node consumes whatever shares are handed to it, assuming they are correct for the intended message. Because `handle_signature_for_round()` never verifies that each `(r,s)` share signs the round's digest under the group/share pubkey before interpolation, any malformed or wrong-message share can be accepted. One bad-but-well-formed share can cause `aggregate_signatures` to fail, the round can be stuck.

```
//2190:2244:9d3440fe169a63244ca5f3080e36096e090865d6/src/signing/mod.rs
pub async fn aggregate_signatures(
    &self,
    signatures: Vec<RecoverableSignature>,
    peer_to_dkg_index: Vec<(String, u32)>,
) -> Result<RecoverableSignature> {
    let n_shares = signatures.len();
    if n_shares < self.threshold { return Err(anyhow!(...)); }

    // r, recid taken from first share; no per-share validation
    let (first_recid, first_compact) = { ... };
    let agg_r = first_compact[0..32].to_vec();

    // Build (index, s_bytes) from all shares; no message/hash check
    let s_shares: Vec<(u32, Vec<u8>)> = signatures
        .into_iter()
        .zip(peer_to_dkg_index.iter())
        .map(|(sig, (peer_id, dkg_index))| {
            let (_recid, compact) = sig.serialize_compact();
            (*dkg_index, compact[32..64].to_vec())
        })
        .collect();

    // Lagrange interpolate s; errors abort aggregation
    let mut agg_s = self.lagrange_interpolate(&s_shares)?;
```

Recommendation

- Before calling `aggregate_signatures`, verify every share against the expected message/tx hash and the signer's share pubkey (or group pubkey) and reject/evict invalid shares.
- If more than `threshold` shares exist, on aggregation failure drop the offending share(s) and retry with another subset rather than always using the first `threshold`.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue by verifying signature against partial public key in commit 66a2f399e91d0ea43a5fd9699e0ff0e7e1848211

QCA-19 | The `broadcast_transaction` May Return Incorrect `txid`

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 609, 3549~3561, 3905~3917	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The code has a potential issue where the `broadcast_transaction` function may return `Ok(Some("broadcast_success".to_string()))` when it fails to extract a transaction ID (`txid`) from the response, despite the transaction being successfully broadcast. This could lead to the calling code mistakenly storing the string "broadcast_success" as if it were a real transaction ID in the database.

The current implementation:

`src/signing/mod.rs`

```
604  if let Some(txid) = txid_opt {
605      info!("✅ Transaction broadcasted successfully! TXID: {}", txid);
606      Ok(Some(txid))
607  } else {
608
    info!("✅ Transaction broadcasted successfully, but no txid found in response:
{}", body_trimmed);
609      Ok(Some("broadcast_success".to_string()))
610  }
```

It returns `Ok(Some(txid))` when txid is successfully extracted, this is correct. However, it returns `Ok(Some("broadcast_success"))` when broadcast succeeds but no txid is found.

The following code will call `broadcast_transaction` and store the `txid` :

`src/signing/mod.rs`

```
3549  match Self::broadcast_transaction(&signed_tx_hex, false).await {
3550      Ok(Some(txid)) => {
3551
        info!("✅ [SIGNING] Bitcoin transaction broadcasted successfully: {}",
txid);
3552
        self.store_transaction_id_for_round(round, &txid,
signature.user_eth_address.clone(), crate::types::TransactionNetwork::BTC).await;
3553      },
3554      Ok(None) => {
3555
        warn!("⚠️ [SIGNING] Bitcoin transaction broadcasted but no txid returned");
3556      },
3557      Err(e) => {
3558
        error!("❌ [SIGNING] Failed to broadcast Bitcoin transaction: {}", e);
3559
        self.store_transaction_error_for_round(round, &e.to_string(),
crate::types::TransactionNetwork::BTC).await;
3560      }
3561  }
```

`src/signing/mod.rs`

```
3905  match Self::broadcast_transaction(&raw_hex, is_testnet).await {
3906          Ok(Some(txid)) => {
3907
            info!("✅ [BTC] Bitcoin transaction broadcasted successfully: {}",
txid);
3908
            self.store_transaction_id_for_round(round, &txid, user_eth_address,
crate::types::TransactionNetwork::BTC).await;
3909          },
3910          Ok(None) => {
3911
            warn!("⚠️ [BTC] Bitcoin transaction broadcasted but no txid returned");
3912          },
3913          Err(e) => {
3914
            error!("❌ [BTC] Failed to broadcast Bitcoin transaction: {}", e);
3915
            self.store_transaction_error_for_round(round, &e.to_string(),
crate::types::TransactionNetwork::BTC).await;
3916          }
3917      }
```

The string "broadcast_success" is being treated as a valid txid and be stored in database after calling `store_transaction_id_for_round`. The `Ok(None)` case is never used despite being a possible return variant.

Recommendation

Consider following fix:

```
if let Some(txid) = txid_opt {
    info!("✅ Transaction broadcasted successfully! TXID: {}", txid);
    Ok(Some(txid))
} else {

   info!("✅ Transaction broadcasted successfully, but no txid found in response:
{}", body_trimmed);
    Ok(None)
}
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 588bf41dd3f94cf1903914b3b22fd61c17edd1db.

QCA-20 Hardcoded Infura URL With Placeholder `YOUR_PROJECT_ID` Will Cause Runtime Failures

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 813, 860	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `fetch_ethereum_nonce` and `broadcast_ethereum_transaction` functions contain hardcoded Infura RPC endpoints (for Ethereum mainnet, chain_id=1) with an invalid placeholder (`YOUR_PROJECT_ID`) instead of a dynamically configured API key. This violates security best practices and will cause runtime failures when interacting with Ethereum mainnet.

The code explicitly defines:

`src/signing/mod.rs`

```
 810  let rpc_url = match chain_id {
 811              9029 => std::env::var("MPC_ETHEREUM_RPC_URL")
 812
                .unwrap_or_else(|_| "https://rpc-
testnet.qubetics.work".to_string()),
 813
            1 => "https://mainnet.infura.io/v3/YOUR_PROJECT_ID".to_string(), //
Ethereum mainnet
 814
            _ => return Err(anyhow!("Unsupported chain ID for nonce fetching: {}",
chain_id)),
 815          };
```

`src/signing/mod.rs`

```
857  let rpc_url = match chain_id {
858              9029 => std::env::var("MPC_ETHEREUM_RPC_URL")
859
               .unwrap_or_else(|_| "https://rpc-
testnet.qubetics.work".to_string()),
860
           1 => "https://mainnet.infura.io/v3/YOUR_PROJECT_ID".to_string(), //
Ethereum mainnet
861
           _ => return Err(anyhow!("Unsupported chain ID for broadcasting: {}",
chain_id)),
862         };
```

For `chain_id=1` (mainnet), it hardcodes the URL to `https://mainnet.infura.io/v3/YOUR_PROJECT_ID`. This will always fail because `YOUR_PROJECT_ID` is a placeholder, not a valid Infura API key. Infura requires authentication via a project-specific API key.

Recommendation

Consider using an environment variable, for example:

```
let rpc_url =match chain_id {
        9029 => Ok(std::env::var("MPC_ETHEREUM_RPC_URL")
            .unwrap_or_else(|_| "https://rpc-testnet.qubetics.work".to_string())),
        1 => Ok(std::env::var("INFURA_ETHEREUM_RPC_URL")?),
        _ => Err(anyhow!("Unsupported chain ID: {}", chain_id)),
    }
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit ddbd8d2a36cd3d41b86988102d713a76c61e7d87.

QCA-21 | The `create_script_sig_for_input` Does Not Return Valid Redeem Script For P2SH-Wrapped SegWit

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 1461~1464	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The current implementation of `create_script_sig_for_input` correctly handles native SegWit inputs (`P2WPKH` , `P2WSH` , `P2TR`) by returning an empty `scriptSig` , but it incorrectly defaults to an empty scriptSig for all other `SegWit` cases. This is problematic for P2SH-wrapped SegWit inputs (the most common SegWit migration path), which require their redeem script to be placed in the scriptSig field.

The current implementation:

`src/signing/mod.rs`

```
 1446  if let Some(witness_utxo) = &input.witness_utxo {
 1447
           // For SegWit inputs, script_sig is usually empty or contains redeem
script
 1448              match witness_utxo.script_pubkey.len() {
 1449
               22 if witness_utxo.script_pubkey[0] == 0x00 &&
witness_utxo.script_pubkey[1] == 0x14 => {
 1450                      // P2WPKH - empty script_sig
 1451                      Ok(vec![])
 1452                  },
 1453
               34 if witness_utxo.script_pubkey[0] == 0x00 &&
witness_utxo.script_pubkey[1] == 0x20 => {
 1454                      // P2WSH - empty script_sig
 1455                      Ok(vec![])
 1456                  },
 1457
               34 if witness_utxo.script_pubkey[0] == 0x51 &&
witness_utxo.script_pubkey[1] == 0x20 => {
 1458                      // P2TR - empty script_sig
 1459                      Ok(vec![])
 1460                  },
 1461                  _ => {
 1462                      // P2SH-wrapped SegWit - need redeem script
 1463                      Ok(vec![])
 1464                  }
 1465              }
 1466          }
```

The default case assumes all unrecognized SegWit inputs are P2SH-wrapped but fails to include the required redeem script. For `P2SH-P2WPKH` or `P2SH-P2WSH` inputs, the scriptPubKey falls into the `_` branch, yet the function still returns an empty `scriptSig`. Consensus rules require pushing the redeem script (`0x0014…` or `0x0020…`) onto the stack; without it, script evaluation fails immediately.

Impact:

Any PSBT containing P2SH-wrapped SegWit UTXOs signed by this code will be rejected by every Bitcoin node. If the protocol ever locks assets into such outputs (common for hardware-wallet compatibility or legacy address support), the funds cannot be swept, creating a denial-of-service/funds-lock scenario.

Recommendation

Consider implementing redeem script logic for P2SH-wrapped SegWit or returning error if you don't want to support P2SH-wrapped SegWit.

Alleviation

[Qubetics, 12/09/2025]: The team resolved the issue by refactoring the code to only support `P2PKH` and `P2WPKH` in

commit ebe6b8438b972a8fcddc99f294bf6df21c2ffce2.

QCA-22 | The `create_witness_stack_for_input` Function Incorrectly Assumes All `SegWit` Inputs Are `P2WPKH`

Category	Severity	Location	Status
Inconsistency	Medium	src/signing/mod.rs (chain-abstraction): 1487~1493	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `create_witness_stack_for_input` function currently assumes all `SegWit` inputs are `P2WPKH` (Pay-to-Witness-Public-Key-Hash), constructing a witness stack as `[signature, pubkey]`. However, Bitcoin supports multiple SegWit input types, including `P2WSH` (Pay-to-Witness-Script-Hash) and `P2TR` (Pay-to-Taproot), each requiring different witness stack formats.

The current implementation:

`src/signing/mod.rs`

```
 1485
async fn create_witness_stack_for_input(&self, input: &BitcoinInput, signature: &
[u8], transaction_type: &Option<crate::types::TransactionType>, user_eth_address:
Option<String>) -> Result<Vec<Vec<u8>>> {
 1486          // All inputs are P2WPKH, so they all have witness UTXOs
 1487          if let Some(_witness_utxo) = &input.witness_utxo {
 1488              // P2WPKH witness stack: [signature, pubkey]
 1489
           let group_key_bytes =
self.get_group_public_key_bytes(transaction_type.clone(),
user_eth_address.clone()).await?;
 1490
           info!("🔍 [SIGNING] P2WPKH witness stack - signature: {} bytes, pubkey:
{} bytes",
 1491                   signature.len(), group_key_bytes.len());
 1492              Ok(vec![signature.to_vec(), group_key_bytes])
 1493          } else {
 1494
           return Err(anyhow::anyhow!("P2WPKH input must have witness UTXO"));
 1495          }
 1496      }
```

It always builds `[signature, pubkey]`, which is incorrect for `P2WSH` / `P2TR`.

Unlike `create_script_sig_for_input` function:

`src/signing/mod.rs`

```
 1448  match witness_utxo.script_pubkey.len() {
 1449
               22 if witness_utxo.script_pubkey[0] == 0x00 &&
witness_utxo.script_pubkey[1] == 0x14 => {
 1450                      // P2WPKH - empty script_sig
 1451                      Ok(vec![])
 1452                  },
 1453
               34 if witness_utxo.script_pubkey[0] == 0x00 &&
witness_utxo.script_pubkey[1] == 0x20 => {
 1454                      // P2WSH - empty script_sig
 1455                      Ok(vec![])
 1456                  },
 1457
               34 if witness_utxo.script_pubkey[0] == 0x51 &&
witness_utxo.script_pubkey[1] == 0x20 => {
 1458                      // P2TR - empty script_sig
 1459                      Ok(vec![])
 1460                  },
 1461                  _ => {
 1462                      // P2SH-wrapped SegWit - need redeem script
 1463                      Ok(vec![])
 1464                  }
 1465              }
```

it does not check `script_pubkey` format. As a result, the current implementation would lead to invalid transactions for `P2WSH` / `P2TR` inputs.

Recommendation

Consider implementing the specific logic to handle `P2WSH` / `P2TR` in `create_witness_stack_for_input` .

Alleviation

[Qubetics, 12/09/2025]: The team resolved the issue by refactoring the code to only support `P2PKH` and `P2WPKH` in commit 0d3f37476cd727b1e3485a4c258399507432c940.

QCA-23 | VRF Input Order Manipulation Allows Selection Bias

Category	Severity	Location	Status
Volatile Code	Medium	src/vrf/mod.rs (chain-abstraction): 290~301	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/vrf/mod.rs

`generate_selection_proof` hashes the `round`, the local `node_id`, and the bytes of available_nodes exactly as provided. Because upstream callers do not canonicalize that list—and it often originates from unordered collections, a malicious selector can reorder `available_nodes` arbitrarily before invoking the VRF. Different permutations lead to different hash inputs, letting the adversary iterate through permutations until the derived VRF value ranks within the target threshold. Honest peers using other permutations derive incompatible results, breaking fairness and fragmenting consensus on the selected committee.

```
   pub fn generate_selection_proof(&self, round: u64, available_nodes: &[PeerId]) ->
VrfOutput {
        let mut input = Vec::new();
        input.extend_from_slice(&round.to_le_bytes());
        input.extend_from_slice(&self.node_id.to_bytes());

        // Include all available nodes in the input for deterministic selection
        for node in available_nodes {
            input.extend_from_slice(&node.to_bytes());
        }

        self.generate_vrf_proof(&input)
    }
```

Recommendation

Before deriving the VRF input, deterministically canonicalize the participant list (e.g., sort available_nodes by PeerId bytes or hash each ID and sort lexicographically). Include that canonical representation—or its digest—in the proof so verifiers can

rebuild the exact input prior to verification. This removes selector-controlled degrees of freedom and ensures every honest participant computes identical VRF challenges.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by removing `available_nodes` and only use round and self node_id to generate proof in commit 60a12d70dc87ee166907e7cd3833dbd3447c8155.

QCA-24 | Deadlock Risk

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 984, 1021, 1787, 1800, 1939, 1951	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

A high-probability deadlock vulnerability has been identified in the transaction signing node. The core issue stems from inconsistent lock acquisition ordering between transaction reconstruction and signing operations, creating a classic circular dependency that will inevitably lead to system deadlock under production load conditions.

The deadlock originates from diametrically opposed locking strategies between two critical code paths:

The `reconstruct_signed_ethereum_transaction` function:

```
let pending_transactions = self.pending_transactions.read().await;  // Lock #1
// ...
let user_eth_address = {
    let signatures_by_round = self.signatures_by_round.read().await;  // Lock #2
    // ...
};
```

The `sign_message` and `sign_contract_message` functions:

```
let mut signatures_by_round = self.signatures_by_round.write().await;  // Lock #2
// ...
let mut pending_transactions = self.pending_transactions.write().await;  // Lock #1
```

The deadlock occurs due to inconsistent locking order:

1. Function A (`reconstruct_signed_ethereum_transaction`):

Acquires `pending_transactions.read()` Attempts to acquire `signatures_by_round.read()`

2. Function B (`sign_message` / `sign_contract_message`):

Acquires `signatures_by_round.write()` Attempts to acquire `pending_transactions.write()`

When executed concurrently:

1. Task A holds `pending_transactions` read lock, waits for `signatures_by_round` read lock
2. Task B holds `signatures_by_round` write lock, waits for `pending_transactions` write lock
3. Result: Circular dependency → Deadlock

Recommendation

Consider applying consistent order: `signatures_by_round` FIRST, then `pending_transactions` , for example:

```
pub async fn reconstruct_signed_ethereum_transaction(&self, ...) {
    let signatures = self.signatures_by_round.read().await;      // First
    let pending = self.pending_transactions.read().await;        // Second
    // ...
}

pub async fn sign_message(&mut self, ...) {
    let mut signatures = self.signatures_by_round.write().await; // First
    let mut pending = self.pending_transactions.write().await;   // Second
    // ...
}

pub async fn sign_contract_message(&mut self, ...) {
    let mut signatures = self.signatures_by_round.write().await; // First
    let mut pending = self.pending_transactions.write().await;   // Second
    // ...
}
```

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue in commit f537c766e035aabcf9b7bc18f2ae310d58be8cab.

QCA-38 The `verify_aggregated_signature` Does Not Fully Verify The ECDSA Signature

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 2437~2481	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `verify_aggregated_signature` function does not fully verify the ECDSA signature. Instead, it only performs basic checks (signature length, presence of signer IDs), but does not cryptographically validate the signature against the message and public keys. This is a critical security risk, as an attacker could forge messages by bypassing these superficial checks.

The current implementation:

`src/signing/mod.rs`

```
 2437
async fn verify_aggregated_signature(&self, signed_message: &SignedMessage) ->
Result<bool> {
 2438         info!(
 2439             "🔍 [SIGNING] Node {} verifying aggregated signature",
 2440             self.id
 2441         );
 2442         info!(
 2443             "✍️ [SIGNING] Aggregated signature: {:?}",
 2444             signed_message.aggregated_signature
 2445         );
 2446         info!("👥 [SIGNING] Signers: {:?}", signed_message.signer_ids);
 2447         info!("🕒 [SIGNING] Timestamp: {}", signed_message.timestamp);
 2448
 2449         let public_keys = self.public_keys.read().await;
 2450
 2451         // Check if we have public keys for all signers
 2452         for signer_id in &signed_message.signer_ids {
 2453             if !public_keys.contains_key(signer_id) {
 2454
              warn!("❌ [SIGNING] Missing public key for signer: {}", signer_id);
 2455                 return Ok(false);
 2456             }
 2457         }
 2458
 2459         info!("🔍 [SIGNING] Node {} verifying ECDSA signature", self.id);
 2460
 2461         // Basic signature validation
 2462         if signed_message.aggregated_signature.len() != 65 {
 2463
           warn!("❌ [SIGNING] Invalid signature length: expected 65 bytes, got
{}", signed_message.aggregated_signature.len());
 2464             return Ok(false);
 2465         }
 2466
 2467         // Check that we have valid signer IDs
 2468         if signed_message.signer_ids.is_empty() {
 2469             warn!("❌ [SIGNING] No signer IDs provided");
 2470             return Ok(false);
 2471         }
 2472
 2473         // In a full implementation, you would:
 2474         // 1. Parse the signature components (r, s, v)
 2475         // 2. Recover the public key from the signature and message hash
 2476         // 3. Verify against the expected public keys
 2477         // 4. Check threshold requirements
 2478
 2479
       info!("✅ [SIGNING] Signature validation passed for {} signers",
signed_message.signer_ids.len());
 2480         Ok(true)
 2481     }
```

The function checks if the signature is 65 bytes (typical ECDSA length) but does not:

1. Parse the signature into (r, s, v) components.
2. Recover the signer's public key from the signature.
3. Verify that the signature matches the message hash.

Without these steps, the system cannot confirm that the message was actually signed by the claimed signers.

Recommendation

Consider implementing full ECDSA verification in `verify_aggregated_signature`.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 282ab70e8ff7cdff5143da8b0cfc80fc0cfca092.

QCA-39 Distributed K-Generation Introduces Single-Point-Of-Failure (SPoF) Due To Strict Full-Node Availability Requirement

Category	Severity	Location	Status
Design Issue	Medium	src/signing/mod.rs (nonce-k-fix)	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `8a0c09991843732c5f9ab35d366cdf834d57e177`

Files:

- `src/signing/mod.rs`

The issue was introduced in a fix version `8a0c09991843732c5f9ab35d366cdf834d57e177` .

The distributed nonce generation protocol requires all participating nodes to broadcast and contribute their local `k_i` share before `k_aggregated` can be computed:

```
if node_shares.len() < expected_nodes {
    return Err(anyhow!(
        "Insufficient k_nonce shares: got {} but expected {}",
        node_shares.len(),
        expected_nodes
    ));
}
```

`expected_nodes` is derived directly from:

```
self.get_available_nodes_from_manager().await.len()
```

This means the protocol assumes *all* nodes listed as “available” must respond. If **any single node becomes unavailable** (network partition, crash, restart, update lag), the system is unable to gather all `k_i` shares, causing the signing process to stall indefinitely (until timeout), effectively halting transaction signing and preventing system operation.

This creates a **single-point-of-failure**.

Impact

- If any node becomes unresponsive, the entire signing round fails.
- Real-world reliability issues (node dropout, rolling updates, network latency) may cause system-wide signing outages.

Recommendation

- Replace strict `n-of-n` participation requirement with a **threshold-based** scheme:
  - Require only **t-of-n** nodes to contribute `k_i` shares.
  - Use additive secret sharing or Feldman/Verifiable DKG to support missing participants.

If full t-of-n threshold signing is not immediately feasible, a short-term mitigation is to:

1. Ensure nodes are health-checked and restarted automatically.
2. Do not treat all nodes in the manager list as required; **track only active responders**.

Alleviation

[Qubetics, 12/29/2025]: The team heeded the advice and resolved the issue in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143

QCA-40 Using Default DKG Index 1 For Peer Will Generate Invalid Aggregated Signatures

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 4063~4099	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The functions `calculate_dkg_index_for_peer` and `calculate_dkg_index_for_peer_sync` default to returning 1 if a peer is not found in `available_nodes`. This could lead to duplicate DKG indices in `peer_to_dkg_index`, which breaks the correctness of Lagrange interpolation in `aggregate_signatures`, potentially causing invalid aggregated signatures or failed signature reconstruction.

The current implementation for `calculate_dkg_index_for_peer`:

`src/signing/mod.rs`

```
 4064  pub async fn calculate_dkg_index_for_peer(&self, peer_id: &str) -> u32 {
 4065          if let Some(nodes) = &self.available_nodes {
 4066
           let mut ids: Vec<String> = nodes.iter().map(|p|
p.to_string()).collect();
 4067              ids.sort();
 4068
 4069              let index = ids.iter()
 4070                  .position(|id| id == peer_id)
 4071                  .map(|i| i + 1)
 4072                  .unwrap_or(1);
 4073
 4074
           info!("🔢 [SIGNING] Calculated DKG index {} for peer {}", index,
peer_id);
 4075              index as u32
 4076          } else {
 4077
           info!("⚠️ [SIGNING] No available nodes, using default DKG index 1 for
peer {}", peer_id);
 4078              1
 4079          }
 4080      }
```

If `available_nodes` exists, it sorts the peer IDs and assigns an index based on position (`index = position + 1`). If `available_nodes` is `None` or the peer is not found, it defaults to `1` . The same logic exists in `calculate_dkg_index_for_peer_sync` .

The DKG index will be collected into `peer_to_dkg_index` :

`src/signing/mod.rs`

```
3435 let peer_to_dkg_index: Vec<(String, u32)> = {
3436       // Get the peer IDs in the same order as threshold_signatures
3437       let signatures_by_round = self.signatures_by_round.read().await;
3438       if let Some(round_signatures) = signatures_by_round.get(&round) {
3439
        let mut sorted_signatures: Vec<_> = round_signatures.iter().collect();
3440           sorted_signatures.sort_by(|a, b| a.0.cmp(b.0));
3441           let sorted_peer_ids: Vec<String> = sorted_signatures
3442               .iter()
3443               .take(self.threshold)
3444               .map(|(id, _)| (*id).clone())
3445               .collect();
3446           info!(
3447
            "🧩 [SIGNING] Selected peers (PeerId sorted) for this aggregation:
{:?}",
3448               sorted_peer_ids
3449           );
3450
3451           // Calculate DKG indices for each peer
3452           let mut dkg_indices = Vec::new();
3453           for peer_id in &sorted_peer_ids {
3454
            let dkg_index = self.calculate_dkg_index_for_peer(peer_id).await;
3455               dkg_indices.push((peer_id.clone(), dkg_index));
3456           }
3457
3458           dkg_indices
3459       } else {
3460           Vec::new()
3461       }
3462   };
```

and then used in `aggregate_signatures` function:

src/signing/mod.rs

```
3473
match self.aggregate_signatures(threshold_signatures, peer_to_dkg_index).await {
```

src/signing/mod.rs

```
2228 // Use DKG indices for signature aggregation
2229         let s_shares: Vec<(u32, Vec<u8>)> = signatures
2230             .into_iter()
2231             .zip(peer_to_dkg_index.iter())
2232             .map(|(sig, (peer_id, dkg_index))| {
2233                 let (_recid, compact) = sig.serialize_compact();
2234                 info!(
2235 
                  "🔍 [SIGNING] Peer {} using DKG index: {} for aggregation",
2236                     peer_id, dkg_index
2237                 );
2238                 (*dkg_index, compact[32..64].to_vec())
2239             })
2240             .collect();
```

In aggregate_signatures:

1. Lagrange interpolation is used to reconstruct the final `s` value from partial signatures.
2. This requires unique DKG indices for each participant.
3. If multiple peers have the same index (e.g., `1`), the interpolation fails mathematically, leading to incorrect signature aggregation.

In summary, if multiple peers are missing from `available_nodes`, they all get index = 1. When passed to `aggregate_signatures`, this causes collisions in `s_shares`, breaking interpolation.

Recommendation

Modify `calculate_dkg_index_for_peer` and `calculate_dkg_index_for_peer_sync` to return an **error** if a peer is not found.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 99e4be49566f03882a00d37f588aeb31e8e3c5c2.

QCA-41 | VRF Input Domain Not Locally Reconstructed

Category	Severity	Location	Status
Volatile Code	Medium	src/vrf/mod.rs (chain-abstraction): 259~287	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/vrf/mod.rs

`verify_vrf_proof` accepts an `input` buffer supplied by the caller and trusts it without rebuilding the canonical VRF domain (e.g., round id, sender id, sorted participant list). A malicious peer can therefore craft a valid proof over an attacker-chosen payload and still pass verification, breaking determinism of the selection protocol and enabling selective bias or equivocation across peers.

```
//259:287:src/vrf/mod.rs
    pub fn verify_vrf_proof(
        &self,
        node_public_key: &[u8],
        input: &[u8],
        output: &VrfOutput,
    ) -> bool {
        // Deserialize the public key
        let pubkey = match VrfPk::from_bytes(node_public_key.try_into().unwrap_or(&
[0; 32])) {
            Ok(pk) => pk,
            Err(_) => {
                debug!("Failed to deserialize public key");
                return false;
            }
        };

        // Deserialize the proof
        let proof = match
VrfProof::from_bytes(&output.proof.clone().try_into().unwrap_or([0; 96]))
        {
            Ok(p) => p,
            Err(_) => {
                debug!("Failed to deserialize VRF proof");
                return false;
            }
        };

        // Verify the proof
        let hash: [u8; 32] = output.hash.clone().try_into().unwrap_or([0; 32]);
        verify(input, &pubkey, &hash, &proof)
    }
```

Recommendation

Derive the VRF input locally from trusted state before verification (e.g., reconstruct `round || node_id || sorted(nodes)`), and reject proofs when the received payload does not match the locally computed value. This preserves a single, deterministic domain for the VRF and blocks adversaries from biasing selections via crafted inputs.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by constructing inputs locally in commit f8c950d8adecdfc8168ff6276fee0fc959f561fd

QCA-42 Unauthenticated First-Node Timestamp & Restart Race Break 5-Minute Join Guard

Category	Severity	Location	Status
Volatile Code	Medium	src/network/mod.rs (fix-1024): 400~404, 1006~1021, 3628~3639	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `e98d1d42332be4cd50cd56e989476ccde32f37ee`

File:

- src/network/mod.rs

The issue was introduced in a fix version `e98d1d42332be4cd50cd56e989476ccde32f37ee`.

At startup every node schedules `CheckIfFirstNode` after three seconds; if its local membership view still only contains itself, it unilaterally sets `first_node_start_time = now`, before remote peers answer (or if they never answer). Later responses are ignored because the field is already `Some(_)`.
A node that briefly restarts while isolated (intentionally or due to network delay) reopens its join window and can even distribute this fresher `t1` to newcomers, fragmenting policy across the network.

```
//401:404:src/network/mod.rs
tokio::spawn(async move {
    tokio::time::sleep(Duration::from_secs(3)).await;
    let _ = tx_clone.send(ChannelMessage::CheckIfFirstNode).await;
});
```

```
3626:3640:src/network/mod.rs
if self.first_node_start_time.is_none() && total_nodes == 1 {
    let t1 = SystemTime::now().duration_since(UNIX_EPOCH).unwrap().as_secs();
    self.first_node_start_time = Some(t1);
    info!("🕐 First node detected after bootstrap timeout (total_nodes=1) - setting
t1: {}", t1);
    if let Err(e) = self.subscribe_to_topics_with_time_check(swarm).await {
        warn!("❌ Failed to subscribe to topics: {:?}", e);
    }
}
```

Later, the node accepts the **first** `first_node_start_time` reply it sees—over RPC or gossipsub—without any authentication, sanity checking, or ability to update. A Byzantine peer can therefore pin `t1` arbitrarily far in the future (disabling the join guard forever) or in the past (permanently blocking new joins and subscriptions).

```
//1006:1020:src/network/mod.rs
RPCRequest::ShareFirstNodeStartTime { first_node_start_time } => {
    ...
    if self.first_node_start_time.is_none() {
        self.first_node_start_time = Some(first_node_start_time);
        info!("✅ Synced first node start time (t1) to: {}", first_node_start_time);
    }
    RPCResponse::Ack
},
```

Because the first value “wins,” the node never recovers without a restart, and the 5-minute guard becomes meaningless under attacker control.

Recommendation

Defer setting `first_node_start_time` until the node has collected trustworthy peer information, and authenticate/sanity-check any remote timestamps. Consider deriving `t1` from an authenticated quorum , bounding acceptable skew, and allowing corrections if contradictory but credible data arrives. Avoid granting a fresh window on self-observation alone—wait for peer responses or explicit consensus before updating the guard.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit 282ab70e8ff7cdff5143da8b0cfc80fc0cfca092

QCA-43 Missing Challenge And Verification In VRF Implementation Leads To Forged Or Unverified Node Selections

Category	Severity	Location	Status
Design Issue	Medium	src/vrf/mod.rs (chain-abstraction): 259~301	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/vrf/mod.rs`

1. The function `generate_vrf_proof` produces a VRF proof using `prove(input, &self.vrf_secret_key)` and derives a `value` by taking the first 8 bytes of `hash` as little-endian. There is no explicit **challenge function** or domain separation applied to the input (e.g. no `domain || round || purpose` concatenation) before proving, and no canonical expansion from the VRF output into an unbiased selection value.

2. The function `verify_vrf_proof` performs proper deserialization and calls `verify(input, &pubkey, &hash, &proof)`, but it appears to be used **only in tests** and not invoked in the live protocol path. As a result the runtime selection/leader-election code may accept VRF outputs without verifying proofs or without binding the proof to a specific challenge/context.

3. The combined effect is that malicious or buggy nodes can (a) produce proofs for inputs that are not canonical (replay across contexts), (b) submit forged or invalid proofs that are never verified by honest peers, or (c) bias selection by manipulating how `value` is derived from the `hash`. This undermines liveness and fairness of leader/node selection and can enable targeted attacks on the selection process.

Scenario

1. Alice implements a node that emits VRF outputs for selection. She calls `generate_vrf_proof(&input)` which returns a `VrfOutput` whose `value` is derived from the first 8 bytes of `hash`.
2. Bob (attacker) crafts a forged `VrfOutput` where `hash` (and thus `value`) is manipulated so his node appears favorable for selection; because the production code does not call `verify_vrf_proof` on received proofs, Bob's forged output is accepted.

3. Bob is repeatedly elected and can bias protocol outcomes (e.g., be selected as aggregator/broadcaster), causing denial-of-service or malicious aggregation behavior.

Recommendation

1. **Add a Canonical Challenge / Domain Separation** — Derive the VRF input using a clear challenge function that binds context, e.g. `challenge = H(domain || round || purpose || input)` before calling `prove`. This prevents replay across domains and ties proofs to a specific round and purpose.

2. **Verify Proofs In Production Path** — Ensure `verify_vrf_proof` (or inline equivalent) is invoked on every received `VrfOutput` before accepting the `value`. Do **not** rely on test-only verification. Reject and penalize peers that submit invalid proofs.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by adding a verifying function in commit f8c950d8adecdfc8168ff6276fee0fc959f561fd

QCA-52 The DKG Protocol Will Be Unnecessarily And Repeatedly Triggered During Each Peer Connection Events

Category	Severity	Location	Status
Logical Issue	Medium	src/network/mod.rs (chain-abstraction): 665~692	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`
- `src/node_management/vrf_manager.rs`
- `src/node_management/mod.rs`

The current implementation contains a logical flaw where the Distributed Key Generation (DKG) protocol may be unnecessarily and repeatedly triggered during peer connection events. This occurs because the `threshold` calculation creates a condition that's always `true` when sufficient peers are connected, leading to potential system instability and resource waste.

The current implementation of `node_manager` : **`src/node_management/vrf_manager.rs`**

```
168 fn get_total_nodes(&self) -> usize {
169         self.base_nodes.len()
170     }
171
172   fn get_threshold(&self) -> usize {
173       self.threshold_calculator
174           .calculate_threshold(self.get_total_nodes())
175   }
```

`src/node_management/vrf_manager.rs`

```
53 impl ThresholdCalculator for DefaultThresholdCalculator {
54     fn calculate_threshold(&self, _total_nodes: usize) -> usize {
55         _total_nodes/2 + 1
56     }
57
58     fn clone_box(&self) -> Box<dyn ThresholdCalculator> {
59         Box::new(self.clone())
60     }
61 }
```

The `threshold` is calculated by the `_total_nodes`. In `SwarmEvent::ConnectionEstablished`, the DKG will be started only when the following condition is true:

`src/network/mod.rs`

```
665 let total = self.node_manager.get_total_nodes();
666 let threshold = self.node_manager.get_threshold();
667
668 if total >= threshold + 1 {
```

This condition always evaluates to `true` when `3` or more peers are connected. That means the DKG will be unnecessarily and repeatedly triggered during peer connection events.

Impact

1. Potential conflicts between different DKG instances
2. Possible race conditions in key generation
3. Increased chance of inconsistent shared key states across nodes

Recommendation

Consider starting DKG only once during peer discovering period in the P2P network.

Alleviation

[Qubetics, 12/08/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> With every new node connection, we trigger the DKG process because it is required to redistribute the network's private key whenever a new node joins, if we don't do that new node wouldn't be able to participate in the network transaction signing process.

QCA-56 | Inconsistent Threshold Calculation In DKG Start Methods

Category	Severity	Location	Status
Inconsistency	Medium	src/network/mod.rs (chain-abstraction): 779~814, 2926~2949	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

There are two different methods for starting Distributed Key Generation (DKG) in network mod:

1. `ScheduledDKGStart` (triggered automatically at a scheduled time)
2. `RPCRequest::StartADKG` (triggered via an RPC call from a peer)

The issue here is that there is inconsistent threshold calculation in these two methods.

In `RPCRequest::StartADKG`:

`src/network/mod.rs`

```
 791  if let Some(signing_node) = &mut self.signing_node {
 792      signing_node.set_threshold(self.node_manager.get_total_nodes() - 1);
 793  }
 794  self.node_manager
 795      .set_threshold(total - 1);
 796
info!("🔄 DKG parameters updated: total_nodes={}, threshold={}", total, threshold);
```

The threshold is calculated as `total_nodes - 1`, which ensures only fully validated setups proceed (tolerates only one failure).

In `ScheduledDKGStart`:

`src/network/mod.rs`

```
2926  ChannelMessage::ScheduledDKGStart => {
2927                  let current_time = SystemTime::now()
2928                      .duration_since(UNIX_EPOCH)
2929                      .unwrap()
2930                      .as_secs();
2931
2932                  let total = self.node_manager.get_total_nodes();
2933                  let threshold = self.node_manager.get_threshold();
2934                  if let Some(dkg) = &mut self.dkg_node {
2935                      dkg.update_node_params(total, threshold);
2936                      dkg.reset().await;
2937                      if let Some(signing_node) = &mut self.signing_node {
2938                          signing_node.set_threshold(total/2 + 1);
2939                      }
2940                      self.node_manager
2941                          .set_threshold(total/2 + 1);
2942
                  info!("🔄 DKG parameters updated: total_nodes={}, threshold={}",
total, threshold);
2943
                  if let Err(e) = dkg.start_dkg(self.node_manager.clone()).await {
2944                          warn!("❌ Scheduled DKG start failed: {:?}", e);
2945                      }
2946                  } else {
2947                      warn!("❌ DKG node not initialized at scheduled start!");
2948                  }
2949              }
```

The threshold is calculated as `total_nodes / 2 + 1`, which is common in Byzantine Fault Tolerant (BFT) systems.

Since both methods are used in the same system, they may produce incompatible signatures due to different threshold requirements because `total - 1` (used in RPC mode) is much stricter than `total / 2 + 1` (used in scheduled mode), which could cause availability problems (DKG may fail if even one node is offline).

Impact

1. Inconsistent signing behavior: Some signatures may be accepted in one mode but rejected in another.
2. DKG may fail under total - 1: If even one node is offline, RPC-based DKG may not complete.

Recommendation

Consider using consistent threshold calculation in both two methods, in `RPCRequest::StartADKG`:

```
let total = self.node_manager.get_total_nodes();
let threshold = self.node_manager.get_threshold();
dkg.update_node_params(total, threshold);
dkg.reset().await;
if let Some(signing_node) = &mut self.signing_node {
    signing_node.set_threshold(threshold);
}
// the threshold will be calculated by ThresholdCalculator when calling
get_threshold(), so comment this code
// self.node_manager.set_threshold(total - 1);
info!("🔄 DKG parameters updated: total_nodes={}, threshold={}", total, threshold);
```

In `ScheduledDKGStart`:

```
let total = self.node_manager.get_total_nodes();
let threshold = self.node_manager.get_threshold();
if let Some(dkg) = &mut self.dkg_node {
      dkg.update_node_params(total, threshold);
      dkg.reset().await;
      if let Some(signing_node) = &mut self.signing_node {
          // use threshold directly
          signing_node.set_threshold(threshold);
      }
      // the threshold will be calculated by ThresholdCalculator when calling
get_threshold(), so comment this code
      //self.node_manager.set_threshold(total/2 + 1);
      info!("🔄 DKG parameters updated: total_nodes={}, threshold={}", total,
threshold);
      if let Err(e) = dkg.start_dkg(self.node_manager.clone()).await {
          warn!("❌ Scheduled DKG start failed: {:?}", e);
      }
  }
```

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit 75b3ac44e94d7e6a4fa01ed1fd6ff97d8d7182c7.

QCA-57 | Node Count Increase May Cause DKG Generation Failure

Category	Severity	Location	Status
Logical Issue	Medium	src/adkg_secp256k1/mod.rs (chain-abstraction): 592~594	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The identified issue reveals a critical inconsistency in the Distributed Key Generation (DKG) protocol implementation where `generate_shares()` uses the initially configured `self.total_nodes` while `check_completion()` dynamically queries the current node count via `node_manager.get_total_nodes()`. This discrepancy can cause DKG generation failure if the node count increases during DKG execution.

The `generate_shares()` creates with `self.total_nodes` shares:

`src/adkg_secp256k1/mod.rs`

```
245  fn generate_shares(&self, polynomial: &[Scalar]) -> Result<Vec<KeyShare>> {
246          info!("🔍 [DKG] === SECRET SHARE GENERATION ===");
247
       info!("📊 [DKG] Node {} generating shares for {} nodes", self.id,
self.total_nodes);
248          info!(
249              "🔢 [DKG] Node {} polynomial coefficients: {:?}",
250              self.id, polynomial
251          );
252
253          let mut shares = Vec::with_capacity(self.total_nodes);
```

While `check_completion()` requires N^2 validations where `N = node_manager.get_total_nodes()`:

`src/adkg_secp256k1/mod.rs`

```
592
if valid_validations >= (node_manager.get_total_nodes()) *
(node_manager.get_total_nodes())
593             && shares_len >= node_manager.get_total_nodes()
594         {
595             info!(
596
               "✅ [DKG] Required validations: {} and valid validations: {}",
(node_manager.get_total_nodes()) *
(node_manager.get_total_nodes()),valid_validations
597             );
598
           info!("🎉 [DKG] DKG completion criteria met! Generating final keys...");
599             self.generate_group_key(node_manager).await?;
600         }
```

If the `node_manager.get_total_nodes()` result increases, the `if` condition in `check_completion()` may never meet and the `generate_group_key` will never be called.

impact

1. The protocol may permanently stall despite having sufficient participants
2. Adversarial nodes could join to intentionally block completion

Scenario

Initial Condition: 3 nodes (A,B,C)

- Shares generated: 3 (for A,B,C)
- Expected validations: 9 (3×3)

After Node Join: 4 nodes (A,B,C,D)

- Required validations: 16 (4×4)
- Available validations: 9 (from initial 3 nodes)
- Result: DKG never completes

Recommendation

Consider using `self.total_nodes` in `check_completion` function.

Alleviation

[Qubetics, 12/25/2025]: The team heeded the advice and resolved the issue by implementing **proactive resharing**, where the shares of each node are refreshed whenever a new node joins or an existing node leaves the network in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143.

QCA-65 Exposure Of HMAC Constant In User Info Could Allow An Attacker To Reverse-Engineer The Original `dkg_secret_share`

Category	Severity	Location	Status
Logical Issue	Medium	src/rpc_server/mod.rs (chain-abstraction): 2068	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`
- `src/user_registry/db_registry.rs`

The `handle_get_user_info` function exposes the `hmac_constant` in its response, which—when combined with a leaked `tweaked_secret_share`—could allow an attacker to reverse-engineer the original `dkg_secret_share`. This poses a security risk to the Distributed Key Generation (DKG) scheme, as the secrecy of individual shares is critical for threshold cryptography.

The endpoint returns the full `hmac_constant` as a hex-encoded string in the `UserInfoResponse`:

`src/rpc_server/mod.rs`

```
 2059  match user_registry.get_user_by_address(&addr_cs).await {
 2060          Some(registration) => {
 2061              info!("✅ HTTP: Found user info for {}", addr_cs);
 2062
 2063              // Successful response
 2064              let response = UserInfoResponse {
 2065                  status: "success".to_string(),
 2066                  user_id: Some(registration.user_id),
 2067                  ethereum_address: Some(registration.ethereum_address),
 2068                  hmac_constant: Some(hex::encode(registration.hmac_constant)),
 2069
               has_tweaked_share:
Some(registration.tweaked_secret_share.is_some()),
 2070
               has_user_group_key: Some(registration.user_group_key.is_some()),
 2071                  derived_eth_address: registration.derived_eth_address,
 2072                  derived_btc_address: registration.derived_btc_address,
 2073                  created_at: Some(registration.created_at.to_rfc3339()),
 2074                  error: None,
 2075              };
 2076
 2077              Ok(warp::reply::json(&response))
 2078          }
```

The `hmac_constant` is used to calculate the user's `tweaked_secret_share`:

src/user_registry/db_registry.rs

```
 110  if let Some(share) = dkg_secret_share {
 111
    let tweaked_share = self.compute_tweaked_share(share,
&registration.hmac_constant);
 112      registration.tweaked_secret_share = Some(tweaked_share);
 113  }
```

src/user_registry/db_registry.rs

```
 410  fn compute_tweaked_share(
 411          &self,
 412          share: &Scalar,
 413          hmac_constant: &[u8; 32],
 414      ) -> SerializableScalar {
 415          let hmac_scalar = Scalar::from_repr((*hmac_constant).into()).unwrap();
 416          let tweaked_scalar = *share + hmac_scalar;
 417          SerializableScalar(tweaked_scalar)
 418      }
```

If the `tweaked_secret_share` is leaked, the attack can contains both `tweaked_secret_share` and `hmac_constant` and is able to recover the original `dkg_secret_share` by following calculation:

```
dkg_secret_share=tweaked_scalar−hmac_scalar
```

If an adversary recovers even one `dkg_secret_share`, they may compromise the security guarantees of the scheme (e.g., reconstruct the master private key if enough shares are leaked).

Recommendation

Consider removing `hmac_constant` from Response.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-66 The Solver Reward Distribution Logic Completely Omits BTC Reward

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 4352~4357	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`
- `src/network/mod.rs`

The current implementation exhibits an inconsistency in reward distribution handling. While the `calculate_reward_per_solver` function properly calculates and stores both `BTC` and `TICS` (EVM chain) rewards, the subsequent message processing logic (`ChannelMessage::SolverReward` and `GossipsubMessage::SolverReward`) only implements distribution for `TICS` rewards on the EVM chain, completely omitting `BTC` reward distribution.

The `calculate_reward_per_solver` function calculates and stores both `BTC` and `TICS` (EVM chain) rewards:

`src/signing/mod.rs`

```
 4248
pub async fn calculate_reward_per_solver(&self, intent_hash: &str) -> Result<()> {
 4249         ...
 4250         ...
 4251
 4252         for (solver, amounts) in solver_liquidity_map.iter() {
 4253             ...
 4254             ...
 4255
 4256             if source_chain == "btc" || source_chain == "bitcoin" {
 4257
               let _ = user_registry.set_solver_reward_btc(solver,
updated_ethereum_types).await;
 4258             } else {
 4259
               let _ = user_registry.set_solver_reward_tics(solver,
updated_ethereum_types).await;
 4260             }
 4261
 4262
           info!("💰 [SOLVER] Reward for {} updated to {}", solver,
updated_ethers);
 4263         }
 4264
 4265         ...
 4266         ...
 4267
 4268         Ok(())
 4269     }
```

However, the logic in `ChannelMessage::SolverReward` and `GossipsubMessage::SolverReward` only implements distribution for evm rewards:

`src/network/mod.rs`

```
2773  ChannelMessage::SolverReward { solver_address, reward } => {
2774                  ...
2775                  ...
2776
2777                  // Use fetch_ethereum_nonce from transaction utils
2778
              let nonce =
crate::utils::transaction::fetch_ethereum_nonce(&from_address, chain_id).await?;
2779
2780                  let ethereum_tx = EthereumTransaction {
2781                      to: solver_address.clone(),
2782                      value: format!("0x{:x}", reward),
2783                      gas_limit: 300000,
2784                      gas_price: get_default_gas_price_for_chain(chain_to_use),
2785                      nonce,
2786                      data: None,
2787                      chain_id,
2788                  };
2789
2790                  ...
2791                  ...
2792              }
```

`src/network/mod.rs`

```
1560  GossipsubMessage::SolverReward { solver_address, reward } => {
1561                  ...
1562                  ...
1563
1564                  // Use fetch_ethereum_nonce from transaction utils
1565
              let nonce =
crate::utils::transaction::fetch_ethereum_nonce(&from_address, chain_id).await?;
1566
1567                  let ethereum_tx = EthereumTransaction {
1568                      to: solver_address.clone(),
1569                      value: format!("0x{:x}", reward),
1570                      gas_limit: 300000,
1571                      gas_price: get_default_gas_price_for_chain(chain_to_use),
1572                      nonce,
1573                      data: None,
1574                      chain_id,
1575                  };
1576
1577                  ...
1578                  ...
1579              }
```

There is no BTC transaction construction or signing logic. As a result, BTC rewards remain "virtual" - calculated but never paid out. This creates maintenance debt as BTC support appears implemented but isn't.

Recommendation

Consider implementing reward distribution logic for BTC.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-67 Premature Loop Termination Leads To Incomplete Address Derivation Waiting

Category	Severity	Location	Status
Logical Issue	Medium	src/rpc_server/mod.rs (chain-abstraction): 1014~1040	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The logic inside this loop is intended to wait until a user's `derived_eth_address` and `derived_btc_address` become available in the registry. The function repeatedly queries `user_registry.get_user_by_address(&addr_cs)` and only stops once both derived addresses are populated or the maximum number of attempts is reached.

```
1014                while attempts < MAX_ATTEMPTS {
1015                    // Check if derived addresses are now available
1016                    match user_registry.get_user_by_address(&addr_cs).await {
1017                        Some(updated_user) => {
1018                            if updated_user.derived_eth_address.is_some()
1019                                && updated_user.derived_btc_address.is_some()
1020                            {
1021                                final_registration = updated_user;
1022                                info!("✅ Derived addresses are now available!");
1023                                break;
1024                            }
1025                        }
1026                        None => {
1027
                             error!("❌ User disappeared from registry during address
derivation");
1028                            break;
1029                        }
1030                    }
1031
1032                    attempts += 1;
1033                    if attempts < MAX_ATTEMPTS {
1034                        info!(
1035
                         "⏳ Attempt {}/{}: Derived addresses not ready yet, waiting
1 second...",
1036                            attempts, MAX_ATTEMPTS
1037                        );
1038
                     tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
1039                    }
1040                }
```

However, when the registry returns `None`, the code immediately executes `break`, exiting the loop prematurely. This contradicts the intended behavior—continuing to wait until the derived addresses are ready—and may cause downstream logic to use an incomplete or outdated `final_registration`, leading to inconsistent user state or failures in follow-up operations that assume derived addresses exist.

Recommendation

Remove the `break` inside the `None` branch so the loop continues waiting for the registry to stabilize. Instead of breaking, log the event and allow subsequent attempts to retry. For example:

```
match user_registry.get_user_by_address(&addr_cs).await {
    Some(updated_user) => {
        if updated_user.derived_eth_address.is_some()
            && updated_user.derived_btc_address.is_some()
        {
            final_registration = updated_user;
            info!("✅ Derived addresses are now available!");
            break;
        }
    }
    None => {
        warn!("⚠️ User temporarily missing from registry, retrying...");
        // do not break; continue waiting
    }
}
```

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue as we recommended in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-68 Unsupported Chain Fallback Leads To Incorrect Chain Handler Usage

Category	Severity	Location	Status
Logical Issue	Medium	src/network/mod.rs (chain-abstraction): 1776~1778, 2376~2378	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

1. The code intends to choose and install a chain handler on `signing_node` based on `chain_to_use` (EVM chains use an Ethereum handler; `"bitcoin"` / `"btc"` use a Bitcoin handler with different group keys depending on `transaction_type`).

```
 1776                    } else {
 1777
                     warn!("❌ [GOSSIPSUB] Unsupported chain: {}, using default
Ethereum handler", chain_to_use);
 1778                    }
```

2. If `chain_to_use` is unsupported the code only emits a `warn!` and does not set, clear, or return — leaving whatever handler was **set earlier (the "last one set") active**. This implicit fallback is dangerous because it silently continues execution using a handler that does not match the requested chain.
3. The impact includes sending transactions to the wrong chain, using incorrect signing/serialization logic or group keys, exposing keys to incorrect handlers, producing invalid or mis-signed transactions, and ultimately causing funds loss or failed/undetected operations.

Recommendation

Do not silently fall back to the previously set handler. Choose one of these safer approaches and apply consistently:

- **Fail Fast:** Return an error or `Err(...)` when `chain_to_use` is unsupported, so the caller must handle the condition instead of continuing. Example: `return Err(anyhow!("Unsupported chain: {}", chain_to_use));`.

- **Explicit Default:** If a default handler is acceptable, explicitly create and set a clearly documented default handler (e.g., a read-only/no-op handler) instead of reusing a stale one.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue as we recommended in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-69 | Using Non-Reduced HMAC As Scalar Leads To Panic

Category	Severity	Location	Status
Design Issue	Medium	src/adkg_secp256k1/mod.rs (chain-abstraction): 730~739	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

1. The code derives a vault tweak by computing `vault_hmac = hmac256_from_addr(&eth_address, &self.chain_code)` and then constructs a scalar with `k256::Scalar::from_repr(vault_hmac.into()).unwrap()`. This value is intended as the tweak scalar for DKG/vault key derivation.

```
730         // Derive vault addresses using the same flow as user registration
731         let vault_hmac = hmac256_from_addr(&eth_address, &self.chain_code)
732             .map_err(|e| anyhow!("Failed to generate vault HMAC: {}", e))?;
733 
734         info!("🏦 [DKG] === VAULT GENERATION STARTED ===");
735         info!("🏦 [DKG] Main ETH address: {}", eth_address);
736         info!("🏦 [DKG] Vault HMAC: {:?}", vault_hmac);
737 
738         let tweak_scalar = k256::Scalar::from_repr(vault_hmac.into()).unwrap();
739         info!("🏦 [DKG] Vault tweak scalar: {:?}", tweak_scalar);
```

2. `k256::Scalar::from_repr` expects a canonical scalar representation (i.e., a 32-byte big-endian integer strictly less than the curve order). A raw 32-byte HMAC is uniformly distributed in [0, 2^256) and may be ≥ curve order, causing `from_repr` to return `None`. Calling `.unwrap()` on `None` will panic. Even if no panic occurs, using a non-reduced value may produce an unintended scalar that does not map uniformly into the group or breaks protocol assumptions. In `from_repr()`:

```
/// Attempts to parse the given byte array as an SEC1-encoded scalar.
    ///
    /// Returns None if the byte array does not contain a big-endian integer in the
range
    /// [0, p).
    fn from_repr(bytes: FieldBytes) -> CtOption<Self> {
        let inner = U256::from_be_byte_array(bytes);
        CtOption::new(Self(inner), inner.ct_lt(&Secp256k1::ORDER))
    }
```

3. A panic here aborts the async task (potentially crashing the node or leaving state inconsistent). If the code avoided panic but used a non-reduced scalar, derived keys would be incorrect or non-portable across implementations, risking loss of access to vault funds or interoperability failures.

Scenario

1. Alice triggers vault generation. The code computes `vault_hmac` from her `eth_address`.
2. The HMAC happens to produce bytes that are numerically greater than or equal to the secp256k1 group order.
3. `k256::Scalar::from_repr(...).unwrap()` returns `None` and panics, crashing the current task — the vault generation stops mid-flow and any dependent state or external observers assume derivation succeeded. Later operations fail or funds become inaccessible.

Recommendation

Do not pass raw HMAC bytes directly into `Scalar::from_repr(...).unwrap()`. Instead:

- **Reduce the HMAC modulo the curve order** to map the 32-byte HMAC into a valid scalar. Use a library helper that performs a safe modular reduction (for example, a `from_bytes_reduced` / `from_bytes_mod_order` style API if available), or perform an explicit big-integer `mod n` reduction against `k256::Scalar::ORDER` and then construct the scalar from the reduced value.
- **Avoid** `unwrap()` — handle the `Option` from `from_repr` safely and return an explicit error if conversion fails. Prefer deterministic, well-specified transforms so the same HMAC always yields the same scalar on different nodes/implementations.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 3aab22bf177592ac47fe1996ab7b91b370092569

QCA-70 Hardcoded Decimal Conversion Logic Leads To Incorrect Transaction Amounts

Category	Severity	Location	Status
Volatile Code	Medium	src/rpc_server/mod.rs (chain-abstraction): 669~695	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `rpc_server/mod.rs`

The function `convert_src_smallest_to_dst_smallest` calculates the decimal adjustment factor based solely on the `src_chain`, ignoring the `dst_chain`'s decimals.

- It hardcodes a division by 10^{10} if the source is "tics" (assuming the target is always "btc" with 8 decimals).
- It hardcodes a multiplication by 10^{10} if the source is "btc" (assuming the target is always "tics" with 18 decimals).

This logic fails for:

1. **Same-chain transactions:** A TICS $\rightarrow$ TICS transfer is incorrectly divided by 10^{10}, resulting in significant fund loss. A BTC $\rightarrow$ BTC transfer is incorrectly multiplied by 10^{10}, resulting in massive inflation.
2. **Other chains:** Any chain not explicitly matched falls into the `_` case, which applies no adjustment. If a future chain with different decimals is added, the conversion will be incorrect.

```
fn convert_src_smallest_to_dst_smallest(
    amount_src_smallest: u128,
    src_chain: &str,
    dst_chain: &str,
    prices: &HashMap<String, Decimal>,
) -> Result<u128, TransactionError> {
    let src_sym = symbol_for(src_chain);
    let dst_sym = symbol_for(dst_chain);

    let p_src = *prices.get(src_sym).ok_or(TransactionError::PricingUnavailable)?;
    let p_dst = *prices.get(dst_sym).ok_or(TransactionError::PricingUnavailable)?;

    let src_dec = decimals_for(src_chain);
    let dst_dec = decimals_for(dst_chain);
    // ...

    // Apply chain-specific multiplier before price conversion
    let adjusted_amount = match src_chain.to_lowercase().as_str() {
        "tics" | "qubetics" => {
            // For TICS source chain: divide by 10^10
            let divisor = pow10_dec(10);
            let result = Decimal::from_u128(amount_src_smallest)
                .ok_or(TransactionError::Overflow)?
                / divisor;
            info!("  🔄 TICS conversion: {} / 10^10 = {}", amount_src_smallest,
result);
            result
        }
        "btc" | "bitcoin" => {
            // For BTC source chain: multiply by 10^10
            let multiplier = pow10_dec(10);
            let result = Decimal::from_u128(amount_src_smallest)
                .ok_or(TransactionError::Overflow)?
                * multiplier;
            info!("  🔄 BTC conversion: {} * 10^10 = {}", amount_src_smallest,
result);
            result
        }
        _ => {
            // For other chains: no multiplier
            let result = Decimal::from_u128(amount_src_smallest)
                .ok_or(TransactionError::Overflow)?;
            info!("  🔄 Other chain conversion: {} (no multiplier)",
amount_src_smallest);
            result
        }
    };

    // Convert to whole units using actual decimals
```

```
    let src_whole = adjusted_amount ;
    info!("  📏 Converted to whole units: {} / 10^{} = {}", adjusted_amount,
src_dec, src_whole);

    // Apply price conversion
    let dst_whole = src_whole * p_src / p_dst;
    info!("  💱 Price conversion: {} * {} / {} = {}", src_whole, p_src, p_dst,
dst_whole);

    // Convert back to smallest units
    let dst_smallest = dst_whole.floor();
    let result: u128 = dst_smallest.to_u128().ok_or(TransactionError::Overflow)?;
    // ...
    Ok(result)
}
```

Recommendation

Refactor the decimal adjustment logic to be dynamic rather than hardcoded. The adjustment factor should be calculated using the difference between the destination and source decimals.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit 75b3ac44e94d7e6a4fa01ed1fd6ff97d8d7182c7

QCA-78 | Potential Infinite Transaction Processing Loop

Category	Severity	Location	Status
Logical Issue	Medium	src/signing/mod.rs (chain-abstraction): 2805~2815, 2828~2866, 3592~3606	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `handle_signature_for_round` and `handle_btc_signature_for_round` functions exhibit a potential risk of duplicate transaction processing where:

1. A node broadcasts a signed transaction to the mempool
2. Stores the transaction ID locally
3. Broadcasts the transaction ID to other nodes
4. Receiving nodes may reprocess the same transaction, leading to infinite loop

While existing safeguards (round aggregation markers, transaction status checks) likely prevent actual infinite loops, the architectural pattern creates theoretical vulnerability to redundant processing cycles.

In `handle_signature_for_round` function, after it broadcasts raw transaction to mempool via `broadcast_transaction`, it stores transaction ID using `store_transaction_id_for_round`:

`src/signing/mod.rs`

```
3592  match Self::broadcast_ethereum_transaction(&signed_tx_hex, chain_id).await {
3593      Ok(Some(tx_hash)) => {
3594
        info!("✅ [SIGNING] Ethereum transaction broadcasted successfully: {}",
tx_hash);
3595          // Store transaction ID if we have intent hash and user registry
3596
        self.store_transaction_id_for_round(round, &tx_hash,
signature.user_eth_address.clone(), crate::types::TransactionNetwork::TICS).await;
3597      },
3598      Ok(None) => {
3599
        warn!("⚠️ [SIGNING] Ethereum transaction broadcasted but no tx_hash
returned");
3600      },
3601      Err(e) => {
3602
        error!("❌ [SIGNING] Failed to broadcast Ethereum transaction: {}", e);
3603          // Store the error message for the user to see
3604
        self.store_transaction_error_for_round(round, &e.to_string(),
crate::types::TransactionNetwork::TICS).await;
3605      }
3606  }
```

`src/signing/mod.rs`

```
 2698
/// Store transaction ID for a round if intent hash is available and broadcast to
other nodes
 2699
    async fn store_transaction_id_for_round(&self, round: u64, tx_id: &str,
user_eth_address: Option<String>, txn_network: crate::types::TransactionNetwork) {
 2700
        // Get intent hash and transaction type from the appropriate pending
transactions mapping
 2701          let (intent_hash, transaction_type) = match txn_network {
 2702              crate::types::TransactionNetwork::TICS => {
 2703                  // Check EVM pending transactions
 2704
                  let pending_transactions = self.pending_transactions.read().await;
 2705
                  if let Some((_tx, _tx_bytes, intent_hash, transaction_type)) =
pending_transactions.get(&round) {
 2706                      (intent_hash.clone(), transaction_type.clone())
 2707                  } else {
 2708                      (None, None)
 2709                  }
 2710              },
 2711              crate::types::TransactionNetwork::BTC => {
 2712                  // Check Bitcoin pending transactions
 2713
                  let pending_transactions_btc =
self.pending_transactions_btc.read().await;
 2714
                  if let Some((_tx, _tx_bytes, intent_hash, transaction_type,
_extras)) = pending_transactions_btc.get(&round) {
 2715                      (intent_hash.clone(), transaction_type.clone())
 2716                  } else {
 2717                      (None, None)
 2718                  }
 2719              }
 2720          };
 2721
 2722          match (intent_hash, &self.user_registry) {
 2723              (Some(ref intent_hash), Some(user_registry)) => {
 2724
                  // Determine transaction type based on the actual transaction type
from pending transactions
 2725
                  let (transaction_type_str, storage_result) = match transaction_type
{
 2726                      Some(crate::types::TransactionType::UserToVault) => {
 2727                          // User-to-vault transaction
 2728                          let result = user_registry
 2729
                              .store_user_to_vault_tx_id(intent_hash, tx_id, None)
 2730                              .await;
 2731                          match &result {
```

```
2732                                Ok(()) => {
2733
                                    info!("💾 [SIGNING] Stored user-to-network
transaction ID {} for intent {} (round {})", tx_id, intent_hash, round);
2734                                },
2735                                Err(e) => {
2736
                                    error!("❌ [SIGNING] Failed to store user-to-network
transaction ID for intent {}: {}", intent_hash, e);
2737                                }
2738                            }
2739                            ("user_to_vault", result)
2740                        },
2741                        Some(crate::types::TransactionType::NetworkToTarget) => {
2742                            // Network-to-target transaction
2743                            let result = user_registry
2744
                                .store_network_to_target_tx_id(intent_hash, tx_id,
Some(&self.id))
2745                                .await;
2746                            match &result {
2747                                Ok(()) => {
2748
                                    info!("💾 [SIGNING] Stored network-to-target
transaction ID {} for intent {} (round {})", tx_id, intent_hash, round);
2749                                },
2750                                Err(e) => {
2751
                                    error!("❌ [SIGNING] Failed to store network-to-
target transaction ID for intent {}: {}", intent_hash, e);
2752                                }
2753                            }
2754                            ("network_to_target", result)
2755                        },
2756                        Some(crate::types::TransactionType::VaultToNetwork) => {
2757                            // Vault-to-network transaction
2758                            let result = user_registry
2759
                                .store_vault_to_network_tx_id(intent_hash, tx_id, None)
2760                                .await;
2761                            match &result {
2762                                Ok(()) => {
2763
                                    info!("💾 [SIGNING] Stored vault-to-network
transaction ID {} for intent {} (round {})", tx_id, intent_hash, round);
2764                                },
2765                                Err(e) => {
2766
                                    error!("❌ [SIGNING] Failed to store vault-to-
network transaction ID for intent {}: {}", intent_hash, e);
2767                                }
2768                            }
2769                            ("vault_to_network", result)
```

```
                    },
                    None => {
                        // Fallback to old logic based on user_eth_address
                        if let Some(eth_addr) = &user_eth_address {
                            // User-to-network transaction
                            let result = user_registry
                                .store_user_to_vault_tx_id(intent_hash, tx_id, None)
                                .await;
                            match &result {
                                Ok(()) => {
                                    info!("💾 [SIGNING] Stored user-to-network transaction ID {} for intent {} (round {}) for user {}", tx_id, intent_hash, round, eth_addr);
                                },
                                Err(e) => {
                                    error!("❌ [SIGNING] Failed to store user-to-network transaction ID for intent {}: {}", intent_hash, e);
                                }
                            }
                            ("user_to_network", result)
                        } else {
                            // Network-to-target transaction
                            let result = user_registry
                                .store_network_to_target_tx_id(intent_hash, tx_id, None)
                                .await;
                            match &result {
                                Ok(()) => {
                                    info!("💾 [SIGNING] Stored network-to-target transaction ID {} for intent {} (round {})", tx_id, intent_hash, round);
                                },
                                Err(e) => {
                                    error!("❌ [SIGNING] Failed to store network-to-target transaction ID for intent {}: {}", intent_hash, e);
                                }
                            }
                            ("network_to_target", result)
                        }
                    }
                };

            // If storage was successful, record transaction status and broadcast to other nodes
            if storage_result.is_ok() {
                if let Err(e) = user_registry
```

```
                        .store_transaction_status(tx_id, TransactionStatus::Pending)
 2809                           .await
 2810                       {
 2811                           error!(
 2812
                             "❌ [SIGNING] Failed to store transaction status for tx
{}: {}",
 2813                                tx_id, e
 2814                           );
 2815                       }
 2816
                    self.broadcast_transaction_id(&intent_hash, tx_id,
transaction_type_str).await;
 2817                   }
 2818               },
 2819               (None, _) => {
 2820
                info!("ℹ️ [SIGNING] No intent hash available for round {}, skipping
transaction ID storage", round);
 2821               },
 2822               (_, None) => {
 2823
                info!("ℹ️ [NETWORK] No user registry available, skipping transaction
ID storage");
 2824               }
 2825           }
 2826       }
```

If storage was successful, it calls `broadcast_transaction_id` in the end:

`src/signing/mod.rs`

```
 2828
async fn broadcast_transaction_id(&self, intent_hash: &str, tx_id: &str,
transaction_type: &str) {
 2829
       let broadcast_message =
crate::types::GossipsubMessage::TransactionIdBroadcast {
 2830              intent_hash: intent_hash.to_string(),
 2831              transaction_id: tx_id.to_string(),
 2832              transaction_type: transaction_type.to_string(),
 2833              node_id: self.id.clone(),
 2834          };
 2835
 2836          let channel_message = crate::types::ChannelMessage::Broadcast {
 2837              topic: "transaction-id-sync".to_string(),
 2838              data: serde_json::to_vec(&broadcast_message).unwrap_or_default(),
 2839          };
 2840
 2841
       let channel_message_transaction_id_broadcast =
crate::types::ChannelMessage::TransactionIdBroadcast {
 2842              intent_hash: intent_hash.to_string(),
 2843              transaction_id: tx_id.to_string(),
 2844              transaction_type: transaction_type.to_string(),
 2845              node_id: self.id.clone(),
 2846          };
 2847
 2848
       // Call update_solver_amounts_for_user_to_vault only for user_to_vault
transaction type
 2849          if transaction_type == "user_to_vault" {
 2850
           if let Err(e) =
self.update_solver_amounts_for_user_to_vault(intent_hash).await {
 2851
               warn!("⚠️ [SIGNING] Failed to update solver amounts for UserToVault
transaction: {}", e);
 2852              }
 2853          }
 2854
 2855          if let Err(e) = self.message_tx.send(channel_message).await {
 2856
           error!("❌ [SIGNING] Failed to broadcast transaction ID to other nodes:
{}", e);
 2857          } else {
 2858
           info!("📡 [SIGNING] Broadcasted transaction ID {} for intent {} (type:
{}) to other nodes", tx_id, intent_hash, transaction_type);
 2859          }
 2860
 2861
       if let Err(e) =
self.message_tx.send(channel_message_transaction_id_broadcast).await {
```

```
2862
            error!("❌ [SIGNING] Failed to broadcast transaction ID to other nodes:
{}", e);
2863          } else {
2864
            info!("📡 [SIGNING] Broadcasted transaction ID {} for intent {} (type:
{}) to other nodes", tx_id, intent_hash, transaction_type);
2865          }
2866      }
```

The `broadcast_transaction_id` sends two messages: `ChannelMessage::TransactionIdBroadcast` and `GossipsubMessage::TransactionIdBroadcast`.

When handling `ChannelMessage::TransactionIdBroadcast` (the logic in `GossipsubMessage::TransactionIdBroadcast` is similar):

`src/network/mod.rs`

```
 2667
ChannelMessage::TransactionIdBroadcast { intent_hash, transaction_id,
transaction_type, node_id } => {
 2668
                info!("📡 [NETWORK] Received transaction ID broadcast from node {}",
node_id);
 2669                   info!("🔗 [NETWORK] Intent hash: {}", intent_hash);
 2670                   info!("💳 [NETWORK] Transaction ID: {}", transaction_id);
 2671                   info!("🔄 [NETWORK] Transaction type: {}", transaction_type);
 2672
 2673                   // Store the transaction ID in our local user registry
 2674                   if let Some(user_registry) = &self.user_registry {
 2675                       let storage_result = match transaction_type.as_str() {
 2676                           "user_to_vault" => {
 2677
                            // Also update solver amounts for user_to_vault using
signing node, if available
 2678                               if let Some(signing_node) = &self.signing_node {
 2679
                                if let Err(e) =
signing_node.update_solver_amounts_for_user_to_vault(&intent_hash).await {
 2680
                                    warn!("⚠️ [SIGNING] Failed to update solver
amounts for UserToVault transaction: {}", e);
 2681                                } else {
 2682
                                    // Calculate rewards after successfully updating
solver amounts
 2683
                                    if let Err(e) =
signing_node.calculate_reward_per_solver(&intent_hash).await {
 2684
                                        warn!("⚠️ [SIGNING] Failed to calculate
rewards for UserToVault transaction: {}", e);
 2685                                    } else {
 2686
                                        info!("✅ [SIGNING] Successfully calculated
rewards for UserToVault transaction: {}", intent_hash);
 2687                                    }
 2688                                }
 2689                            }
 2690
                            user_registry.store_user_to_vault_tx_id(&intent_hash,
&transaction_id, Some(&node_id)).await
 2691                           },
 2692                           "network_to_target" => {
 2693

user_registry.store_network_to_target_tx_id(&intent_hash, &transaction_id,
Some(&node_id)).await
 2694                           },
 2695                           "vault_to_network" => {
```

```
 2696
                            user_registry.store_vault_to_network_tx_id(&intent_hash,
&transaction_id, Some(&node_id)).await
 2697                           },
 2698                           _ => {
 2699
                            warn!("❌ [NETWORK] Unknown transaction type: {}",
transaction_type);
 2700                               return Ok(());
 2701                           }
 2702                       };
 2703
 2704                       match storage_result {
 2705                           Ok(()) => {
 2706                               if let Err(e) = user_registry
 2707
                                .store_transaction_status(&transaction_id,
TransactionStatus::Pending)
 2708                                    .await
 2709                                {
 2710                                    error!(
 2711
                                    "❌ [NETWORK] Failed to store transaction status
for broadcast tx {}: {}",
 2712                                        transaction_id, e
 2713                                    );
 2714                                }
 2715                                info!(
 2716
                                "✅ [NETWORK] Successfully synchronized transaction
ID {} for intent {} (type: {})",
 2717                                    transaction_id, intent_hash, transaction_type
 2718                                );
 2719
 2720
                            // Execute contract call when transaction ID is received
 2721
                            if let Some(user_registry) = &self.user_registry {
 2722
                                if let Some(intent) =
user_registry.get_intent(&intent_hash).await {
 2723
                                    // Convert transaction type string to enum
 2724
                                    let tx_type_enum = match
transaction_type.as_str() {
 2725
                                        "user_to_vault" =>
crate::types::TransactionType::UserToVault,
 2726
                                        "network_to_target" =>
crate::types::TransactionType::NetworkToTarget,
 2727
```

```
                                    "vault_to_network" =>
crate::types::TransactionType::VaultToNetwork,
 2728                                           _ => {
 2729
                                        warn!("❌ [CONTRACT] Unknown transaction
type for contract call: {}", transaction_type);
 2730                                               return Ok(());
 2731                                           }
 2732                                       };
 2733
 2734                                       // Get vault group key
 2735
                                    let vault_group_key = if let Some(dkg_node) =
&self.dkg_node {
 2736                                           dkg_node.get_vault_group_key().await
 2737                                       } else {
 2738                                           None
 2739                                       };
 2740
 2741
                                    if let Some(vault_group_key) = vault_group_key {
 2742
 2743                                           // Execute contract call
 2744
                                        if let Err(e) =
self.execute_contract_call_for_transaction(
 2745                                               &intent,
 2746                                               &intent_hash,
 2747                                               tx_type_enum,
 2748                                               &transaction_id,
 2749                                               Some(vault_group_key),
 2750                                               intent.amount,
 2751                                           ).await {
 2752
                                            warn!("❌ [CONTRACT] Failed to execute
contract call for transaction ID {}: {}", transaction_id, e);
 2753                                           } else {
 2754
                                            info!("✅ [CONTRACT] Successfully
executed contract call for transaction ID {}", transaction_id);
 2755                                           }
 2756                                       } else {
 2757
                                        warn!("❌ [CONTRACT] No vault group key
available for contract call execution");
 2758                                       }
 2759                                   } else {
 2760
                                    warn!("❌ [CONTRACT] Could not retrieve intent
{} for contract call execution", intent_hash);
 2761                                   }
 2762                               }
 2763                           },
```

```
 2764                         Err(e) => {
 2765
                            error!("❌ [NETWORK] Failed to store synchronized
transaction ID: {}", e);
 2766                         }
 2767                     }
 2768                 } else {
 2769
                    warn!("❌ [NETWORK] No user registry available to store
synchronized transaction ID");
 2770                 }
 2771             }
```

It stores transaction ID again and potentially initiates new contract calls by calling `execute_contract_call_for_transaction`:

`src/network/mod.rs`

```
3020  async fn execute_contract_call_for_transaction(
3021          &mut self,
3022          intent: &crate::rpc_server::DepositIntent,
3023          intent_hash: &str,
3024          transaction_type: crate::types::TransactionType,
3025          tx_hash: &str,
3026          vault_group_key: Option<k256::ProjectivePoint>,
3027          amount: u128,
3028      ) -> Result<()> {
3029          if let Some(signing_node) = &mut self.signing_node {
3030
            info!("📝 [CONTRACT] Executing contract call for transaction type:
{:?}", transaction_type);
3031
3032              tokio::time::sleep(tokio::time::Duration::from_secs(5)).await;
3033
3034
            let contract_call_result =
crate::utils::transaction::create_chain_transaction_from_contract_call(
3035                  vault_group_key,
3036                  amount,
3037                  intent,
3038                  intent_hash,
3039                  tx_hash,
3040                  transaction_type.clone()
3041              ).await;
3042
3043              match contract_call_result {
3044                  Ok(contract_transaction) => {
3045                      info!("📝 [CONTRACT] Created contract call transaction");
3046
                   let available_nodes: Vec<PeerId> =
self.node_manager.get_active_nodes().into_iter().collect();
3047                      signing_node.set_available_nodes(available_nodes);
3048
3049                      match contract_transaction {
3050
                       crate::chain::ChainTransaction::Ethereum(ref eth_tx) => {
3051
                           let contract_tx = crate::rpc_server::ContractTransaction
{
3052                                  to: eth_tx.to.clone(),
3053                                  value: eth_tx.value.clone(),
3054                                  nonce: eth_tx.nonce,
3055                                  gas_limit: eth_tx.gas_limit,
3056                                  gas_price: eth_tx.gas_price.clone(),
3057                                  chain_id: eth_tx.chain_id,
3058
                              data: eth_tx.data.as_ref().map(|d| format!("0x{}",
hex::encode(d))).unwrap_or_else(|| "0x".to_string()),
3059                              };
3060
```

```
                            let contract_tx_bytes =
crate::utils::transaction::create_transaction_for_signing_contract(&contract_tx);
 3061
                            debug!("→ RLP payload contract call: 0x{}",
hex::encode(&contract_tx_bytes));
 3062
 3063
                            let vault_tweaked_share = if let Some(dkg_node) =
&self.dkg_node {
 3064
                                dkg_node.get_vault_tweaked_secret_share().await
 3065                           } else {
 3066                               None
 3067                           };
 3068
                            signing_node.sign_contract_message(contract_tx_bytes,
&contract_tx, vault_tweaked_share, None).await?;
 3069                       }
 3070                       _ => {
 3071
                            info!("📝 [CONTRACT] Contract call transaction is not
Ethereum type, skipping");
 3072                       }
 3073                   }
 3074               }
 3075               Err(e) => {
 3076
                   warn!("❌ [CONTRACT] Failed to create contract call transaction:
{}", e);
 3077               }
 3078           }
 3079       } else {
 3080
           warn!("❌ [CONTRACT] Signing node not available for contract call
execution");
 3081       }
 3082
 3083       Ok(())
 3084   }
```

The `execute_contract_call_for_transaction` function calls `signing_node.sign_contract_message` to start a new round signature aggregation, which cause an infinite transaction processing loop.

Recommendation

Consider redesigning the logic of `ChannelMessage::TransactionIdBroadcast` and `GossipsubMessage::TransactionIdBroadcast`.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit ea80baf7296f2078b289236abfc94e46b3a791d0.

QCA-79 | Silent Contract Creation In EIP-155 Transaction Encoding

Category	Severity	Location	Status
Logical Issue	Medium	src/utils/transaction.rs (chain-abstraction): 1691~1704	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The function `create_transaction_for_signing` is intended to generate an EIP-155 signing preimage for standard Ethereum transfers. However, it fails to validate the `to` address field properly, allowing an attacker to craft a transaction that silently converts into a contract creation transaction when `tx.to` is malformed (e.g., "0x").

Since the function also hardcodes an **empty** data field, this results in a transaction that deploys a contract with no code, irreversibly locking any sent ETH. This is a critical logic flaw—not merely a validation issue—because it fundamentally changes the transaction semantics while remaining a valid EIP-155 preimage.

The current implementation:

`src/utils/transaction.rs`

```
1691  // To address
1692      encoder = RlpEncoder::new();
1693      encoder.encode_address(&tx.to);
1694      content.extend_from_slice(&encoder.finalize());
1695
1696      // Value
1697      encoder = RlpEncoder::new();
1698      encoder.encode_hex_string(&tx.value);
1699      content.extend_from_slice(&encoder.finalize());
1700
1701      // Data (empty)
1702      encoder = RlpEncoder::new();
1703      encoder.encode_bytes(&[]);
1704      content.extend_from_slice(&encoder.finalize());
```

`src/utils/transaction.rs`

```
105 fn encode_address(&mut self, address: &str) {
106         let bytes = hex::decode(&address[2..]).expect("Invalid address");
107         self.encode_bytes(&bytes);
108     }
```

The `encode_address` function slices `address[2..]`, resulting in an empty string if `to` is `"0x"`. `hex::decode("")` returns `Ok(vec![])`, which RLP-encodes as `0x80` (empty string). In Ethereum, `to = 0x80` means contract creation (not a transfer).

The function also hard-codes an empty data field, so the preimage it produces commits to a contract-creation with zero init code. This is a logical flaw: a function named and used to create a transfer-signing preimage silently turns into a contract-creation preimage when to is empty. In Ethereum, a creation tx with empty init code deploys an empty-code contract. Any ETH sent is irrecoverably stuck at the new contract address (no code means no way to spend).

Recommendation

Enforce that `tx.to` is a valid, non-empty 20-byte address before encoding the signing preimage. Reject any transaction where `tx.to` is missing, empty, or not a properly formatted address.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit a4685306cf0c12f6234a74212c53fc089fd3cdf7.

QCA-80 Unauthenticated DKG Trigger (`StartADKG` / `ScheduleADKG`) Lets Any Peer Reset DKG And Override Thresholds

Category	Severity	Location	Status
Access Control	Medium	src/network/mod.rs (chain-abstraction): 779~814, 850~854	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `e98d1d42332be4cd50cd56e989476ccde32f37ee`

File:

- `src/network/mod.rs`

The NetworkLayer implementation contains a critical security flaw where any connected peer can trigger Distributed Key Generation (DKG) resets and parameter changes without authorization. This allows attackers to:

1. Disrupt DKG completion (Denial-of-Service)
2. Forcibly override threshold parameters, destabilizing the network
3. Desynchronize nodes by enforcing inconsistent thresholds

The issue stems from missing access controls in `handle_swarm_event`, where `RPCRequest::StartADKG` and `RPCRequest::ScheduleADKG` requests are processed without role checks, quorum validation, or cryptographic authentication.

The `handle_swarm_event` (called by the main loop in start) processes incoming `RPCRequest::StartADKG` and `RPCRequest::ScheduleADKG` without authorization:

`src/network/mod.rs`

```
779 RPCRequest::StartADKG// {n, threshold }
780         => {
781             // handle dkg start
782             match self.dkg_node.as_mut() {
783                 Some(dkg) => {
784                     info!("🚀 Received DKG start request from peer {}", peer);
785 
786                     // Update DKG parameters with current network state
787                     let total = self.node_manager.get_total_nodes();
788                     let threshold = self.node_manager.get_threshold();
789                     dkg.update_node_params(total, threshold);
790                     dkg.reset().await;
791                     if let Some(signing_node) = &mut self.signing_node {
792 
signing_node.set_threshold(self.node_manager.get_total_nodes() - 1);
793                     }
794                     self.node_manager
795                         .set_threshold(total - 1);
796 
                    info!("🔄 DKG parameters updated: total_nodes={}, threshold={}",
total, threshold);
797 
798                     match dkg.start_dkg(self.node_manager.clone()).await {
799                         Ok(_) => {
800 
                            info!("✅ Successfully started DKG in response to peer
request");
801 
                            RPCResponse::DKGStarted {peer_id:
SerializablePeerId(peer)}
802                         }
803                         Err(e) => {
804                             warn!("❌ Failed to start DKG: {:?}", e);
805 
                            RPCResponse::DKGError(format!("Failed to start DKG:
{:?}", e))
806                         }
807                     }
808                 }
809                 None => {
810                     warn!("❌ DKG node not initialized");
811 
                    RPCResponse::DKGError("DKG is not initialized.".to_string())
812                 }
813             }
814         }
```

`src/network/mod.rs`

```
850 RPCRequest::ScheduleADKG { start_time } => {
851     info!("🕒 Got DKGStartAt({})", start_time);
852     self.schedule_dkg_at(start_time);
853     RPCResponse::Ack
854 }
```

For `RPCRequest::StartADKG`, on receipt, the node immediately resets the DKG and starts a new round, while also mutating thresholds. For `RPCRequest::ScheduleADKG { start_time }`, on receipt, the node unconditionally schedules a DKG start via `schedule_dkg_at`. When the scheduled task fires (`ChannelMessage::ScheduledDKGStart`), it resets DKG and changes thresholds to `total/2 + 1`, then starts DKG. No role checks, leader election, quorum, or membership validation are performed before executing these state-changing operations. This is a pure logic/authorization flaw directly reachable via the network.

Impact

- Any unprivileged peer can repeatedly reset or restart DKG, preventing completion (DoS of key generation and all dependent signing flows).
- Thresholds are arbitrarily overridden by the attacker-triggered call (to total-1 for `StartADKG`, and to total/2+1 for `ScheduledDKGStart`), desynchronizing parameters across the network and breaking liveness.
- In `StartADKG` specifically, the node updates DKG parameters using the old threshold, then immediately overrides node_manager/signing thresholds to total-1, creating inconsistent local state that further destabilizes the protocol.

Recommendation

Implement strict authentication and role-based authorization for all DKG-control RPC requests (`StartADKG` and `ScheduleADKG`), ensuring that only designated, trusted nodes or authorized leaders may initiate or schedule DKG operations. Validate the caller's identity, membership, and appropriate privileges before allowing any state changes, threshold updates, or DKG resets. Reject unauthorized or malformed requests to prevent unprivileged peers from disrupting key generation or manipulating parameters.

Alleviation

[Qubetics, 12/19/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> right now we are neglecting the faulty nodes flow as the binary will be send to solvers joining the network and the code will not be public.

QCA-81 | Unauthenticated `TransactionIdBroadcast` Triggers Contract Call Execution

Category	Severity	Location	Status
Access Control	Medium	src/network/mod.rs (chain-abstraction): 1499~1510	Partially Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `e98d1d42332be4cd50cd56e989476ccde32f37ee`

File:

- `src/network/mod.rs`

The system unconditionally processes the `TransactionIdBroadcast` gossipsub messages and triggers vault-signed contract calls without proper authentication, transaction validation, or consensus mechanisms. This allows any malicious peer to potentially forge transaction IDs and induce unauthorized contract executions using the vault's signing authority.

The code processes `TransactionIdBroadcast` messages without verifying the sender's identity (`node_id`):

`src/network/mod.rs`

```
 1404
 GossipsubMessage::TransactionIdBroadcast { intent_hash, transaction_id,
transaction_type, node_id } => {
 1405
              info!("📡 [NETWORK] Received transaction ID broadcast from node {}",
node_id);
 1406                 info!("🔗 [NETWORK] Intent hash: {}", intent_hash);
 1407                 info!("💳 [NETWORK] Transaction ID: {}", transaction_id);
 1408                 info!("🔄 [NETWORK] Transaction type: {}", transaction_type);
 1409
 1410                 // Store the transaction ID in our local user registry
 1411                 if let Some(user_registry) = &self.user_registry {
 1412                     let storage_result = match transaction_type.as_str() {
 1413                         "user_to_vault" => {
 1414
                          // Also update solver amounts for user_to_vault using
signing node, if available
 1415                             if let Some(signing_node) = &self.signing_node {
 1416
                              if let Err(e) =
signing_node.update_solver_amounts_for_user_to_vault(&intent_hash).await {
 1417
                                  warn!("⚠️ [SIGNING] Failed to update solver
amounts for UserToVault transaction: {}", e);
 1418                                 } else {
 1419
                                 // Calculate rewards after successfully updating
solver amounts
 1420
                                 if let Err(e) =
signing_node.calculate_reward_per_solver(&intent_hash).await {
 1421
                                     warn!("⚠️ [SIGNING] Failed to calculate
rewards for UserToVault transaction: {}", e);
 1422                                 } else {
 1423
                                     info!("✅ [SIGNING] Successfully calculated
rewards for UserToVault transaction: {}", intent_hash);
 1424                                 }
 1425                             }
 1426                         }
 1427                         user_registry
 1428                             .store_user_to_vault_tx_id(
 1429                                 &intent_hash,
 1430                                 &transaction_id,
 1431                                 Some(&node_id),
 1432                             )
 1433                             .await
 1434                     },
 1435                     "network_to_target" => {
 1436                         user_registry
 1437                             .store_network_to_target_tx_id(
```

```
1438                                        &intent_hash,
1439                                        &transaction_id,
1440                                        Some(&node_id),
1441                                    ).await
1442                            },
1443                            "vault_to_network" => {
1444                                user_registry
1445                                    .store_vault_to_network_tx_id(
1446                                        &intent_hash,
1447                                        &transaction_id,
1448                                        Some(&node_id),
1449                                    )
1450                                    .await
1451                            },
1452                            _ => {
1453
                             warn!("❌ [NETWORK] Unknown transaction type: {}",
transaction_type);
1454                                return Ok(());
1455                            }
1456                        };
1457
1458                        match storage_result {
1459                            Ok(()) => {
1460                                if let Err(e) = user_registry
1461
                                 .store_transaction_status(&transaction_id,
TransactionStatus::Pending)
1462                                    .await
1463                                {
1464                                    error!(
1465
                                     "❌ [NETWORK] Failed to store transaction status
for broadcast tx {}: {}",
1466                                        transaction_id, e
1467                                    );
1468                                }
1469                                info!(
1470
                                 "✅ [NETWORK] Successfully synchronized transaction
ID {} for intent {} (type: {})",
1471                                    transaction_id, intent_hash, transaction_type
1472                                );
1473
1474
                             // Execute contract call when transaction ID is received
1475
                             if let Some(user_registry) = &self.user_registry {
1476
                                 if let Some(intent) =
user_registry.get_intent(&intent_hash).await {
1477
                                     // Convert transaction type string to enum
```

```
 1478
                                    let tx_type_enum = match
transaction_type.as_str() {
 1479
                                        "user_to_vault" =>
crate::types::TransactionType::UserToVault,
 1480
                                        "network_to_target" =>
crate::types::TransactionType::NetworkToTarget,
 1481
                                        "vault_to_network" =>
crate::types::TransactionType::VaultToNetwork,
 1482                                           _ => {
 1483
                                            warn!("❌ [CONTRACT] Unknown transaction
type for contract call: {}", transaction_type);
 1484                                               return Ok(());
 1485                                           }
 1486                                       };
 1487
 1488                                       // Get vault group key
 1489
                                    let vault_group_key = if let Some(dkg_node) =
&self.dkg_node {
 1490                                           dkg_node.get_vault_group_key().await
 1491                                       } else {
 1492                                           None
 1493                                       };
 1494
 1495
                                    if let Some(vault_group_key) = vault_group_key {
 1496
                                        // Use transaction_id as tx_hash for
contract call
 1497
                                        // let tx_hash_bytes =
transaction_id.clone();
 1498                                           // Execute contract call
 1499
                                        if let Err(e) =
self.execute_contract_call_for_transaction(
 1500                                               &intent,
 1501                                               &intent_hash,
 1502                                               tx_type_enum,
 1503                                               &transaction_id,
 1504                                               Some(vault_group_key),
 1505                                               intent.amount,
 1506                                           ).await {
 1507
                                            warn!("❌ [CONTRACT] Failed to execute
contract call for transaction ID {}: {}", transaction_id, e);
 1508                                           } else {
 1509
```

```
                                        info!("✅ [CONTRACT] Successfully
executed contract call for transaction ID {}", transaction_id);
 1510                                       }
 1511                                   } else {
 1512
                                    warn!("❌ [CONTRACT] No vault group key
available for contract call execution");
 1513                                   }
 1514                               } else {
 1515
                                warn!("❌ [CONTRACT] Could not retrieve intent
{} for contract call execution", intent_hash);
 1516                               }
 1517                           }
 1518                       },
 1519                       Err(e) => {
 1520
                        error!("❌ [NETWORK] Failed to store synchronized
transaction ID: {}", e);
 1521                       }
 1522                   }
 1523               } else {
 1524
                warn!("❌ [NETWORK] No user registry available to store
synchronized transaction ID");
 1525               }
 1526           }
```

Any connected peer can publish to gossipsub topics. The code never verifies that the broadcast is from a trusted/selected/final node. The provided `node_id` is only logged and never checked. The presence of an intent is the only gating condition; intents can be injected via another unauthenticated path (`IntentHash`), which the code also stores without verification. As soon as an intent is present, a single forged TransactionIdBroadcast causes the node to sign a contract call with the vault key material (`vault_tweaked_secret_share`).

Practical impact:

- An attacker can cause the node to generate and sign a contract call that finalizes/credits an intent using an arbitrary `transaction_id` , potentially releasing funds or updating on-chain state illegitimately.
- The signature is produced with the vault key share (vault tweaked share). In a real deployment where multiple nodes run the same logic, this leads to coordinated signing and on-chain effect without proper authorization.

Scenario

1. Connect to the P2P network as a normal peer and publish a fake intent so the target stores it locally:
    - Topic: `intent-hashes`
    - Payload: serde_json serialization of `GossipsubMessage::IntentHash { intent_hash, signer, intent }` where: • `intent_hash` : any 32-byte hex string (e.g., deadbeef…32 bytes) – the code does not verify that it matches the provided intent. • signer: any string • intent: a `DepositIntent` object with desired fields (`amount` ,

`source_chain`, `target_chain`, `target_address`). The code stores it via `user_registry.store_intent_hash` without checks. Expected node logs: “Stored intent hash from ”.

2. Trigger the unauthorized contract call execution:
   - Topic: `transaction-id-sync`
   - Payload: serde_json serialization of

```
GossipsubMessage::TransactionIdBroadcast {
   intent_hash: <the same hex string from step 1>,
   transaction_id: any string that looks like a tx hash (e.g., 0x1122…cc),
   transaction_type: one of: "user_to_vault", "network_to_target", or
"vault_to_network",
   node_id: arbitrary string
 }
```

The branch matches on `transaction_type` string, stores the fake `transaction_id`, and then calls `execute_contract_call_for_transaction` with `vault_group_key` and `amount` from the stored intent.

3. Observe the node execute and sign the contract call:
   - In ~5 seconds you should see logs similar to: • “Successfully synchronized transaction ID … for intent … (type: …)” • “Successfully executed contract call for transaction ID …” At this point, the node has produced a contract transaction signed with the vault tweaked secret share, based solely on your forged broadcast.

Recommendation

Implement strict authentication and validation checks on gossipsub `TransactionIdBroadcast` messages before executing any contract call with vault key material. Verify that the sender (`node_id`) is an authorized and trusted node, ensure the `transaction_id` corresponds to a legitimate, finalized on-chain transaction, and require multi-node consensus or explicit finalization for contract execution. Reject or ignore any request that does not meet these criteria to prevent unauthorized actions.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and partially resolved the issue in commit 18691117fab273fc50869bdf248a9e6017a3a00a

> Fixed the node_id authorization and trusted node verification in this commit. Since the transaction ID is broadcast immediately after creating the EVM transaction, on-chain verification is not feasible in this case.

[CertiK, 12/26/2025]: While verifying the sender as an active node is a step forward, we recommend implementing stricter validation for `TransactionIdBroadcast` messages to ensure full security. The `node_id` should not just be any active node, but specifically one of the authorized MPC-selected final nodes responsible for this transaction. Furthermore, we strongly

advise delaying the `TransactionIdBroadcast` until after the transaction is confirmed on-chain (with sufficient block confirmations) rather than broadcasting immediately after creation. This provides cryptographic proof of legitimacy and prevents premature execution of contract calls. The current approach still leaves a window where malicious active nodes could forge transactions before they're actually mined. Combining both MPC node verification and on-chain confirmation would provide robust protection against unauthorized contract executions.

QCA-82 | Wrong `scriptCode` Used For Non-P2WPKH SegWit Inputs (`P2WSH` / `P2SH-P2WPKH`) In BIP143 Preimage

Category	Severity	Location	Status
Logical Issue	Medium	src/chain/mod.rs (chain-abstraction): 441~459	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/chain/mod.rs`

The `create_transaction_bytes` function in the Bitcoin chain handler incorrectly implements BIP143's scriptCode derivation rules for `SegWit` transactions. While it properly handles basic `P2WPKH` cases, it fails to correctly process `P2WSH` and nested `SegWit` (`P2SH-P2WPKH`) transactions by improperly using the witness UTXO's `script_pubkey` directly as `scriptCode` in all non-P2WPKH cases.

In the `SegWit` path of `create_transaction_bytes` , the code builds the BIP143 preimage using scriptCode derived as follows:

`src/chain/mod.rs`

```
442  let script_code = if witness_utxo.script_pubkey.len() == 22
443                          && witness_utxo.script_pubkey[0] == 0x00
444                          && witness_utxo.script_pubkey[1] == 0x14
445                      {
446                          // P2WPKH -> convert to P2PKH script
447                          let pubkey_hash = &witness_utxo.script_pubkey[2..];
448                          let mut script = Vec::new();
449                          script.push(0x76); // OP_DUP
450                          script.push(0xa9); // OP_HASH160
451                          script.push(0x14); // push 20 bytes
452                          script.extend_from_slice(pubkey_hash);
453                          script.push(0x88); // OP_EQUALVERIFY
454                          script.push(0xac); // OP_CHECKSIG
455                          script
456                      } else {
457
                        // For other SegWit types, use the witness UTXO's
script_pubkey directly
458                          witness_utxo.script_pubkey.clone()
459                      };
```

If the prevout is detected as `P2WPKH` (`len 22, 0x00 0x14`), it correctly constructs the P2PKH scriptCode from the 20-byte pubkey hash. Otherwise (any other SegWit type), it incorrectly uses `witness_utxo.script_pubkey` directly as scriptCode.

According to **BIP143**, for `SegWit` inputs:

- **P2WPKH:** scriptCode is the corresponding P2PKH script (what the code does only in the very specific 22-byte `OP_0 push20` case).
- **P2WSH:** scriptCode must be the `witnessScript` (the actual redeem script), not the scriptPubKey (which is `OP_0 <32-byte-hash>`). The current code uses the `scriptPubKey` instead of the `witnessScript` .
- **P2SH-P2WPKH (nested):** scriptCode must be the `P2PKH` script derived from the witness program, not the outer P2SH scriptPubKey (`OP_HASH160 <20> OP_EQUAL`). The current code will go into the fallback branch and use that P2SH scriptPubKey as scriptCode, which is wrong.

Impact:

Signing the wrong preimage means the returned signature does not correspond to what consensus will verify for `P2WSH` or nested `P2WPKH` spends. A caller can make the signer produce signatures that are invalid on-chain (DoS), or worse, cause the signer to believe they signed one spending condition while the network verifies another. This is a pure logic bug in preimage construction, not a best-practice or library issue.

Recommendation

Update the `SegWit` logic in `create_transaction_bytes` to correctly derive scriptCode for each input type per BIP143:

- For `P2WSH` inputs, set scriptCode to the corresponding witnessScript.
- For `P2SH-P2WPKH` inputs, set scriptCode to the 25-byte P2PKH script constructed from the 20-byte keyhash inside the witness program.

- Only for native `P2WPKH` inputs (22-byte OP_0 push20), continue to build the scriptCode as the corresponding P2PKH script.

Do not use `witness_utxo.script_pubkey` as scriptCode for any `SegWit` input except native `P2WPKH` . Validate the scriptPubKey and derive scriptCode accordingly to ensure all BIP143 preimages are constructed as consensus requires.

Alleviation

[Qubetics, 12/18/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> for now we have only p2pkh addresses

QCA-83 | Leader Election Biased By Lexicographic PeerId Tie-Breaker

Category	Severity	Location	Status
Volatile Code	Medium	src/consensus/mod.rs (chain-abstraction): 99~118; src/signing/mod.rs (chain-abstraction): 2980~2985	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/consensus/mod.rs`
- `src/signing/mod.rs`

In `ConsensusNode::start_consensus` (called by `start_consensus_for_round`), the final leader is selected by lexicographically sorting candidate PeerIds and picking the first:

- When multiple nodes share the highest selection frequency, candidates are built as either: (a) all unique nodes with max_frequency, or (b) all `selected_nodes` when all frequencies are equal.
- Then `candidates.sort_by(|a,b| a.0.to_bytes().cmp(&b.0.to_bytes()));` `let final_node = candidates[0].clone();`

Because PeerId bytes are fully determined by the node's own keypair and the code never randomizes or salts tie-breaking, a node can precompute a PeerId that sorts before others. This gives that node a systematic advantage: whenever it ties for the top frequency (including the common case where all frequencies are equal), it will always be chosen as the `final_node`. This is a logic flaw in leader selection fairness and is independent of any technical error or best practice; it is a deterministic bias introduced by the tie-break rule itself.

Why this matters in this function: `start_consensus_for_round` uses this biased result to:

- Broadcast the consensus result; and
- If the biased node is `final_node`, immediately call `trigger_aggregate_signature_for_final_node(round)`, causing other nodes to send ECDSA/BTC signatures to the attacker-controlled final node. This elevates the attacker's control and influence in aggregation rounds beyond what VRF intended.

```
// consensus/mod.rs
      // Gather all candidates that have max_frequency
      let mut candidates: Vec<SerializablePeerId> = if all_same_frequency {
          // everyone tied → *all* selected_nodes are candidates
          selected_nodes.clone()
      } else {
          // only those whose frequency == max_frequency
          node_frequencies
              .iter()
              .filter(|(_, &freq)| freq == max_frequency)
              .map(|(node, _)| node.clone())
              .collect()
      };

      // Sort them by their inner PeerId bytes and pick the first
      candidates.sort_by(|a, b| a.0.to_bytes().cmp(&b.0.to_bytes()));
      info!(
          "📋 [CONSENSUS] Candidates with max frequency {}: {:?}",
          max_frequency, candidates
      );
      let final_node = candidates[0].clone();
```

Scenario

1. Grind a lexicographically-small PeerId
    - Offline, generate libp2p keypairs and compute `PeerId::from_public_key` for each. Keep the keypair whose `PeerId.to_bytes()` is minimal among a set (e.g., starts with many 0x00 bytes). This is standard identity grinding and requires no privileges.
    - Start your node with self.id set to the base58 of that PeerId (`SigningNode::new` will parse it and use it in `VrfService/SerializablePeerId`).
2. Participate normally until a round with tied top frequency occurs
    - This happens whenever multiple nodes are selected with the same max frequency (including the all-same-frequency case). No external manipulation is required; ties are natural in VRF sampling, especially with small sample sizes per round.
3. Call `start_consensus_for_round(round)`
    - The function fetches `all_selected_nodes` from the VRF service and delegates to `consensus_node.start_consensus(round)`.
    - In `start_consensus`, since there's a tie, candidates are sorted by PeerId bytes and the first is chosen. Your precomputed PeerId sorts first, so you become result.final_node.
4. Observe privileged follow-on effects in the same call

- `start_consensus_for_round` detects you are the final node (`consensus_node.is_final_node`) and immediately invokes `trigger_aggregate_signature_for_final_node(round)` , which broadcasts `SigningMessage::RequestSignatures` or `RequestBtcSignatures` to collect signatures from other nodes. You now repeatedly gain the leader/aggregator role whenever ties occur.

Recommendation

Replace lexicographic sorting of `PeerIds` as a tie-breaking mechanism in leader selection with a method that does not allow nodes to bias the outcome by precomputing advantageous PeerIds. Use a cryptographically secure and unpredictable randomization technique—such as deriving a shared random value from the VRF output or round-specific entropy—to break ties among candidate nodes with equal frequency, ensuring each candidate has an equal and unbiased chance of selection.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 1c959b9de6872219be6d8fc1d494c17f87a71385

QCA-84 | VRF Committee Size Hard-Coded To One Signer Bypassing Threshold Security

Category	Severity	Location	Status
Volatile Code	Medium	src/signing/mod.rs (chain-abstraction): 1742~1756	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The VRF selection routines always request exactly one signer instead of using the configured MPC/threshold committee size. Every call sets `threshold = 1`, so `VrfNodeSelector::select_mpc_nodes` (`src/vrf/mod.rs`) trims the sorted VRF results down to a single peer, even though downstream signing logic expects multiple shares to satisfy `self.threshold`. Example from the Ethereum signing path:

```
//1743:1751:src/signing/mod.rs
if let Some(vrf_service) = &self.vrf_service {
    let round = vrf_service.get_current_round().await;
    let _selection_result = vrf_service.perform_node_selection(
        &available_nodes,
        1,
        round,
        self.chain_handler.chain_type()
    ).await?;
}
```

Because the VRF module simply samples `.take(threshold)` (`src/vrf/mod.rs` lines 304-349), the constant `1` effectively tells the network that a single node is sufficient. Nodes can therefore self-select every round, and observers relying on VRF broadcasts are misled into thinking a one-node committee satisfies the protocol.

Recommendation

Pass the actual signing quorum (e.g., `self.threshold` or `node_manager.get_threshold()`) into `perform_node_selection` for every signing flow, and refuse to proceed unless the local node appears in the VRF-selected

committee. This keeps VRF outputs aligned with the MPC threshold and prevents compromised nodes from unilaterally “winning” every round. If VRF selection is not yet enforced, disable the broadcast to avoid a false sense of security until the threshold-aware logic is in place.

Alleviation

[Qubetics, 12/23/2025]: The team acknowledged the issue is an intended design and decided not to implement the recommended change in the current engagement.

QCA-09 | Silently Defaulting To `index=1` Will Undermine DKG Security

Category	Severity	Location	Status
Logical Issue	Minor	src/adkg_secp256k1/mod.rs (chain-abstraction): 464~470	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The current implementation of `get_node_index` silently defaults to returning `1` when a node's ID is not found in the `node_manager`'s active node list. This behavior introduces critical security vulnerabilities in the Distributed Key Generation (DKG) protocol:

1. Multiple nodes may incorrectly share the same index (1), violating Shamir's Secret Sharing uniqueness requirement.
2. Secret shares are misassigned, leading to incorrect final key computations.
3. Public-private key mismatch, breaking cryptographic consistency (`G^Σsecret_shares ≠ Σcommitments`).
4. Threshold signature schemes fail, as interpolated secrets become invalid.

A deterministic index mapping is essential for DKG correctness, and silent failures must be replaced with explicit error handling.

The current implementation of `get_node_index`:

`src/adkg_secp256k1/mod.rs`

```
464         let index = ids.iter()
465             .position(|id| id == &self.id)
466             .map(|i| i + 1)
467             .unwrap_or_else(|| {
468
                tracing::error!("DKGNode id {} not in NodeManager list", self.id);
469                 1
470             });
471
472          info!("🔍 [DKG] Node {} assigned index: {}", self.id, index);
473          info!("🔍 [DKG] === END NODE INDEX CALCULATION ===");
474
475          index
```

While logging an error, the code still returns `1` as a default value. In threshold cryptography (`(t,n)` schemes), each node must have a unique index (`x_i`) assigned to its polynomial share (`f(x_i)`). If multiple nodes default to `index=1`, they incorrectly aggregate the same shares, breaking the mathematical guarantees of secret reconstruction.

Recommendation

Consider returning error instead, for example:

```
ids.iter()
        .position(|id| id == &self.id)
        .map(|i| i + 1)
        .ok_or_else(|| anyhow!("Node {} not in active list", self.id)) // Fail fast
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 18797b174ef2eca40c0e9a9a697162eb45a31146.

QCA-113 | Incorrect Ethereum Address Validation

Category	Severity	Location	Status
Logical Issue	Minor	src/rpc_server/mod.rs (chain-abstraction): 319~323	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The `validate_ethereum_address` function has logical flaws in its condition checks, particularly around length and prefix validation. A valid Ethereum address must:

1. Start with "0x",
2. Be exactly 42 characters long ("0x" + 40 hex digits),
3. Contain only valid hexadecimal characters (0-9, a-f, A-F),
4. Have a correct checksum (if mixed-case).

The implementation incorrectly used `&&` (logical `AND`) instead of `||` (logical `OR`), allowing invalid addresses to pass validation if they failed only one condition (e.g., missing "0x" or wrong length).

The `validate_ethereum_address` function has following check:

`src/rpc_server/mod.rs`

```
319  if !address.starts_with("0x") && address.len() != 42 {
320          return Err(RpcError::InvalidAddress(
321              "Address must be 42 characters and start with 0x".to_string(),
322          ));
323      }
```

The condition `if !address.starts_with("0x") && address.len() != 42` fails only if **both** conditions are true. This means addresses like "1234567890123456789012345678901234567890" (no "0x" but correct length) or "0x123" (has "0x" but wrong length) would **not** be rejected.

Recommendation

Consider following fix:

```
if !address.starts_with("0x") || address.len() != 42
```

Alleviation

[Qubetics, 11/03/2025]: The team heeded the advice and resolved the issue in commit 86c2db00ba26176472ba6e7eea8ace322ba56e01.

QCA-114 | Missing Authentication In HTTP Server

Category	Severity	Location	Status
Access Control	Minor	src/rpc_server/mod.rs (chain-abstraction): 479~506	Partially Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The provided HTTP server implementation in `start_http_server` function lacks authentication mechanisms for its endpoints, making it vulnerable to unauthorized access and potential abuse. Attackers can freely interact with sensitive endpoints (e.g., `submit_transaction`, `register_user`, `start_dkg`) without any verification, leading to risks such as:

1. DoS Attacks: Flooding endpoints with excessive requests.
2. Unauthorized Operations: Malicious actors could trigger critical functions (e.g., DKG initiation, reward claims).
3. Data Leakage: Sensitive user information (/user_info) is exposed without access control.

The current implementation:

`src/rpc_server/mod.rs`

```
let routes = submit_tx
        .or(register_user)
        .or(get_user_info)
        .or(get_node_id)
        .or(get_status)
        .or(get_dkg_status)
        .or(start_dkg)
        .or(user_mpc_deposit)
        .or(solver_reward)
        .or(get_reward_solver)
        .or(heartbeat)
        .or(uptime);

    info!("🌐 Starting HTTP RPC server on port {}", port);
    // Enable CORS for all origins and common HTTP methods/headers
    let cors = warp::cors()
    .allow_credentials(true)
    .allow_headers(vec!["Content-Type", "Authorization"])
    .allow_methods(vec!["GET", "POST", "OPTIONS"])
    .allow_origin("http://localhost:3000")
    .allow_origin("http://localhost:3001")
    .allow_origin("http://127.0.0.1:3000")
    .allow_origin("http://127.0.0.1:3001")
    .allow_origin("https://dev-frontend.qubetics.work");

    warp::serve(routes.with(cors))
        .run(([0, 0, 0, 0], port))
        .await;
```

All endpoints (`POST /submit_transaction`, `POST /register_user`, `GET /user_info`, etc.) are publicly accessible. Attackers can send unlimited requests (e.g., spamming `/register_user`), degrading server performance.

Recommendation

Consider using API keys, JWT tokens, or OAuth for sensitive endpoints.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and partially resolved the issue in commit ea8d536cdb63ea1d8fdf42588e046fd929c91143.

[CertiK, 12/24/2025]: The `start_dkg` endpoint still lacks of authentication protection in latest code:

```
let start_dkg = warp::path("start_dkg")
        .and(warp::post())
        .and(with_message_tx(message_tx.clone()))
        .and(with_dkg_storage(dkg_storage.clone()))
        .and_then(handle_start_dkg);
```

Consider adding `.and(with_auth())` to the `start_dkg` endpoint.

QCA-115 | Initializing Kademlia WIth Empty Bootstrap Nodes

Category	Severity	Location	Status
Logical Issue	Minor	src/network/mod.rs (chain-abstraction): 304~318	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The `start` function initializes an empty list (`bootstrap_nodes`) for Kademlia DHT bootstrapping, meaning the node lacks predefined peers to connect to when joining the network. This can lead to network fragmentation, delayed peer discovery, or complete isolation if no other peers are discovered via alternative methods (e.g., mDNS or manual dialing).

The issue occurs in the following snippet:

`src/network/mod.rs`

```
304                 let bootstrap_nodes: Vec<&'static str> = vec![];
305
306                 for addr in bootstrap_nodes {
307                     if let Ok(addr) = addr.parse::<Multiaddr>() {
308                         if let Some(peer_id) = addr.iter().find_map(|proto| {
309
                            if let libp2p::multiaddr::Protocol::P2p(multihash) =
proto {
310

PeerId::from_multihash(multihash.clone().into()).ok()
311                             } else {
312                                 None
313                             }
314                         }) {
315                             kademlia.add_address(&peer_id, addr);
316                         }
317                     }
318                 }
```

The `bootstrap_nodes` vector is empty, so the node does not attempt to connect to any initial peers. A well-designed P2P network should provide a set of trusted bootstrap nodes (either hardcoded or configurable) to ensure nodes can reliably join the network. The node relies on periodic Kademlia bootstrapping (`swarm.behaviour_mut().kademlia.bootstrap()`), but without initial peers, this may fail if no other nodes are discovered via gossipsub or manual dialing.

Impact

1. Network Isolation: If no other peers are discovered (e.g., via gossipsub or manual dialing), the node remains isolated. In a fresh network deployment, nodes may fail to form a connected overlay.
2. Slow Peer Discovery: Nodes depend on random peer discovery (`get_closest_peers`) or manual dialing (`swarm.dial()`), which is inefficient compared to connecting to known bootstrap peers.

Recommendation

Consider providing default bootstrap nodes.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 78369da7440b9d057a7657bd8850db5bd677545a.

QCA-116 | Hardcoded Bitcoin Testnet In `create_bitcoin_handler`

Category	Severity	Location	Status
Volatile Code	Minor	src/network/mod.rs (chain-abstraction): 1711, 2311	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The `create_bitcoin_handler` function currently hardcodes the use of Bitcoin `testnet`, which presents several limitations:

1. Prevents production deployment on Bitcoin mainnet
2. Lacks flexibility for environment-specific configurations
3. Cannot support multi-network scenarios where both testnet and mainnet operations are required

The current implementation:

`src/network/mod.rs`

```
 1711
let mut chain_handler =
crate::chain::ChainHandlerFactory::create_bitcoin_handler("testnet".to_string());
```

`src/network/mod.rs`

```
 2311
let mut chain_handler =
crate::chain::ChainHandlerFactory::create_bitcoin_handler("testnet".to_string());
```

The network parameter is hardcoded as "testnet" without configuration options.

Impact:

1. Unable to process real Bitcoin transactions in production
2. Limits the system's ability to handle multi-network operations

3. Violates principle of configuration over convention

Recommendation

Consider using environment-specific variable.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit ba3d8035f5f7666e720c4f25abf41ed58f33abc0.

QCA-117 Hardcoded Testnet Restriction In `create_bitcoin_script_pubkey`

Category	Severity	Location	Status
Logical Issue	Minor	src/utils/transaction.rs (chain-abstraction): 716~728, 1636	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The `create_bitcoin_script_pubkey` function enforces **Testnet-only** validation by hardcoding `Network::Testnet`, which is inflexible and incorrect for real-world usage. Bitcoin addresses can belong to multiple networks (`Mainnet`, `Testnet`, `Signet`, `Regtest`), and the function should support all valid networks instead of assuming `Testnet`.

The `classify_address` supports all valid networks:

`src/utils/transaction.rs`

```
716
let net = match [Network::Bitcoin, Network::Testnet, Network::Signet,
Network::Regtest]
717          .into_iter()
718          .find(|n| a.is_valid_for_network(*n))
719      {
720          Some(network) => {
721
            tracing::info!("✅ Address '{}' is valid for network: {:?}", addr,
network);
722              network
723          },
724          None => {
725
            tracing::error!("❌ Address '{}' is not valid for any known network",
addr);
726
            return Err(anyhow::anyhow!("Address '{}' is not valid for any known
network", addr));
727          }
728      };
```

However, the `create_bitcoin_script_pubkey` function forces Testnet validation, regardless of the actual address type:

`src/utils/transaction.rs`

```
1618  fn create_bitcoin_script_pubkey(address: &str) -> Result<Vec<u8>> {
1619      use bitcoin::{Address, Network};
1620      use std::str::FromStr;
1621
1622      // Parse the address and determine network
1623      let parsed_address = Address::from_str(address)
1624
        .map_err(|e| anyhow::anyhow!("Invalid Bitcoin address '{}': {}", address,
e))?;
1625
1626      // Only support P2PKH addresses
1627      // if !is_p2pkh_address(address) {
1628      //     return Err(anyhow::anyhow!(
1629
    //         "Only P2PKH addresses (1.../m.../n...) are supported, got: {}",
1630      //         address
1631      //     ));
1632      // }
1633
1634
    // Require the network to get a checked address. This preserves the network
encoded
1635
    // in the string (mainnet vs testnet/regtest) and prevents silent mismatches.
1636      let network = bitcoin::Network::Testnet;
1637      ...
1638      ...
```

If the system processes real Bitcoin transactions (Mainnet), the function will reject valid addresses.

Recommendation

Modify the function to accept an explicit Network parameter.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit ba3d8035f5f7666e720c4f25abf41ed58f33abc0.

QCA-118 | Potential Inconsistency In The Supported EVM Chains.

Category	Severity	Location	Status
Inconsistency	Minor	src/utils/transaction.rs (chain-abstraction): 234~252	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

There is an inconsistency between `get_chain_id_from_name` and `fetch_ethereum_nonce` in the supported EVM chains.

1. `get_chain_id_from_name` supports 8 EVM chains (Ethereum, Qubetics, Polygon, BSC, Avalanche, Arbitrum, Optimism, Base).
2. other function like `fetch_ethereum_nonce` only supports 2 chains (Ethereum 1, Qubetics 9029).

This mismatch can cause runtime errors when users attempt transactions on unsupported chains (e.g., Polygon, BSC).

The current implementation of `is_evm_chain` supports 8 EVM chains:

`src/utils/transaction.rs`

```
235 /// Chain mapping to convert string names to chain IDs
236 pub fn get_chain_id_from_name(chain_name: &str) -> Option<u64> {
237     match chain_name.to_lowercase().as_str() {
238         "ethereum" | "eth" => Some(1),
239         "qubetics" => Some(9029),
240         "polygon" | "matic" => Some(137),
241         "bsc" | "binance" => Some(56),
242         "avalanche" | "avax" => Some(43114),
243         "arbitrum" => Some(42161),
244         "optimism" => Some(10),
245         "base" => Some(8453),
246         _ => None, // Bitcoin and other non-EVM chains return None
247     }
248 }
249
250 /// Determine if a chain is EVM-compatible
251 pub fn is_evm_chain(chain_name: &str) -> bool {
252     get_chain_id_from_name(chain_name).is_some()
253 }
```

While the other function only support two EVM chains, for example:

`src/utils/transaction.rs`

```
548
pub async fn fetch_ethereum_nonce(address: &str, chain_id: u64) -> Result<u64> {
549     let rpc_url = match chain_id {
550         9029 => std::env::var("MPC_ETHEREUM_RPC_URL")
551
           .unwrap_or_else(|_| "https://rpc-testnet.qubetics.work".to_string()),
552
       1 => "https://mainnet.infura.io/v3/YOUR_PROJECT_ID".to_string(), // Ethereum
mainnet
553         _ => {
554             return Err(anyhow::anyhow!(
555                 "Unsupported chain ID for nonce fetching: {}",
556                 chain_id
557             ))
558         }
559     };
560     ...
561     ...
```

If a user initiates a transaction on Polygon (137) or BSC (56), `get_chain_id_from_name` will succeed, but fetch_ethereum_nonce will fail.

Recommendation

Consider refactoring the `get_chain_id_from_name` function.

Alleviation

[Qubetics, 12/18/2025]: The team heeded the advice and resolved the issue in commit ba3d8035f5f7666e720c4f25abf41ed58f33abc0.

QCA-119 | Solver Attribution Missing At Initial Tx-Id Store

Category	Severity	Location	Status
Volatile Code	Minor	src/signing/mod.rs (chain-abstraction): 2727~2728, 2757~2759	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

When a node stores the tx ID locally in `store_transaction_id_for_round`, it passes `None` as `solver_id` for both `UserToVault` and `VaultToNetwork`, so `final_solver_id` is left unset. Solver attribution is only added later via the broadcast handler.

```
//2724:2768:9d3440fe169a63244ca5f3080e36096e090865d6/src/signing/mod.rs
let (transaction_type_str, storage_result) = match transaction_type {
    Some(crate::types::TransactionType::UserToVault) => {
        let result = user_registry
            .store_user_to_vault_tx_id(intent_hash, tx_id, None)
            .await;
        ...
    },
    ...
    Some(crate::types::TransactionType::VaultToNetwork) => {
        let result = user_registry
            .store_vault_to_network_tx_id(intent_hash, tx_id, None)
            .await;
        ...
    },
```

- Broadcast handling later writes the same tx ID with `Some(node_id)`, which both (a) relies on broadcast delivery for attribution and (b) can overwrite an existing `intent_hash` with `final_solver_id` will fail to update it.

```
//1404:1451:9d3440fe169a63244ca5f3080e36096e090865d6/src/network/mod.rs
GossipsubMessage::TransactionIdBroadcast { intent_hash, transaction_id,
transaction_type, node_id } => {
    ...
    user_registry
        .store_user_to_vault_tx_id(&intent_hash, &transaction_id, Some(&node_id))
        .await
    ...
    user_registry
        .store_vault_to_network_tx_id(&intent_hash, &transaction_id, Some(&node_id))
        .await
}
```

Recommendation

Set `solver_id` to `Some(&self.id)` in the initial `store_user_to_vault_tx_id` and `store_vault_to_network_tx_id` calls inside `store_transaction_id_for_round`, so attribution is captured even without broadcast.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue in commit f84e4cc72375d7c467137683108459520bd3f1d2

QCA-13 | Potential Discrepancy In `derive_peer_id_from_string`

Category	Severity	Location	Status
Logical Issue	Minor	src/signing/db_signing.rs (chain-abstraction): 73~84	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/db_signing.rs`

The `derive_peer_id_from_string` function claims to generate a deterministic keypair using a string hash as seed, but its implementation fails to use the computed hash, instead generating a completely random keypair. This creates a discrepancy between the documented behavior and actual functionality.

The current implementation:

`src/signing/db_signing.rs`

```
74  fn derive_peer_id_from_string(id: &str) -> PeerId {
75          use libp2p::identity::Keypair;
76          use sha2::{Digest, Sha256};
77
78          let mut hasher = Sha256::new();
79          hasher.update(id.as_bytes());
80          let _hash = hasher.finalize();
81          // Use the hash as seed to generate a deterministic keypair
82          let keypair = Keypair::generate_ed25519();
83          PeerId::from(keypair.public())
84      }
```

The `_hash` is computed but never used. It generates random key by calling `Keypair::generate_ed25519()`. However, the comment states: "Use the hash as seed to generate a deterministic keypair".

Recommendation

Consider using hash as seed for deterministic key generation, for example:

```
let keypair = Keypair::ed25519_from_bytes(_hash);
PeerId::from(keypair.public())
```

Alleviation

[Qubetics, 11/26/2025]: The team heeded the advice and resolved the issue by removing the function in commit 16322e709f0d3be0cde8e097114a2dbd11f4c585.

QCA-25 | Missing Public Key Hash Validation In Legacy P2PKH Handling

Category	Severity	Location	Status
Logical Issue	Minor	src/signing/mod.rs (chain-abstraction): 409~431	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `sign_bitcoin_transaction` function contains a critical security flaw in its handling of Legacy P2PKH (Pay-to-Public-Key-Hash) transactions. Unlike the P2WPKH (Pay-to-Witness-Public-Key-Hash) branch, which properly verifies that the witness program matches the signer's public key hash (`signer_h160`), the P2PKH branch fails to validate whether the script's embedded public key hash matches `signer_h160` .

This oversight could allow malicious actors to:

1. Trick nodes into signing transactions for UTXOs they don't own.
2. Waste computational resources on invalid signing attempts.
3. Potentially enable replay attacks if combined with other protocol weaknesses.

The function correctly enforces that the witness program (`program20`) matches `signer_h160` :

`src/signing/mod.rs`

```
392  if is_p2wpkh {
393              // --- P2WPKH (v0) ---
394              let prev = maybe_prev.expect("Some above");
395              let program20 = &prev.script_pubkey[2..22];
396
397              // Safety: signer pubkey must match the witness program
398              if signer_h160 != program20 {
399                  anyhow::bail!(
400
                    "Input {}: signer HASH160(pubkey) != witness program ({} !=
{})",
401                      i,
402                      hex::encode(signer_h160),
403                      hex::encode(program20)
404                  );
405              }
406
407
            let digest = self.compute_bip143_sighash_p2wpkh(tx, i, prev.value,
program20)?;
408              digests.push(digest);
409          }
```

However, in `P2PKH` branch, the code blindly accepts `prev.script_pubkey` without verifying its embedded hash:

`src/signing/mod.rs`

```
412  let prev_spk: Vec<u8> = if let Some(prev) = maybe_prev {
413                  prev.script_pubkey.clone()
414              }
```

Scenario

An attacker could craft a transaction where:

1. A UTXO's script_pubkey appears to be a valid P2PKH script (OP_DUP OP_HASH160 PUSH20 OP_EQUALVERIFY OP_CHECKSIG).
2. However, the does not correspond to the signer's public key (signer_h160).
3. The node would still proceed with signing, producing an invalid or unintended signature.

Recommendation

Consider following fix:

```
if prev_spk.len() != 25
                || !(prev_spk[0] == 0x76 && prev_spk[1] == 0xa9 && prev_spk[2] ==
0x14
                    && prev_spk[23] == 0x88 && prev_spk[24] == 0xac)
            {
                anyhow::bail!("Input {}: prevout script is not canonical P2PKH", i);
            }

    // Extract & compare pubkey hash
    let script_h160 = &prev.script_pubkey[3..23];
    if signer_h160 != script_h160 {
        anyhow::bail!(
            "Input {}: Script pubkey hash mismatch (expected {}, got {})",
            i,
            hex::encode(signer_h160),
            hex::encode(script_h160)
        );
    }
    ...
    ...
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit bdc1facd1851063a1f6da950e258900915dcbdd4.

QCA-26 | Unsafe Nonce Handling

Category	Severity	Location	Status
Logical Issue	Minor	src/signing/mod.rs (chain-abstraction): 840	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `fetch_ethereum_nonce` uses an unsafe fallback mechanism (`unwrap_or("0x0")`) when parsing Ethereum nonce values from JSON-RPC responses. This creates critical reliability and security issues in transaction processing.

The current implementation:

`src/signing/mod.rs`

```
 839  if let Some(nonce_hex) = result.get("result") {
 840      let nonce_str = nonce_hex.as_str().unwrap_or("0x0");
 841
    let nonce =
u64::from_str_radix(nonce_str.strip_prefix("0x").unwrap_or(nonce_str), 16)?;
 842      info!("✅ Fetched nonce {} for address {}", nonce, address);
 843      Ok(nonce)
 844  }
```

Ethereum nonces are sequential counters that MUST increment exactly by 1 for each new transaction from an address. Nodes enforce strict ordering: A transaction with `nonce=0` will only succeed if the address has no prior transactions. If `nonce_hex.as_str()` fails (e.g., due to malformed RPC response), the code silently defaults to `nonce=0` . Transactions will be rejected with "nonce too low" errors unless the address is brand new.

Recommendation

Consider returning error if the nonce is missing or malformed, for example:

```
let nonce_str = nonce_hex.as_str()
    .ok_or_else(|| anyhow!("Nonce must be a string"))?; // No unwrap_or!
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit e4399a3d04e68bb1190509830b368b824b6768a1.

QCA-27 | VRF Secret Key Exposure

Category	Severity	Location	Status
Volatile Code	Minor	src/vrf/mod.rs (chain-abstraction): 385~393	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/vrf/mod.rs

`get_secret_key()` and `get_secret_key_hex()`, which directly expose the VRF (Verifiable Random Function) private key. The VRF private key is a critical cryptographic secret used to generate unpredictable and verifiable random numbers.

These functions are not currently called anywhere in the project. Therefore, there is no immediate exploitable vulnerability. However, the existence of these functions in production code constitutes a high-severity latent risk. They create a "loaded gun" that could be accidentally used by developers in the future, leading to a critical compromise of the chain's randomness mechanism.

```
    /// Get VRF secret key (for testing purposes)
    pub fn get_secret_key(&self) -> Vec<u8> {
        self.vrf_secret_key.to_bytes().to_vec()
    }
```

Recommendation

If these functions are remnants of a past debugging effort and are no longer used in tests, they should be removed completely.

Alleviation

[Qubetics, 11/26/2025]: The team heeded the advice and resolved the issue in commit 26632fd2868875d220da704af56d96256d0a3749

QCA-44 Potential Concurrency Issues In `calculate_reward_per_solver`

Category	Severity	Location	Status
Concurrency	Minor	src/signing/mod.rs (chain-abstraction): 4259~4266	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `calculate_reward_per_solver` function is susceptible to race conditions when processing the same `intent_hash` concurrently. This could lead to duplicate reward calculations, violating the single-processing guarantee required for financial correctness.

The function first checks `intent_reward_status`:

`src/signing/mod.rs`

```
 4260
if let Some(true) = user_registry.get_intent_reward_status(intent_hash).await {
 4261             info!(
 4262                 "⏭️ [SOLVER] Rewards already calculated for intent {}",
 4263                 intent_hash
 4264             );
 4265             return Ok(());
 4266 }
```

and later updates it:

`src/signing/mod.rs`

```
 4367 user_registry
 4368             .set_intent_reward_status(intent_hash, true)
 4369             .await
 4370
          .map_err(|e| anyhow!("Failed to set intent reward status: {}", e))?;
```

but these steps are not atomic. Under high concurrency, multiple threads/processes might simultaneously see `status = false` and proceed to compute rewards.

Impact:

1. Duplicate Rewards: Solvers may receive multiple payouts for the same intent.
2. Data Corruption: The `solver_reward_btc` / `solver_reward_tics` balances could become incorrect.
3. Financial Loss: If rewards are over-distributed, the protocol may lose funds.

Recommendation

Consider using a lock to guard intent processing.

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue in commit 6bed6a7b36a840070eb6b62df68a1ed36a710780.

QCA-45 | Duplicate Processing Of Consensus Messages

Category	Severity	Location	Status
Logical Issue	Minor	src/network/mod.rs (chain-abstraction): 1244~1256; src/signing/mod.rs (chain-abstraction): 3079~3084	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`
- `src/network/mod.rs`

There is a duplicate processing issue in the handling of `ConsensusMessage`, where the same message may be processed twice by the `consensus_node` under certain conditions.

The current implementation:

`src/network/mod.rs`

```
 1244  GossipsubMessage::Consensus(consensus_msg) => {
 1245
               info!("📥 [CONSENSUS] Received consensus message: {:?}",
consensus_msg);
 1246
 1247                  // Handle consensus message in signing node
 1248                  if let Some(signing_node) = &mut self.signing_node {
 1249
                      signing_node.handle_consensus_message(consensus_msg.clone()).await?;
 1250                  }
 1251
 1252                  // Also handle in standalone consensus node if available
 1253                  if let Some(consensus_node) = &mut self.consensus_node {
 1254                      consensus_node.handle_message(consensus_msg).await?;
 1255                  }
 1256              }
```

The code forwards the `GossipsubMessage::Consensus` message to both `signing_node` and `consensus_node` (if they exist). However, the `signing_node.handle_consensus_message()` method also forwards the message to the `consensus_node` internally:

`src/signing/mod.rs`

```
 3079
pub async fn handle_consensus_message(&mut self, msg: ConsensusMessage) ->
Result<()> {
 3080         if let Some(consensus_node) = &mut self.consensus_node {
 3081             consensus_node.handle_message(msg).await?;
 3082         }
 3083         Ok(())
 3084     }
```

This results in redundant execution of `consensus_node.handle_message()` when both nodes are active.

Recommendation

Consider modifying the code logic to avoid duplicate forwarding.

Alleviation

[Qubetics, 12/12/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> Since `GossipsubMessage::Consensus` is effectively unused, I think we can skip this audit ticket. Your analysis is only correct if that message type is actually used.

QCA-46 | No Guard Against `share_index == 0`

Category	Severity	Location	Status
Volatile Code	Minor	src/adkg_secp256k1/mod.rs (chain-abstraction): 478~510	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/adkg_secp256k1/mod.rs

`share_index` in `verify_share()` is never validated to be not 0. If an index of 0 ever slipped in, all higher powers of x collapse to zero, so `expected` reduces to the constant-term commitment, the code would verify the polynomial at x = 0, effectively checking the constant term (`commitments[0]`) instead of a participant's share. Any scalar equal to that constant term will pass verification, completely bypassing checks for the intended receiver index. The surrounding code (`RPCRequest::SendAVSSShare`) currently accepts an externally supplied index, so this becomes a practical way to inject bogus shares that still verify.

```
    pub fn verify_share(
        &self,
        share: &SerializableScalar,
        share_index: usize,
        commitments: &[SerializablePoint],
    ) -> bool {
        if commitments.is_empty() {
            return false;
        }
        let x = Scalar::from(share_index as u64);
        let mut expected = ProjectivePoint::IDENTITY;

        for (j, commitment) in commitments.iter().enumerate() {
// ...
  }
```

Recommendation

Validate `share_index` against the known participant count and disallow 0.

Alleviation

[Qubetics, 11/26/2025]: The team heeded the advice and resolved the issue in commit 067abbaffde3f0b4970b21c7df99400aff6093a8

QCA-58 The Network Does Not Remove Nodes Upon Unsubscription And ConnectionClosed

Category	Severity	Location	Status
Logical Issue	Minor	src/network/mod.rs (chain-abstraction): 1077~1113, 1127~1148, 2069~2101	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The current implementation adds peers to the `node_manager` when they subscribe to the discovery topic (`MSG_TOPIC_PEER_DISCOVERY`) but does not remove them upon unsubscription or close. This leads to inaccurate node counts, which can affect:

1. Threshold calculations (e.g., consensus requirements).
2. Network health monitoring (e.g., falsely assuming disconnected peers are still active).

The current behavior:

`src/network/mod.rs`

```
1077  SwarmEvent::Behaviour(NetworkEvent::Gossipsub(gossipsub::Event::Subscribed {
1078                  peer_id,
1079                  topic,
1080              })) => {
1081                  info!("📡 Peer {} subscribed to topic: {}", peer_id, topic);
1082
1083
                // If this is the peer discovery topic, we can now safely broadcast
our presence
1084                  if topic.as_str() == MSG_TOPIC_PEER_DISCOVERY {
1085                      info!(
1086
                       "🎯 Peer discovery topic subscription confirmed by: {}",
1087                          peer_id
1088                      );
1089
1090                      // Small delay to ensure subscription is fully processed
1091                      // tokio::time::sleep(Duration::from_millis(50)).await;
1092
1093                      // Broadcast our presence to this newly subscribed peer
1094                      let msg = GossipsubMessage::PeerDiscovered {
1095                          peer: self.local_peer_id.clone(),
1096                          sequence: Some(1), // Add sequence number
1097                      };
1098                      if let Ok(payload) = serde_json::to_vec(&msg) {
1099
                       if let Err(e) = self.broadcast(MSG_TOPIC_PEER_DISCOVERY,
&payload).await {
1100                              warn!(
1101
                               "Failed to broadcast presence to newly subscribed
peer: {:?}",
1102                                  e
1103                              );
1104                          }
1105                      }
1106                  }
1107              }
```

When the `Subscribed` event is discovered, if the topic is `MSG_TOPIC_PEER_DISCOVERY`, the node broadcasts its presence via `PeerDiscovered`. Receivers add the peer to `node_manager` (if not already present): **`src/network/mod.rs`**

```
 2069  GossipsubMessage::PeerDiscovered { peer, sequence } => {
 2070                  info!(
 2071                      "🎯 Peer discovered via gossipsub: {:?} (seq: {:?})",
 2072                      peer, sequence
 2073                  );
 2074
 2075                  // Check if we already have this peer to avoid duplicates
 2076                  let current_nodes = self.node_manager.get_active_nodes();
 2077                  if current_nodes.contains(&peer) {
 2078
                  info!("ℹ️ Peer {} already exists in node manager, skipping",
peer);
 2079                      return Ok(());
 2080                  }
 2081
 2082                  match self.node_manager.add_node(peer) {
 2083                      Ok(_) => {
 2084
                       info!("✅ Successfully added peer {} to node manager",
peer);
 2085
 2086                          // Log updated node count
 2087                          let total = self.node_manager.get_total_nodes();
 2088                          let threshold = self.node_manager.get_threshold();
 2089
                       info!("📊 Updated node count: {}, threshold: {}", total,
threshold);
 2090
                       // Update signing node threshold to match current network
state
 2091                          if let Some(signing_node) = &mut self.signing_node {
 2092
                           let new_threshold = self.node_manager.get_threshold();
 2093                              signing_node.set_threshold(new_threshold);
 2094
                           info!("🔄 [THRESHOLD] Updated signing node threshold to
{} (total nodes: {})", new_threshold, total);
 2095                          }
 2096                      }
 2097                      Err(e) => {
 2098
                       warn!("❌ Failed to add peer {} to node manager: {:?}",
peer, e);
 2099                      }
 2100                  }
 2101              }
```

However, when an `Unsubscribed` or `ConnectionClosed` event is discovered, it only logs a warning (warn!):

`src/network/mod.rs`

```
1108
SwarmEvent::Behaviour(NetworkEvent::Gossipsub(gossipsub::Event::Unsubscribed {
1109                 peer_id,
1110                 topic,
1111             })) => {
1112
             warn!("📡 Peer {} unsubscribed from topic: {}", peer_id, topic);
1113             }
```

```
1127 SwarmEvent::ConnectionClosed {
1128                 peer_id: _,
1129                 connection_id: _,
1130                 endpoint: _,
1131                 num_established: _,
1132                 cause,
1133             } => {
1134                 if let Some(ca) = cause {
1135                     match ca {
1136                         ConnectionError::IO(e) => {
1137                             info!(
1138
                                 "[ERROR] Kind: {} RAW: {:?} ref: {:?} Full: {}",
1139                                 e.kind(),
1140                                 e.raw_os_error(),
1141                                 e.get_ref().as_slice(),
1142                                 e
1143                             );
1144                         }
1145                         _ => {}
1146                     }
1147                 }
1148             }
```

There is no removal from `node_manager`, causing stale entries.

Impact

1. Inflated peer count: `get_total_nodes()` includes disconnected peers.
2. Incorrect thresholds: Since `get_threshold()` depends on `total_nodes`, the consensus will become harder.
3. Stale network view: The system may attempt to communicate with unreachable peers.

Recommendation

Consider removing Peers on Unsubscription and ConnectionClosed.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit

75b3ac44e94d7e6a4fa01ed1fd6ff97d8d7182c7.

QCA-59 | Hardcoded Chain Code Undermines Key Derivation Entropy

Category	Severity	Location	Status
Volatile Code	Minor	src/main.rs (chain-abstraction): 121; src/network/mod.rs (chain-abstraction): 1366~1370	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- src/main.rs
- src/network/mod.rs The `chain_code` used to seed the HMAC for key tweaking is initialized with a static, hardcoded byte array [42u8; 32] instead of a secure random value. This makes the tweak generation deterministic and public.

```
    fn generate_hmac_constant(&self, address: &str) -> [u8; 32] {
        hmac256_from_addr(address, &self.chain_code).unwrap_or_else(|_| [0u8; 32])
        // Fallback to zero array if HMAC fails
    }
```

In MPC and HD Wallet (Hierarchical Deterministic) systems, a chain_code is typically used alongside a master key to derive child keys (tweaked keys) in a way that produces unique, uncorrelated keys for different purposes (e.g., different users or vaults). The code implements a form of additive key derivation where:

`Child_Share = Master_Share + HMAC(Address, Chain_Code)`

This "tweak" (the HMAC output) is intended to effectively salt the key derivation. If the chain_code is random and secret, the relationship between the Master Key and Child Key is hidden.

The impacts are:

- All user “tweaks,” user-specific group keys, and derived ETH/BTC addresses become predictable from public information (address + published commitments). Privacy and unlinkability guarantees collapse; observers can precompute every future deposit address and correlate them with the user’s canonical address.
- If an attacker manages to compromise a single user's key share (or the vault's key share), they can trivially compute the Master DKG Secret Share by subtracting the known public tweak.

Recommendation

Treat `chain_code` as a secret key: load it from secure configuration (environment variable, vault, HSM) or generate it per deployment and store it securely. Rotate existing deployments to a high-entropy secret so that future HMAC-derived tweaks regain unpredictability.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit 3aab22bf177592ac47fe1996ab7b91b370092569

QCA-71 | No Proof Of Address Ownership Is Required For Registration

Category	Severity	Location	Status
Volatile Code	Minor	src/rpc_server/mod.rs (chain-abstraction): 911~912, 911~912	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

`handle_register_user` only checks that the supplied Ethereum address is well-formed and unique, then immediately derives tweaked shares and addresses for it; the user never has to sign anything to prove they control that key. A malicious actor can preregister victim addresses, obtaining the derived addresses and potentially blocking legitimate users or steering deposits to attacker-controlled MPC outputs.

```
//src/rpc_server/mod.rs
let addr_cs = match validate_ethereum_address(&request.ethereum_address) { ... }
if let Some(_existing_user) = user_registry.get_user_by_address(&addr_cs).await {
    ...
}
match user_registry.register_user(&addr_cs, dkg_secret_share.as_ref(),
&node_id).await { ... }
```

Recommendation

- Add a signature (and optionally the original message) to `UserRegistrationRequest`. On the client side, have the user sign a challenge such as `keccak256("\x19Ethereum Signed Message:...register:address:nonce")` using personal_sign/eth_sign.
- On the server side (`handle_register_user`), verify the signature with `recover_eth_address_secp` against the normalized `ethereum_address` before touching the registry. Reject if the recovered address doesn't match. Keep a short-lived nonce or timestamp in the signed payload so the proof can't be replayed.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue by add signature in `userRegistreationRequest` in commit 80f67e68650f74033ca4a4091e4a90e6dcea58af

QCA-85 Transactions With Unknown Transaction Type Should Not Be Handled

Category	Severity	Location	Status
Logical Issue	Minor	src/network/mod.rs (chain-abstraction): 1452~1455, 2698~2701	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The current implementation of `GossipsubMessage::TransactionIdBroadcast` and `ChannelMessage::TransactionIdBroadcast` incorrectly allows the logic to proceed when an unknown `transaction_type` is encountered, due to a misplaced `return Ok(())` inside a match branch. This leads to `storage_result` being assigned `Ok(())`, causing unintended continuation of the execution flow.

The current logic of `GossipsubMessage::TransactionIdBroadcast`:

`src/network/mod.rs`

```
1411  if let Some(user_registry) = &self.user_registry {
1412                      let storage_result = match transaction_type.as_str() {
1413                          "user_to_vault" => {
1414
                             // Also update solver amounts for user_to_vault using
signing node, if available
1415                                 if let Some(signing_node) = &self.signing_node {
1416
                                  if let Err(e) =
signing_node.update_solver_amounts_for_user_to_vault(&intent_hash).await {
1417
                                     warn!("⚠️ [SIGNING] Failed to update solver
amounts for UserToVault transaction: {}", e);
1418                                    } else {
1419
                                     // Calculate rewards after successfully updating
solver amounts
1420
                                     if let Err(e) =
signing_node.calculate_reward_per_solver(&intent_hash).await {
1421
                                        warn!("⚠️ [SIGNING] Failed to calculate
rewards for UserToVault transaction: {}", e);
1422                                        } else {
1423
                                        info!("✅ [SIGNING] Successfully calculated
rewards for UserToVault transaction: {}", intent_hash);
1424                                        }
1425                                    }
1426                                }
1427                                user_registry
1428                                    .store_user_to_vault_tx_id(
1429                                        &intent_hash,
1430                                        &transaction_id,
1431                                        Some(&node_id),
1432                                    )
1433                                    .await
1434                            },
1435                            "network_to_target" => {
1436                                user_registry
1437                                    .store_network_to_target_tx_id(
1438                                        &intent_hash,
1439                                        &transaction_id,
1440                                        Some(&node_id),
1441                                    ).await
1442                            },
1443                            "vault_to_network" => {
1444                                user_registry
1445                                    .store_vault_to_network_tx_id(
1446                                        &intent_hash,
1447                                        &transaction_id,
1448                                        Some(&node_id),
```

```
 1449                                    )
 1450                                    .await
 1451                            },
 1452                            _ => {
 1453
                             warn!("❌ [NETWORK] Unknown transaction type: {}",
transaction_type);
 1454                                return Ok(());
 1455                            }
 1456                        };
 1457
 1458                        match storage_result {
 1459                            Ok(()) => {
 1460                                if let Err(e) = user_registry
 1461
                                 .store_transaction_status(&transaction_id,
TransactionStatus::Pending)
 1462                                    .await
 1463                                {
 1464                                    error!(
 1465
                                     "❌ [NETWORK] Failed to store transaction status
for broadcast tx {}: {}",
 1466                                        transaction_id, e
 1467                                    );
 1468                                }
 1469                                info!(
 1470
                                "✅ [NETWORK] Successfully synchronized transaction
ID {} for intent {} (type: {})",
 1471                                    transaction_id, intent_hash, transaction_type
 1472                                );
 1473
 1474
                             // Execute contract call when transaction ID is received
 1475
                             if let Some(user_registry) = &self.user_registry {
 1476
                                 if let Some(intent) =
user_registry.get_intent(&intent_hash).await {
 1477
                                     // Convert transaction type string to enum
 1478
                                     let tx_type_enum = match
transaction_type.as_str() {
 1479
                                         "user_to_vault" =>
crate::types::TransactionType::UserToVault,
 1480
                                         "network_to_target" =>
crate::types::TransactionType::NetworkToTarget,
 1481
```

```
                                    "vault_to_network" =>
crate::types::TransactionType::VaultToNetwork,
 1482                                        _ => {
 1483
                                        warn!("❌ [CONTRACT] Unknown transaction
type for contract call: {}", transaction_type);
 1484                                            return Ok(());
 1485                                        }
 1486                                    };
 1487
 1488                                    // Get vault group key
 1489
                                    let vault_group_key = if let Some(dkg_node) =
&self.dkg_node {
 1490                                        dkg_node.get_vault_group_key().await
 1491                                    } else {
 1492                                        None
 1493                                    };
 1494
 1495
                                    if let Some(vault_group_key) = vault_group_key {
 1496
                                        // Use transaction_id as tx_hash for
contract call
 1497
                                        // let tx_hash_bytes =
transaction_id.clone();
 1498                                        // Execute contract call
 1499
                                        if let Err(e) =
self.execute_contract_call_for_transaction(
 1500                                            &intent,
 1501                                            &intent_hash,
 1502                                            tx_type_enum,
 1503                                            &transaction_id,
 1504                                            Some(vault_group_key),
 1505                                            intent.amount,
 1506                                        ).await {
 1507
                                        warn!("❌ [CONTRACT] Failed to execute
contract call for transaction ID {}: {}", transaction_id, e);
 1508                                        } else {
 1509
                                        info!("✅ [CONTRACT] Successfully
executed contract call for transaction ID {}", transaction_id);
 1510                                        }
 1511                                    } else {
 1512
                                    warn!("❌ [CONTRACT] No vault group key
available for contract call execution");
 1513                                    }
 1514                                } else {
 1515
```

```
                                warn!("❌ [CONTRACT] Could not retrieve intent
{} for contract call execution", intent_hash);
 1516                                }
 1517                            }
 1518                        },
 1519                        Err(e) => {
 1520
                         error!("❌ [NETWORK] Failed to store synchronized
transaction ID: {}", e);
 1521                        }
 1522                    }
 1523                } else {
 1524
                 warn!("❌ [NETWORK] No user registry available to store
synchronized transaction ID");
 1525                }
```

When an unknown `transaction_type` is received, the code logs a warning and `returns Ok(())` inside the match branch. However, this `return` only exits the `match` arm, not the entire function. As a result, `storage_result` is set to `Ok(())`, and the subsequent logic (e.g., storing transaction status, executing contract calls) proceeds incorrectly.

The same issue exists in the logic of `ChannelMessage::TransactionIdBroadcast`: **`src/network/mod.rs`**

```
 2698  _ => {
 2699      warn!("❌ [NETWORK] Unknown transaction type: {}", transaction_type);
 2700      return Ok(());
 2701  }
```

Recommendation

Consider returning an `Error` in the Match Branch, for example:

```
let storage_result = match transaction_type.as_str() {
    // ... valid cases ...
    _ => {
        warn!("❌ [NETWORK] Unknown transaction type: {}", transaction_type);
        return Err(anyhow!("Unknown transaction type: {}", transaction_type));
    }
};
```

Alleviation

[Qubetics, 12/23/2025]: The team heeded the advice and resolved the issue in commit ebef65083275f40584bbc34445965ae889d3a144.

QCA-86 | Incorrect Implementation Of `handle_get_status`

Category	Severity	Location	Status
Volatile Code	Minor	src/rpc_server/mod.rs (chain-abstraction): 2108~2118	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The current implementation of `handle_get_status` returns hardcoded dummy data, which provides misleading information to API consumers. This needs to be fixed to return actual node status information including real peer ID, connection counts, DKG status, and proper readiness indication.

The current implementation:

`src/network/mod.rs`

```
2108  async fn handle_get_status(
2109
    _message_tx: std::sync::Arc<tokio::sync::Mutex<mpsc::Sender<ChannelMessage>>>,
2110  ) -> Result<impl warp::Reply, warp::Rejection> {
2111      info!("here at handle status!! ");
2112      Ok(warp::reply::json(&NodeStatus {
2113          peer_id: "dummy-peer-id".to_string(),
2114          connected_peers: 0,
2115          dkg_status: "unknown".to_string(),
2116          is_ready: true,
2117      }))
2118  }
```

The `peer_id` is always returned as `dummy-peer-id`. The `dkg_status` is always returned as `unknown`. The `connected_peers` is always returned as 0. The `is_ready` is always returned as true.

Impact:

1. Monitoring systems cannot detect real node status
2. Administrators lack visibility into node health

3. DKG coordination may fail due to inaccurate readiness reporting

Recommendation

Consider providing accurate, reliable node status information in `handle_get_status`.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 9e017be8ba6e48500330fb56ff307fb85bdc2353.

QCA-87 The `select_utxos_for_amount()` Function Fails To Enforce The Dust Limit (546 Satoshis) On The Recipient Output

Category	Severity	Location	Status
Logical Issue	Minor	src/utils/transaction.rs (chain-abstraction): 408~416	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The `select_utxos_for_amount()` function in the Bitcoin UTXO selection logic fails to enforce the dust limit (546 satoshis) on the recipient output (`required_amount`). While it correctly checks that the change output meets the dust threshold, it does not validate whether the primary payment output is above the minimum relayable amount. This oversight allows the creation of non-standard transactions that Bitcoin nodes will reject, leading to potential denial-of-service (DoS) attacks, UTXO locking, and transaction failures.

The function `select_utxos_for_amount()` ensures that change outputs (`change >= DUST_LIMIT_SATS`) are valid but does not verify that the `required_amount` (the recipient's output) meets the same dust limit:

`src/utils/transaction.rs`

```
482  if total_value >= required_with_change {
483              let change = total_value - required_with_change;
484
485              if change >= DUST_LIMIT_SATS {
486                  // Use change path - this is optimal
487                  tracing::info!(
488
                    "✅ [UTXO_SELECTION] Using change path with {} inputs (fee {}
sats, change {} sats)",
489                      n_inputs,
490                      fee_with_change,
491                      change
492                  );
493
494                  return Ok(UtxoSelectionResult {
495                      selected_utxos: selected,
496                      total_value,
497                      fee: fee_with_change,
498                      change,
499                      change_output: true,
500                  });
501              }
502          }
```

Bitcoin nodes enforce a minimum output value (dust limit) for all transaction outputs (`BIP-125`). Transactions with outputs below this threshold are considered non-standard and will not be relayed.

Exploitable Scenario:

1. An attacker can repeatedly request transactions with `required_amount < 546 satoshis` .
2. The function will approve these transactions, but they will fail when broadcast to the Bitcoin network.
3. This can lead to:

- UTXO lockups (temporarily stuck in mempool).
- Service disruption (failed transactions requiring manual intervention).
- Spam attacks (flooding the system with invalid transactions).

Recommendation

Consider adding a dust check for the recipient output at the start of `select_utxos_for_amount()` :

```
if required_amount < DUST_LIMIT_SATS {
    return Err(anyhow!("Recipient amount below dust limit (min: {} sats)",
DUST_LIMIT_SATS));
}
```

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 2d7f2ed4032ef8c664e2a8def8605956575aff76.

QCA-88 The `create_chain_transaction_from_deposit_intent_internal` Function Unconditionally Unwraps `group_public_key` Before Transaction Type Evaluation

Category	Severity	Location	Status
Logical Issue	Minor	src/utils/transaction.rs (chain-abstraction): 796~833	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/utils/transaction.rs`

The `create_chain_transaction_from_deposit_intent_internal` function unconditionally unwraps `group_public_key` before transaction type evaluation, causing panics for legitimate `UserToVault` transactions on EVM chains where the group key isn't required. This creates a denial-of-service (DoS) vector where any user can crash the service process by submitting properly formed but incomplete requests.

The function prematurely unwraps `group_public_key` before determining if it's actually needed:

`src/utils/transaction.rs`

```
 796  // Problematic flow:
 797
let network_address = get_eth_address_from_group_key(group_public_key.unwrap());
 798  // ^-- Early unwrap before transaction type check
 799
 800  match transaction_type {
 801      TransactionType::UserToVault => {
 802          // Uses derived_eth_addr instead, making group_public_key unnecessary
 803      }
 804      // ... other cases actually need group_public_key
 805  }
```

For `TransactionType::UserToVault` (on EVM chains), `group_public_key` is not needed—the transaction uses the user's `derived_eth_addr` as the sender and the vault address as the recipient. If `group_public_key` is `None` (a legitimate input

for this flow), the `unwrap` triggers a panic instead of returning a Result error. Because the function is async and intended to be used in a service context, this panic is exploitable as a denial-of-service: a single call can crash the request path (and typically the process), preventing transaction creation for other users.

Recommendation

Refactor `create_chain_transaction_from_deposit_intent_internal` to extract and use `group_public_key` only when required by the `transaction_type` logic.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit 6f46721b27790b24529f2d1789424d5e51d5c381.

QCA-89 | Ignored `manual_max_fee` In `derive_fees` Fallback Path

Category	Severity	Location	Status
Logical Issue	Minor	src/intent_manager/mod.rs (chain-abstraction): 103~106	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/intent_manager/mod.rs`

The `derive_fees` function in the provided code has a critical logical flaw where a user-specified `manual_max_fee` is silently ignored when the Ethereum RPC provider fails to return the latest block (i.e., `get_block(Latest)` returns `None`). Instead of respecting the caller's intended gas fee cap, the function falls back to computing `max_fee = gp + prio` (where `gp` is the current gas price and `prio` is the priority fee).

The `derive_fees` function is designed to compute EIP-1559 fee parameters (`max_fee_per_gas` and `max_priority_fee_per_gas`) with the following logic:

`src/intent_manager/mod.rs`

```
async fn derive_fees(
    provider: &Provider<Http>,
    manual_max_fee: Option<U256>,
    manual_max_priority: Option<U256>,
) -> Result<(U256, U256)> {
    if let (Some(mf), Some(mp)) = (manual_max_fee, manual_max_priority) {
        return Ok((mf, mp));
    }
    let default_prio = U256::from(1_500_000_000u64); // ~1.5 gwei

    if let Some(block) = provider
        .get_block(BlockId::Number(BlockNumber::Latest))
        .await?
    {
        let base = block.base_fee_per_gas.unwrap_or_default();
        let prio = manual_max_priority.unwrap_or(default_prio);
         let max  = manual_max_fee.unwrap_or(base * 2 + prio);
         Ok((max, prio))
     } else {
         // fallback for weird RPCs
         let gp   = provider.get_gas_price().await?;
         let prio = manual_max_priority.unwrap_or(default_prio);
         Ok((gp + prio, prio))
     }
}
```

When `provider.get_block(Latest)` returns `None`, the function falls back to using `provider.get_gas_price()` and computes fees as (`gp + prio, prio`). In this fallback branch, `manual_max_fee` is never used, even if the caller supplied it. As a result, a caller-specified hard cap on `max_fee_per_gas` is silently bypassed whenever the RPC returns `None` for the latest block. This violates the expected invariant that a provided `manual_max_fee` is always respected.

Recommendation

Update the fallback logic in derive_fees to enforce manual_max_fee whenever it is provided.

Alleviation

[Qubetics, 12/12/2025]: The team heeded the advice and resolved the issue in commit e48e91b76350d08bdfc4160005c5abd8704567a9.

QCA-90 Signature Verification Bypass In Ethereum And Bitcoin Handlers

Category	Severity	Location	Status
Logical Issue	Minor	src/chain/mod.rs (chain-abstraction): 311~341, 658~695	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/chain/mod.rs`

Both `verify_signature` implementations (`Ethereum` and `Bitcoin`) contain critical security flaws that allow attackers to bypass signature verification entirely.

The current implementation:

`src/chain/mod.rs`

```
311 fn verify_signature(
312         &self,
313         transaction: &ChainTransaction,
314         signature: &ChainSignature,
315         public_key: &[u8],
316     ) -> Result<bool> {
317         match (transaction, signature) {
318 
          (ChainTransaction::Ethereum(eth_tx), ChainSignature::Ethereum(eth_sig))
=> {
319                 // Basic validation
320                 if eth_sig.r.len() != 32 || eth_sig.s.len() != 32 {
321                     return Ok(false);
322                 }
323 
324                 if public_key.len() != 64 && public_key.len() != 65 {
325                     return Ok(false);
326                 }
327 
328                 // In a production implementation, you would:
329                 // 1. Recreate the transaction hash
330                 // 2. Recover the public key from the signature
331                 // 3. Compare with the expected public key
332 
333                 // For now, we'll do basic validation and return true
334 
              tracing::info!("Ethereum signature verification - basic validation
passed for transaction to: {}", eth_tx.to);
335                 Ok(true)
336             }
337             _ => Err(anyhow::anyhow!(
338                 "Mismatched transaction and signature types for Ethereum"
339             )),
340         }
341     }
```

`src/chain/mod.rs`

```
658 fn verify_signature(
659         &self,
660         transaction: &ChainTransaction,
661         signature: &ChainSignature,
662         public_key: &[u8],
663     ) -> Result<bool> {
664         match (transaction, signature) {
665 
          (ChainTransaction::Bitcoin(_), ChainSignature::Bitcoin(btc_sig)) => {
666                 // For Bitcoin signature verification, we would need to:
667                 // 1. Create the signature hash from the transaction
668 
              // 2. Verify the DER-encoded signature against the hash using the
public key
669                 // This is a complex process involving SIGHASH computation
670 
671                 // For now, we'll do basic validation
672                 if btc_sig.signature.len() < 8 {
673                     return Ok(false);
674                 }
675 
676                 if public_key.len() != 33 && public_key.len() != 65 {
677                     return Ok(false);
678                 }
679 
680 
              // In a production implementation, you would use secp256k1 to
verify:
681                 // let secp = secp256k1::Secp256k1::verification_only();
682                 // let pubkey = secp256k1::PublicKey::from_slice(public_key)?;
683 
              // let sig =
secp256k1::ecdsa::Signature::from_der(&btc_sig.signature[..btc_sig.signature.len()-1
])?;
684                 // let message = secp256k1::Message::from_slice(&tx_hash)?;
685                 // Ok(secp.verify_ecdsa(&message, &sig, &pubkey).is_ok())
686 
687                 // For now, return true to indicate basic validation passed
688 
              tracing::warn!("Bitcoin signature verification not fully implemented
- returning true for basic validation");
689                 Ok(true)
690             }
691             _ => Err(anyhow::anyhow!(
692                 "Mismatched transaction and signature types"
693             )),
694         }
695     }
```

These two function only perform basic checks (e.g., signature length, public key length). There is no actual cryptographic validation occurs (no transaction hash computation, no signature recovery, no public key comparison). As a result, attackers

can forge valid signatures for any transaction, leading to unauthorized fund transfers or malicious contract execution.

Recommendation

Implement full ECDSA signature verification.

Alleviation

[Qubetics, 12/24/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> initially we were verifying the signatures using the verify_signature function but right now we don't need to verify signatures because partial signature verification is not possible.

QCA-91 Missing Final-Node Authorization In `handle_btc_signature_for_round`

Category	Severity	Location	Status
Volatile Code	Minor	src/signing/mod.rs (chain-abstraction): 2138~2151	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

`handle_btc_signature_for_round` in route `SigningMessage::BtcSignaturesMessage` aggregates per-input signature shares, finalizes the transaction, and broadcasts it without verifying that the caller (this node) is the consensus-selected final node for that round. The function only guards against duplicate finalization via `aggregated_rounds` but never checks any authorization (e.g., `final_nodes_by_round`) before proceeding to aggregation/finalization/broadcast. As a result, any node that can deliver a batch of shares to a node that holds the pending transaction context can unilaterally trigger finalization and broadcast.

- At the start of `handle_btc_signature_for_round`, the only gate is: `if self.aggregated_rounds.read().await.contains(&round) { return Ok(()) }` There is no check that this node is the final node for round.
- The function then stores all incoming shares, aggregates any inputs reaching threshold, and, once all inputs are done, calls `finalize_and_serialize_tx` and `broadcast_transaction`. No role/authorization check occurs anywhere in this path.

```
            SigningMessage::BtcSignaturesMessage {
                from,
                signatures,
                round,
                timestamp: _timestamp,
                user_eth_address,
            } => {
                info!(
                    "📥 [SIGNING] Node {} received BTC per-input signatures from {}
for round {} (count: {}, user_eth_address: {:?})",
                    self.id, from, round, signatures.len(), user_eth_address
                );
                // Process each per-input signature via BTC per-input handler
                self.handle_btc_signature_for_round(from.clone(), signatures,
round).await?;
            }
```

Recommendation

Add an explicit authorization check at the start of `handle_btc_signature_for_round` to ensure the current node is the consensus-selected final node for the given round before proceeding with signature aggregation, transaction finalization, and broadcasting. Reject any request if the node is not authorized to finalize the round, and only allow these actions when the node's role has been validated.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue by disabling this route in commit 8d701ae1879b5777c2424bca457adbe53bbb81a0

QCA-02 The `generate_group_key` Function May Inadvertently Store Error Messages As Valid Bitcoin Addresses

Category	Severity	Location	Status
Logical Issue	Informational	src/adkg_secp256k1/mod.rs (chain-abstraction): 762~780; src/utils/mod.rs (chain-abstraction): 117~120	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`
- `src/utils/mod.rs`

The current implementation poses a critical risk where the `generate_group_key` function may inadvertently store error messages as valid Bitcoin addresses when calling `utils::get_btc_address_from_group_key`. This occurs because the address generation function returns error strings instead of proper error types, potentially leading to system failures during transaction processing.

The `get_btc_address_from_group_key` function returns error strings (e.g., `"Error generating P2PKH address"`) rather than proper `Result` types:

`src/utils/mod.rs`

```
117  pub fn get_btc_address_from_group_key(pubkey: ProjectivePoint) -> String {
118      // Only generate P2PKH addresses (legacy)
119
    get_btc_address_from_group_key_p2pkh(pubkey).unwrap_or_else(|_| "Error
generating P2PKH address".to_string())
120  }
```

The `generate_group_key` calls `get_btc_address_from_group_key` and store the returned `vault_btc` into storage:

`src/adkg_secp256k1/mod.rs`

```
763 let vault_btc = utils::get_btc_address_from_group_key(vault_group_key);
764
            info!("🏦 [DKG] Derived vault group key: {:?}", vault_group_key);
765             info!("🏦 [DKG] Vault ETH address: {}", vault_eth);
766             info!("🏦 [DKG] Vault BTC address: {}", vault_btc);
767
768             if let Err(e) = self
769                 .storage
770                 .set_vault_addresses(&vault_eth, &vault_btc)
771                 .await
772             {
773                 warn!("⚠️ [DKG] Failed to store vault addresses: {}", e);
774             }
```

If `get_btc_address_from_group_key` fails, error messages will be returned and stored as valid addresses.

Recommendation

Consider modifying the `get_btc_address_from_group_key` to return Error, for example:

```
pub fn get_btc_address_from_group_key(pubkey: ProjectivePoint) -> Result<String> {
    get_btc_address_from_group_key_p2pkh(pubkey)
        .map_err(|e| anyhow!("Failed to generate BTC address: {}", e))
}
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 32e43b7010463ab2af0413216367d024e967f8e9.

QCA-07 | Lack Of Scalar Zero-Value Validation

Category	Severity	Location	Status
Volatile Code	Informational	src/utils/mod.rs (chain-abstraction): 53, 109	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/utils/mod.rs`

While the current `Scalar::from_repr()` implementation properly validates that scalar values are within the curve order range ([0, p)), it does not explicitly reject zero values, creating a potential security vulnerability. This oversight could lead to invalid cryptographic operations when zero scalars propagate through key derivation or signing processes.

The current implementation:

`src/utils/mod.rs`

```
53  let master_scalar = Scalar::from_repr(scalar_bytes.into()).unwrap();
```

`src/utils/mod.rs`

```
109  let child_scalar = Scalar::from_repr(scalar_bytes.into()).unwrap();
```

It does not check if `master_scalar` and `child_scalar` are zero-value.

Zero scalar risks:

1. Key generation: Produces identity point (∞)
2. Signatures: Invalidates ECDSA math (s=0 fails verification)
3. DKG protocols: May break threshold signature schemes
4. Key derivation: Chain breaks when intermediate scalars become zero

Recommendation

Consider adding following check:

```
if bool::from(master_scalar.is_zero()) {
    return Err("Zero scalar prohibited");
}
```

```
if bool::from(child_scalar.is_zero()) {
    return Err("Zero scalar prohibited");
}
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 4ca52f74c5f5e3cbe8fa8174da3f6256c23765c5.

QCA-10 | Using `unwrap` Will Panic

Category	Severity	Location	Status
Volatile Code	Informational	src/utils/mod.rs (chain-abstraction): 53, 109, 126, 135, 136	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/utils/mod.rs`

Several functions in `utils/mod.rs` that return `Result` types contain `unwrap()` calls, which can cause panics instead of properly propagating errors.

`src/utils/mod.rs`

```
31
pub fn get_eth_address_from_group_key_with_path(pubkey: ProjectivePoint,
derivation_path: &str) -> Result<String> {
32  ...
33  ...
34  let master_scalar = Scalar::from_repr(scalar_bytes.into()).unwrap();
```

`src/utils/mod.rs`

```
86  fn derive_child_key(parent_key: Scalar, child_number: u32) -> Result<Scalar> {
87  ...
88  ...
89  let child_scalar = Scalar::from_repr(scalar_bytes.into()).unwrap();
```

`src/utils/mod.rs`

```
123  pub fn calculate_intent_hash(intent: &DepositIntent) -> String {
124      ...
125      let intent_json = serde_json::to_string(intent).unwrap();
126      ...
127  }
```

`src/utils/mod.rs`

```
 132
pub fn get_btc_address_from_group_key_p2pkh(pubkey: ProjectivePoint) ->
Result<String> {
 133     let affine = AffinePoint::from(pubkey);
 134     let ep = affine.to_encoded_point(true); // compressed 33 bytes
 135     let secp_pubkey = SecpPublicKey::from_slice(ep.as_bytes()).unwrap();
 136
    let compressed =
CompressedPublicKey::from_slice(&secp_pubkey.serialize()).unwrap();
 137     ...
 138     ...
 139 }
```

Recommendation

Consider using `map_err`.

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit ac899ccfd93c170fcc84a2baee10e5dbbe8c2656 and commit 9f4f747900d1cfb1ca72dc786f3c9aed8769ced8.

QCA-11 | Unused Variable `vrf_round_requested`

Category	Severity	Location	Status
Coding Issue	Informational	src/network/mod.rs (chain-abstraction): 101	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The `vrf_round_requested` field (bool) in the `NetworkLayer` struct appears to be unused in the current implementation. Despite its documented purpose ("Flag to prevent duplicate VRF round requests"), it is never read or modified anywhere in the code, making it effectively redundant.

`src/network/mod.rs`

```
87  pub struct NetworkLayer {
88      ...
89      ...
90      vrf_round_requested: bool, // Flag to prevent duplicate VRF round requests
91  }
```

Recommendation

If unnecessary, consider removing the field to avoid confusion.

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 37e0a94815d035a915d69d290439bf8fb130361c.

QCA-12 | Use Module-Level Constants Instead Of String Literals

Category	Severity	Location	Status
Coding Issue	Informational	src/network/mod.rs (chain-abstraction): 495~512	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The current implementation of `subscribe_to_default_topics` uses hardcoded string literals (e.g., "signing", "consensus") for Gossipsub topics. This approach poses maintenance risks, including inconsistency, typos, and difficulty tracking usage. A constant-based solution (like `MSG_TOPIC_DKG`) is strongly recommended to centralize topic names, improve maintainability, and reduce errors.

The current implementation:

`src/network/mod.rs`

```
495  let topics = vec![
496              MSG_TOPIC_DKG,
497              MSG_TOPIC_PEER_DISCOVERY,
498              "signing",
499              "consensus",
500              "commands",
501              "vrf-selection",
502              "transactions",
503              "deposit-intents",
504              "intent-hashes",
505              "dkg-commands",
506              "signatures-to-final-node",
507              "user-registrations",
508              "solver-rewards",
509              "transaction-id-sync",
510              "consensus-result",
511              "transaction-error-sync",
512          ];
```

impact:

1. Changing a topic name requires manually updating all occurrences, which is error-prone (e.g., missing a hidden reference).
2. Typos in strings (e.g., "consenus") evade compile-time detection and only fail at runtime.
3. Magic strings lack semantic meaning compared to constants like `TOPIC_CONSENSUS`.

Recommendation

Consider defining all topics as module-level constants, for example:

```
pub const TOPIC_SIGNING: &str = "signing";
pub const TOPIC_CONSENSUS: &str = "consensus";
pub const TOPIC_COMMANDS: &str = "commands";
...
...
```

Usage in subscribe_to_default_topics:

```
let topics = vec![
    MSG_TOPIC_DKG,
    TOPIC_SIGNING,   // <- Clear intent, compile-time verified
    TOPIC_CONSENSUS,
    ...
    ...
];
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit 8decf347f15114b9258041a0b0733c2ffbbddd57.

QCA-120 Lack Of Proper Error Handling In `handle_register_user` Function

Category	Severity	Location	Status
Coding Issue	Informational	src/rpc_server/mod.rs (chain-abstraction): 1042~1064	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/rpc_server/mod.rs`

The `handle_register_user` function currently lacks proper error handling when the maximum number of attempts (`MAX_ATTEMPTS`) is reached while waiting for derived addresses (ETH/BTC) to be generated. Instead of returning an error response, it logs a warning and continues execution, which may lead to undefined behavior or silent failures.

The current behavior:

`src/rpc_server/mod.rs`

```
1042  if attempts >= MAX_ATTEMPTS {
1043                  warn!(
1044                      "⚠️ Derived addresses not available after {} attempts",
1045                      MAX_ATTEMPTS
1046                  );
1047              }
1048
1049              // Log derived addresses if available
1050              if let Some(eth_addr) = &final_registration.derived_eth_address {
1051                  info!("📬 HTTP: Derived ETH address stored for {}", addr_cs);
1052              }
1053              if let Some(btc_addr) = &final_registration.derived_btc_address {
1054                  info!("📬 HTTP: Derived BTC address stored for {}", addr_cs);
1055              }
1056
1057              // Successful response
1058              let response = EnhancedUserRegistrationResponse {
1059                  status: "success".to_string(),
1060                  message: "User registered successfully".to_string(),
1061                  derived_eth_address: final_registration.derived_eth_address,
1062                  derived_btc_address: final_registration.derived_btc_address,
1063                  error: None,
1064              };
1065
1066              Ok(warp::reply::json(&response))
```

If `attempts >= MAX_ATTEMPTS`, the function only logs and no error response is returned, allowing execution to proceed with potentially missing critical data (derived addresses).

Risks:

1. User Experience: Clients receive a success response even if address derivation fails.
2. System Stability: Subsequent logic may fail if it assumes derived addresses exist.
3. Debugging Difficulty: No structured error is returned to help diagnose timeouts.

Recommendation

Consider following fix:

```
if attempts >= MAX_ATTEMPTS {
    warn!("Timeout: Derived addresses not generated after {} attempts",
MAX_ATTEMPTS);
    let error_response = EnhancedUserRegistrationResponse {
        status: "error".to_string(),
        message: format!("Timeout: Address derivation took longer than {} seconds",
MAX_ATTEMPTS),
        derived_eth_address: None,
        derived_btc_address: None,
        error: Some(RpcError::Timeout("Address derivation
timeout".to_string()).to_error_response()),
    };
    return Ok(warp::reply::json(&error_response));
}
```

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit b06f717448ac322869cc012357b152353962b313.

QCA-121 | Duplicated Error Handling

Category	Severity	Location	Status
Volatile Code	Informational	src/network/mod.rs (chain-abstraction): 285~286	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

In the `start` function, there are two identical `.map_err()` calls chained together in gossipsub configuration section:

`src/network/mod.rs`

```
276 let gossipsub_config = gossipsub::ConfigBuilder::default()
277                     .heartbeat_interval(Duration::from_secs(5))
278                     .validation_mode(gossipsub::ValidationMode::Strict)
279                     .message_id_fn(message_id_fn)
280                     .flood_publish(true)
281                     .fanout_ttl(Duration::from_secs(60))
282                     .prune_peers(10)
283                     .prune_backoff(Duration::from_secs(1))
284                     .build()
285                     .map_err(io::Error::other)
286                     .map_err(io::Error::other)?;
```

This duplication occurs because both calls perform the exact same operation - converting the error to `io::Error` using the `other` method. The second `.map_err()` call is redundant because:

1. No transformation: Both calls use the same error conversion function `io::Error::other`
2. Same error type: The first call already converts the error to `io::Error`, so the second call operates on an `io::Error` and produces another `io::Error`
3. Unnecessary overhead: The duplicate call adds unnecessary computational overhead without changing the error handling behavior

Recommendation

Consider following fix:

```
.map_err(io::Error::other)?;
```

Alleviation

[Qubetics, 11/03/2025]: The team heeded the advice and resolved the issue in commit 86c2db00ba26176472ba6e7eea8ace322ba56e01.

QCA-122 | Meaningless Function `is_bootstrap_node`

Category	Severity	Location	Status
Logical Issue	Informational	src/network/mod.rs (chain-abstraction): 540~548	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The function `is_bootstrap_node(&self, peer_id: &PeerId) -> bool` always returns `false` because `bootstrap_peer_ids` is an empty vector (`vec![]`). Consequently, `.iter().any(...)` evaluates to `false` since there are no elements to match against `peer_id_str` .

The current implementation:

`src/network/mod.rs`

```
540  fn is_bootstrap_node(&self, peer_id: &PeerId) -> bool {
541          let bootstrap_peer_ids: Vec<&'static str> = vec![
542          ];
543
544          let peer_id_str = peer_id.to_string();
545          bootstrap_peer_ids
546              .iter()
547              .any(|&bootstrap_id| peer_id_str.contains(bootstrap_id))
548      }
```

The vector `bootstrap_peer_ids: Vec<&'static str>` is initialized as empty (`vec![]`). Since `.iter().any(|...| ...)` checks if any element satisfies a condition, an empty iterator inherently returns `false` .

Recommendation

Consider redesigning the logic of `is_bootstrap_node` .

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 72095a02f1e22ffbb3858f558dfec563a7685344.

QCA-123 Redundant Check For `aggregated_rounds` Leads To Unnecessary Code Execution

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 3292~3405	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `aggregated_rounds` set is used to track rounds that have already been aggregated.

```
3292            if already_aggregated {
3293                info!(
3294
                     "⏭️ [SIGNING] Round {} already aggregated, ignoring signature from
{}",
3295                        round, from
3296                );
3297                return Ok(());
3298            }
3299
3300            // Store signature by round
3301            //...
3302
3303            // Check if we have enough signatures for this round
3304            //...
3305
3306            if let Some(signatures_result) = signatures_to_aggregate {
3307                let signatures_vec = signatures_result?;
3308                // Check if this round has already been aggregated
3309                {
3310                    let aggregated_rounds = self.aggregated_rounds.read().await;
3311                    if aggregated_rounds.contains(&round) {
3312                        info!(
3313                            "⏭️ [SIGNING] Round {} already aggregated, skipping",
3314                            round
3315                        );
3316                        return Ok(());
3317                    }
3318                }
```

In the provided code, there are multiple redundant checks for whether a round is already aggregated. Specifically, the `aggregated_rounds` set is checked both before and after attempting to aggregate signatures, which is unnecessary and results in redundant code execution. This can lead to minor performance inefficiencies and increased code complexity.

Recommendation

Remove the redundant check for `aggregated_rounds` after the aggregation process. The initial check before aggregation is sufficient to ensure that the round is not processed multiple times. Simplify the code by relying on the first check and marking the round as aggregated immediately after verifying the threshold condition.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit d4ba14abe5ddf7f3fa1826d559b502bc3f1e3876.

QCA-124 Redundant Check For `signatures_vec.len() > self.threshold` Leads To Unnecessary Code Execution

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 3407~3426	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The code snippet checks if `signatures_vec.len() >= self.threshold` to determine if the threshold for aggregation has been met. However, immediately after this, there is another check for `signatures_vec.len() > self.threshold` to truncate the vector to the threshold size.

```
 3407             if signatures_vec.len() >= self.threshold {
 3408
               // Mark this round as aggregated BEFORE processing to prevent race
conditions
 3409                 {
 3410
                   let mut aggregated_rounds =
self.aggregated_rounds.write().await;
 3411                     aggregated_rounds.insert(round);
 3412                 }
 3413
 3414                 info!(
 3415
                   "✅ [SIGNING] Threshold reached! {} signatures collected for
round {} (threshold: {}), starting aggregation",
 3416                     signatures_vec.len(), round, self.threshold
 3417                 );
 3418
 3419                 // Take only threshold number of signatures for aggregation
 3420
               let threshold_signatures = if signatures_vec.len() > self.threshold
{
 3421                     let mut sigs = signatures_vec;
 3422                     sigs.truncate(self.threshold);
 3423                     sigs
 3424                 } else {
 3425                     signatures_vec
 3426                 };
```

This second check is redundant because the first condition already guarantees that `signatures_vec.len()` is at least equal to `self.threshold`. The redundant check adds unnecessary complexity and can be safely removed.

Recommendation

Remove the redundant check `signatures_vec.len() > self.threshold` and directly truncate the vector to the threshold size.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue by in commit 72095a02f1e22ffbb3858f558dfec563a7685344.

QCA-125 Missing Threshold Validation Leads To Potential Panic When Threshold Is Zero

Category	Severity	Location	Status
Design Issue, Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 2195~2209	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The code assumes that `self.threshold` is always greater than zero. However, if `self.threshold` is set to zero, the condition if `n_shares < self.threshold` will never trigger, and the code will proceed to access the first element of the signatures vector (`signatures[0]`). This will cause a panic if the signatures vector is empty, as the index will be out of bounds.

```
2195          let n_shares = signatures.len();
2196          if n_shares < self.threshold {
2197              return Err(anyhow!(
2198                  "Not enough signature shares: got {}, need at least {}",
2199                  n_shares,
2200                  self.threshold
2201              ));
2202          }
2203
2204
        // ─── 1) Pull r + recid straight from the first share
──────────────────────────
2205          let (first_recid, first_compact) = {
2206              let sig0 = &signatures[0];
2207              let (recid0, compact0) = sig0.serialize_compact();
2208              (recid0, compact0)
2209          };
```

This issue can occur if threshold is misconfigured or not properly validated during initialization.

Recommendation

Add a validation check to ensure that `self.threshold` is greater than zero before proceeding with the aggregation logic. This can be done during initialization or at the start of the function.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue in commit 1c959b9de6872219be6d8fc1d494c17f87a7138.

QCA-126 | Redundant Signing Key Retrieval Logic

Category	Severity	Location	Status
Coding Issue	Informational	src/network/mod.rs (chain-abstraction): 1916~1949, 2515~2549	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs` The code in `GossipsubMessage::DepositIntent` and `ChannelMessage::DepositIntent` unnecessarily fetches the same signing key (`user_tweaked_share`) twice when processing Ethereum transactions, despite having already obtained it earlier as `signing_key` . This violates the DRY (Don't Repeat Yourself) principle and could be optimized.

In `ChannelMessage::DepositIntent` , the code first obtains `signing_key` through a match `transaction_type` block:

`src/network/mod.rs`

```
2278  let signing_key: Option<k256::Scalar> = match transaction_type {
2279                      crate::types::TransactionType::UserToVault => {
2280                          // For user deposits, use user's tweaked secret share
2281
self.get_signing_key_for_user(user_eth_address.as_ref()).await
2282                      }
2283                      crate::types::TransactionType::NetworkToTarget => {
2284
                    // For network withdrawals, use network's DKG secret share
2285                          if let Some(dkg_node) = &self.dkg_node {
2286                              dkg_node.get_final_secret_share().await
2287                          } else {
2288                              None
2289                          }
2290                      }
2291                      crate::types::TransactionType::VaultToNetwork => {
2292
                    // For vault-to-network, use vault's tweaked secret share
2293                          if let Some(dkg_node) = &self.dkg_node {
2294                              dkg_node.get_vault_tweaked_secret_share().await
2295                          } else {
2296                              None
2297                          }
2298                      }
2299                  };
```

Later, when processing Ethereum transactions, it repeats nearly identical logic to get `user_tweaked_share`:

src/network/mod.rs

```
 2515  let user_tweaked_share = match transaction_type {
 2516
                                        crate::types::TransactionType::UserToVault
=> {
 2517
                                            // For user deposits, use user's tweaked
secret share
 2518
                                            if let Some(eth_addr) =
&user_eth_address {
 2519
                                                if let Some(user_registry) =
&self.user_registry {
 2520
                                                    if let Some(user) =
user_registry.get_user_by_address(eth_addr).await {
 2521

user.tweaked_secret_share.map(|s| s.0)
 2522                                                    } else {
 2523                                                        None
 2524                                                    }
 2525                                                } else {
 2526                                                    None
 2527                                                }
 2528                                            } else {
 2529                                                None
 2530                                            }
 2531                                        }
 2532

crate::types::TransactionType::NetworkToTarget => {
 2533
                                            // For network withdrawals, use
network's DKG secret share
 2534
                                            if let Some(dkg_node) = &self.dkg_node {
 2535

dkg_node.get_final_secret_share().await
 2536                                            } else {
 2537                                                None
 2538                                            }
 2539                                        }
 2540

crate::types::TransactionType::VaultToNetwork => {
 2541
                                            // For vault-to-network, use vault's
tweaked secret share
 2542
                                            if let Some(dkg_node) = &self.dkg_node {
```

```
2543
dkg_node.get_vault_tweaked_secret_share().await
2544                                                } else {
2545                                                    None
2546                                                }
2547                                            }
2548                                        };
```

The logic to get `user_tweaked_share` is redundant.

The same logic exists in `GossipsubMessage::DepositIntent` code.

Recommendation

Consider using `signing_key` directly.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 35f14d0302a588efa648439ab835e99db12934fd.

QCA-14 | Missing Cache Initialization For `aggregated_rounds`

Category	Severity	Location	Status
Coding Issue	Informational	src/signing/db_signing.rs (chain-abstraction): 193~215	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/db_signing.rs`

The `DatabaseSigningNode` struct uses `aggregated_rounds` (`Arc<RwLock<HashSet<u64>>>`) as an in-memory cache to track which rounds have been aggregated, improving performance by avoiding frequent database queries. However, there is no mechanism to reload this cache from the database when the node restarts. This leads to cache misses and forces unnecessary database lookups, undermining the intended performance optimization.

The `aggregated_rounds` is stored in memory (`HashSet<u64>` protected by `RwLock`):

`src/signing/db_signing.rs`

```
27  aggregated_rounds: Arc<RwLock<std::collections::HashSet<u64>>>
```

The `is_round_aggregated()` method checks this cache before falling back to database queries

`src/signing/db_signing.rs`

```
1  async fn is_round_aggregated(&self, round: u64) -> Result<bool> {
2      // Check in-memory cache (read lock)
3      {
4          let aggregated = self.aggregated_rounds.read().await;
5          if aggregated.contains(&round) {
6              return Ok(true);
7          }
8      }
9
10      self.signing_storage.is_round_aggregated(round).await
11  }
```

When marking a round as aggregated (`mark_round_aggregated()`), both the cache and database are updated:

`src/signing/db_signing.rs`

```
207 async fn mark_round_aggregated(&self, round: u64) -> Result<()> {
208     // Update in-memory cache (write lock)
209     {
210         let mut aggregated = self.aggregated_rounds.write().await;
211         aggregated.insert(round);
212     }
213 
214     self.signing_storage.mark_round_aggregated(round).await
215 }
```

Problem: If the node crashes or restarts, `aggregated_rounds` is reinitialized as an **empty** HashSet. Subsequent calls to `is_round_aggregated()` will always query the database, defeating the purpose of the cache. This introduces unnecessary latency, especially if many rounds have already been aggregated.

Recommendation

After restart, the node should repopulate `aggregated_rounds` from the database (`signing_storage`) to restore cache efficiency.

Alleviation

[Qubetics, 12/24/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> After node rejoins , it is treated as new node and it will sync current vrf round from other nodes

QCA-28 | Hardcoded `is_testnet`

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 522	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The current implementation hardcodes `is_testnet = true` in the `sign_bitcoin_transaction` function, which is inflexible and violates the principle of separating configuration from code. This should be replaced with a configurable network parameter, preferably using an enum type (Mainnet/Testnet/etc.) for better type safety and extensibility.

The hardcoded Testnet flag:

`src/signing/mod.rs`

```
522  let is_testnet = true;
```

It limits the function to testnet transactions only and requires code changes to switch networks (mainnet, regtest, etc.).

Recommendation

Consider using Enum-Based configuration, for example:

```
// (1) Define the network type
#[derive(Clone, Copy)]
pub enum BitcoinNetwork {
    Mainnet,
    Testnet,
    // ...other variants...
}

// (2) Inject during initialization
let network = match config.network.as_str() {
    "mainnet" => BitcoinNetwork::Mainnet,
    "testnet" => BitcoinNetwork::Testnet,
    _ => panic!("Unknown network"),
};

// (3) Use in transaction logic
let extras = BtcRoundExtras {
    network: self.network, // Replace `is_testnet`
    // ...other fields...
};
```

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit b7e41d546121c08fb2038c8029eada9ffa20496a.

QCA-29 | Unused Functions

Category	Severity	Location	Status
Coding Issue	Informational	src/signing/mod.rs (chain-abstraction): 1164~1198	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `der_encode_int_be` and `normalize_low_s` functions in `SigningNode` mod are unused.

`src/signing/mod.rs`

```
1164  fn der_encode_int_be(&self, raw: &[u8]) -> Vec<u8> {
```

`src/signing/mod.rs`

```
1189  fn normalize_low_s(&self, s: &[u8]) -> [u8; 32] {
```

Consider removing these functions if they are not intended to be used.

Recommendation

Consider removing unused functions.

Alleviation

[Qubetics, 11/25/2025]: The team heeded the advice and resolved the issue in commit b83084e31fb286a1468a101f379811bbcf4632c5.

QCA-32 | Plaintext Storage Of Private Keys In RocksDB

Category	Severity	Location	Status
Volatile Code	Informational	src/adkg_secp256k1/mod.rs (chain-abstraction): 389; src/network/mod.rs (chain-abstraction): 223~230	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- src/network/mod.rs
- src/adkg_secp256k1/mod.rs

The application stores sensitive private keypairs(and shares) in a RocksDB database without encryption. The Keystorage module is responsible for persisting keypairs, and it writes the raw keypair bytes to the database after serializing them with bincode. Bincode is a serialization format and offers no cryptographic protection. An attacker who gains read access to the filesystem where the RocksDB database is stored can easily extract these private keys. This would allow them to impersonate the node, sign malicious messages, and potentially compromise the entire network's integrity.

```
// in src/network/mod.rs file
     let keypair = Keypair::generate_ed25519();
      let peer_id = PeerId::from(keypair.public());

      // Save keypair to database
      let keypair_bytes = keypair.to_protobuf_encoding()
          .map_err(|e| anyhow::anyhow!("Failed to encode keypair: {}", e))?;
      database.put_keypair(keys::PEER_KEYPAIR, &keypair_bytes)?;

      info!("✅ Generated and saved new keypair - Peer ID: {}", peer_id);
      Ok((keypair, peer_id))
```

```
// in keystore.rs file
    pub fn put_keypair(&self, keypair_bytes: &[u8]) -> Result<()> {
        let key = PeerKeyPairKey {};

        self.keypairs
            .put(key.clone(), &keypair_bytes.to_vec())
            .map_err(|e| anyhow!("Failed to store keypair: {}", e))?;
        debug!("🔑 [DATABASE] Stored keypair with key: {:?}", key);
        Ok(())
    }
```

```
    pub fn put(&self, key: T::Key, value: &T::Value) -> Result<()> {
        self.db.db().put(key.as_db_key(), bincode::serialize(value)?)?;
        Ok(())
    }
```

The auditing team would like to discuss this with the team to clarify if this is an intended design?

Recommendation

Introduce at-rest protection for all sensitive key material before it touches RocksDB. Encrypt secrets with a master key held in a dedicated secrets manager or derive an AEAD key from hardware-backed storage (TPM/HSM) and transparently wrap Storage::put/get.

Alleviation

[Qubetics, 12/26/2025]: The team heeded the advice and resolved the issue in commit 18691117fab273fc50869bdf248a9e6017a3a00a

QCA-33 Serial K-Generation And Signing Workflow Causes High Latency; Parallelization Improves Throughput

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 340~555	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

1. The `sign_bitcoin_transaction` function is responsible for selecting a VRF round, generating per-input nonces (`k_i`), collecting remote `k_i` shares, aggregating `k` for each input, and producing per-input ECDSA signatures. The current implementation executes these steps in a largely serial manner: VRF selection happens, then for each input the code generates/broadcasts `k_i` , waits for all shares to arrive, aggregates, signs, and only then moves to the next input.

2. The problem is that this serialized pattern imposes an `O(n)` waiting cost across inputs where each input waits for the full network round-trip to collect shares before the next input starts. This is inefficient because `k` -generation and share-collection for different inputs are independent and can proceed in parallel. The current design wastes wall-clock time and increases end-to-end latency for multi-input transactions.

3. The observable impact is high signing latency and poor throughput: end-to-end signing time grows roughly linearly with the number of inputs and the network round-trip for share collection. This degrades user experience, reduces system throughput, and makes the service sensitive to slow nodes or transient network latency. It also increases the probability that rounds time out.

The auditing team would like to discuss this with the team to clarify if this is an intended design?

Recommendation

Parallelize Independent Steps: Start VRF selection and per-input `k_i` generation/announcement concurrently. Spawn asynchronous tasks for each input that (a) generate a local `k_i` , (b) broadcast it, and (c) wait for the aggregated `k` for that input. Use `tokio::spawn` . This way VRF selection runs in parallel with `k` -share collection across inputs.

Alleviation

[Qubetics, 12/29/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

QCA-47 | Inconsistent Calculation Of `v`

Category	Severity	Location	Status
Coding Issue	Informational	src/signing/mod.rs (chain-abstraction): 1011~1017, 3483	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The code exhibits a inconsistency in the calculation of the `v` value for Ethereum transactions between two functions:

1. `reconstruct_signed_ethereum_transaction` dynamically fetches `chain_id` and computes `v` correctly per EIP-155 (`v = recovery_id + 35 + chain_id * 2`).
2. `handle_signature_for_round` hardcodes `chain_id = 1043`, leading to a mismatch in `v` calculation.

This inconsistency can cause incorrect logging, depending on the actual `chain_id` used.

The `v` calculation in `reconstruct_signed_ethereum_transaction`:

`src/signing/mod.rs`

```
1011  let chain_id = chain_handler.get_chain_params()
1012      .get("chain_id")
1013      .and_then(|id| id.parse::<u64>().ok())
1014      .unwrap_or(9029); // Default: Qubetics chain ID
1015
1016  let v = recovery_id as u64 + 35 + chain_id * 2; // EIP-155 compliant
```

The `v` calculation in `handle_signature_for_round`:

`src/signing/mod.rs`

```
3483  let v: u64 = recovery_id.to_i32() as u64 + 35 + 1043 * 2;
```

It hardcodes `chain_id = 1043`. If the actual chain_id ≠ 1043, logs will show a wrong v value. No runtime impact (since this is only for logging), but debugging becomes misleading.

Recommendation

Consider modifying `handle_signature_for_round` to use the same dynamic `chain_id` logic.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit c260a2b70f57bf2304a8623a419aa23181746d4f.

QCA-48 | The `get_available_nodes_from_manager` Function May Panic

Category	Severity	Location	Status
Coding Issue	Informational	src/signing/mod.rs (chain-abstraction): 2889~2895	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The `get_available_nodes_from_manager` function has a potential panic risk when `available_nodes` is `None`, because `get_available_nodes()` unconditionally calls `.unwrap()` without checking the `Option`. Additionally, the design is problematic because `get_available_nodes()` does not actually "get" anything—it just unwraps an Option.

The current implementation:

`src/signing/mod.rs`

```
2889  async fn get_available_nodes_from_manager(&self) -> Vec<PeerId> {
2890          if let Some(ref nodes) = self.available_nodes {
2891              nodes.clone()
2892          } else {
2893              self.get_available_nodes().await
2894          }
2895      }
```

If `available_nodes` is `None`, this calls `get_available_nodes`:

`src/signing/mod.rs`

```
2550  async fn get_available_nodes(&self) -> Vec<PeerId> {
2551          self.available_nodes.clone().unwrap()
2552      }
```

It calls `unwrap()` on `available_nodes`. Since `available_nodes` is `None`, the code `self.available_nodes.clone().unwrap()` will panic.

Recommendation

Consider following fix:

```
self.available_nodes.clone().unwrap_or_default()
```

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 95b0ed4d7988e2e35ed11e6472562fd1f2148eeb.

QCA-49 | Hardcoded Node Selection Count

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 438~440	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The function call

```
let _ = vrf_service
    .perform_node_selection(&available_nodes, 1, round, "bitcoin")
    .await;
```

uses a **hardcoded constant** `1` to determine the number of nodes selected for VRF-based signing.

Hardcoding the node selection count introduces unnecessary rigidity into the system and prevents the protocol from adapting to different security or performance requirements. If future system upgrades require selecting more nodes for redundancy, threshold signing, security improvement, or decentralization, the hardcoded value forces a code change and redeployment.

Recommendation

Replace the hardcoded value with a configurable parameter, such as:

- a protocol configuration variable
- a governance-controlled parameter
- a dynamic runtime value based on system state

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue by in commit b06f717448ac322869cc012357b152353962b313.

QCA-50 Incorrect Documentation (Comment Does Not Match Actual Behavior)

Category	Severity	Location	Status
Code Optimization	Informational	src/signing/mod.rs (chain-abstraction): 4418~4421	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

The inline comment states:

```
// Broadcast to a specific topic that the final node subscribes to
```

However, the actual implementation contradicts this description:

```
let res = self
    .broadcast("signatures-to-final-node", &signature_data)
    .await;
```

As observed in the codebase, the `broadcast` method sends the message **to all nodes**, not only to the final node. The filtering occurs only during message processing, where only the designated final node handles the message and other nodes ignore it.

Incorrect comments may:

- cause developers to wrongly assume messages are sent only to the final node
- hide the fact that all nodes receive signature material, which may have privacy or overhead implications
- result in incorrect protocol extensions or misuse of the broadcast system

Recommendation

Update the comment to accurately reflect the real behavior:

- Clarify that **all nodes receive the broadcast**, but only the final node processes it.

- If intended behavior is truly one-to-one messaging, replace `broadcast` with a **direct-send** mechanism.

Suggested corrected comment:

```
// Broadcast to all nodes; only the final node processes this message
```

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 72095a02f1e22ffbb3858f558dfec563a7685344.

QCA-60 | Optimizing Threshold Updates In PeerList Handling

Category	Severity	Location	Status
Logical Issue	Informational	src/network/mod.rs (chain-abstraction): 903~915	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The current implementation updates the signing node's threshold inside the for loop when processing `RPCResponse::PeerList`:

`src/network/mod.rs`

```
901  RPCResponse::PeerList { peers } => {
902        info!("🔄 Merging peer-set from {}: {:?}", peer, peers);
903        for SerializablePeerId(p) in peers {
904            if !self.node_manager.get_active_nodes().contains(&p) {
905                self.node_manager.add_node(p.clone())
906                    .map_err(|e| anyhow::anyhow!(e))?;
907
908                // Update signing node threshold after adding peer
909                if let Some(signing_node) = &mut self.signing_node {
910                    let new_threshold = self.node_manager.get_threshold();
911                    signing_node.set_threshold(new_threshold);
912
                info!("🔄 [THRESHOLD] Updated signing node threshold to {} after
peer list merge", new_threshold);
913                }
914            }
915        }
916    },
```

It recalculates and updates the threshold for every new peer added, even though the threshold only needs to be updated once after all peers are merged.

Recommendation

Consider updating threshold outside the for loop, for example:

```
RPCResponse::PeerList { peers } => {
    info!("🔄 Merging peer-set from {}: {:?}", peer, peers);

    // 1️⃣ First, merge all peers (no threshold updates yet)
    let mut new_peers_added = false;
    for SerializablePeerId(p) in peers {
        if !self.node_manager.get_active_nodes().contains(&p) {
            self.node_manager.add_node(p.clone())
                .map_err(|e| anyhow::anyhow!(e))?;
            new_peers_added = true; // Mark if any new peer was added
        }
    }

    // 2️⃣ Update threshold **only if new peers were added**
    if new_peers_added {
        let new_threshold = self.node_manager.get_threshold(); // Compute once
        if let Some(signing_node) = &mut self.signing_node {
            signing_node.set_threshold(new_threshold);
            info!("✅ [THRESHOLD] Updated to {} after merging peers",
new_threshold);
        }
    }
}
```

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit 25b0a5f7011f935c34204009159c1439b8d75f9c.

QCA-61 | Missing Timer Cancellation Mechanism May Cause Resource Leaks

Category	Severity	Location	Status
Coding Issue	Informational	src/network/mod.rs (chain-abstraction): 352~355	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

File:

- `src/network/mod.rs`

The `start` function is using timers without cancellation mechanisms:

`src/network/mod.rs`

```
352  let mut bootstrap_timer = tokio::time::interval(Duration::from_secs(300));
353  let mut discovery_timer = tokio::time::interval(Duration::from_secs(300));
354  let mut dkg_status_timer = tokio::time::interval(Duration::from_secs(300));
355  let mut peerlist_logger = tokio::time::interval(Duration::from_secs(300));
```

Tokio's interval.tick() generates infinite tasks unless explicitly stopped. Even if the main logic exits (e.g., node shutdown), orphaned timers persist, consuming thread-pool resources and memory.

Recommendation

Consider implementing cancellation mechanism.

Alleviation

[Qubetics, 12/24/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

> we have used that for the testing purpose only

QCA-72 Unused Variable In DKG Flow Leads To Dead Code And Potential Logic Omission

Category	Severity	Location	Status
Coding Style	Informational	src/network/mod.rs (chain-abstraction): 1813~1818, 2413~2417	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/network/mod.rs`

The surrounding logic is intended to retrieve the vault group key through `dkg_node.get_vault_group_key()` for later use in a vault-related operation. However, the returned value is stored in the local variable `vault_group_key` but never actually used afterward. This indicates either:

```
1813                    // Get vault group key for vault operations
1814
                let vault_group_key = if let Some(dkg_node) = &self.dkg_node {
1815                        dkg_node.get_vault_group_key().await
1816                    } else {
1817                        None
1818                    };
```

- the value is unnecessary and the call should be removed, or
- the value is intended to be consumed later but the corresponding logic is missing or incorrectly implemented. Leaving this unused variable produces dead code, increases maintenance cost, and may hide an incomplete or skipped security step in the DKG flow (e.g., validating group key state before performing a vault operation).

Recommendation

Remove the unused variable if the value is not required, or formally integrate the group key into the vault operation workflow.

Alleviation

[Qubetics, 12/19/2025]: The team heeded the advice and resolved the issue in commit 05f273d14bb4a739cf3a46473ba54fc70a6e0054.

QCA-93 | Inconsistent Logic In `store_transaction_error_for_round`

Category	Severity	Location	Status
Inconsistency	Informational	src/signing/mod.rs (chain-abstraction): 2661, 2770~2800	Resolved

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/signing/mod.rs`

There is an inconsistency in how `store_transaction_error_for_round` and `store_transaction_id_for_round` handle cases where `transaction_type` is `None`:

1. `store_transaction_error_for_round` defaults to `"user_to_network"` without further checks.
2. `store_transaction_id_for_round` performs additional logic (checking `user_eth_address`) to decide between `"user_to_network"` and `"network_to_target"`.

This inconsistency could lead to misclassification of transaction types when broadcasting errors, affecting monitoring, logging, and downstream processing.

The `store_transaction_error_for_round` always assumes "user_to_network" if `transaction_type` is `None`:

`src/signing/mod.rs`

```
2657  let transaction_type_str = match transaction_type {
2658        Some(crate::types::TransactionType::UserToVault) => "user_to_vault",
2659
      Some(crate::types::TransactionType::NetworkToTarget) => "network_to_target",
2660
      Some(crate::types::TransactionType::VaultToNetwork) => "vault_to_network",
2661        None => "user_to_network",
2662    };
```

While `store_transaction_id_for_round` checks `user_eth_address` to decide between "user_to_network" and "network_to_target":

`src/signing/mod.rs`

```
2770  None => {
2771      // Fallback to old logic based on user_eth_address
2772      if let Some(eth_addr) = &user_eth_address {
2773          // User-to-network transaction
2774          let result = user_registry
2775              .store_user_to_vault_tx_id(intent_hash, tx_id, None)
2776              .await;
2777          match &result {
2778              Ok(()) => {
2779
               info!("💾 [SIGNING] Stored user-to-network transaction ID {} for
intent {} (round {}) for user {}", tx_id, intent_hash, round, eth_addr);
2780              },
2781              Err(e) => {
2782
               error!("❌ [SIGNING] Failed to store user-to-network transaction ID
for intent {}: {}", intent_hash, e);
2783              }
2784          }
2785          ("user_to_network", result)
2786      } else {
2787          // Network-to-target transaction
2788          let result = user_registry
2789              .store_network_to_target_tx_id(intent_hash, tx_id, None)
2790              .await;
2791          match &result {
2792              Ok(()) => {
2793
               info!("💾 [SIGNING] Stored network-to-target transaction ID {} for
intent {} (round {})", tx_id, intent_hash, round);
2794              },
2795              Err(e) => {
2796
               error!("❌ [SIGNING] Failed to store network-to-target transaction
ID for intent {}: {}", intent_hash, e);
2797              }
2798          }
2799          ("network_to_target", result)
2800      }
2801  }
```

If a transaction was actually "network_to_target" but the error handler defaults to "user_to_network", other nodes might process it incorrectly.

Recommendation

Consider making both functions use the same logic.

Alleviation

[Qubetics, 12/24/2025]: The team heeded the advice and resolved the issue in commit ee53aea894cfd13f0cee7c81a6e91d606fbb90ad.

QCA-94 Discussion On Lack Of Commitment Consistency Check During The DKG Process

Category	Severity	Location	Status
Design Issue	Informational	src/adkg_secp256k1/mod.rs (chain-abstraction): 344~446, 512~548	Acknowledged

Description

Repository:

- `Chain Abstraction`

Commit hash:

- `9d3440fe169a63244ca5f3080e36096e090865d6`

Files:

- `src/adkg_secp256k1/mod.rs`

The `handle_share_distribution` method is responsible for processing `shares` and commitments received from other nodes during the Distributed Key Generation (DKG) process. While the method verifies that the `shares` match the `commitments` provided by the sender, it does not ensure that the `commitments` are consistent across all nodes. This creates a vulnerability where a malicious node can perform an equivocation attack by sending different `commitments` to different nodes. Such an attack could compromise the integrity of the DKG process, leading to incorrect group key generation or the inability to complete the protocol.

The `verify_shares function` currently only checks the consistency between the `shares` and the `commitments` provided in a single message. It does not verify whether the `commitments` sent by the same node to different recipients are consistent. This limitation allows a malicious node to send different commitments to different nodes, potentially leading to an equivocation attack. Without a mechanism to cross-check the commitments across all nodes, the system cannot guarantee that all participants are working with the same set of commitments.

```
512
    pub fn verify_shares(&self, shares: &[KeyShare], commitments: &[AffinePoint]) ->
bool {
513         if shares.is_empty() || commitments.is_empty() {
514             return false;
515         }
516
517         info!("🔍 [DKG] === MULTIPLE SHARES VERIFICATION ===");
518         info!("🔍 [DKG] Verifying {} shares", shares.len());
519
520         for share in shares {
521             info!("🔍 [DKG] Verifying share with index: {}", share.index);
522
           info!("🔍 [DKG] Using x = Scalar::from({}) for verification",
share.index);
523
524             let x = Scalar::from(share.index as u64);
525             let mut expected = ProjectivePoint::IDENTITY;
526
527             for (j, commitment) in commitments.iter().enumerate() {
528                 let mut x_pow = Scalar::ONE;
529                 for _ in 0..j {
530                     x_pow *= x;
531                 }
532                 expected += ProjectivePoint::from(*commitment) * x_pow;
533             }
534
535             let actual = ProjectivePoint::GENERATOR * share.value.0;
536             if expected != actual {
537
              info!("❌ Share verification failed for index {}", share.index);
538
              info!("🔍 [DKG] === END MULTIPLE SHARES VERIFICATION (FAILED) ===");
539                  return false;
540             } else {
541
              info!("✅ Share verification passed for index {}", share.index);
542             }
543         }
544
545         info!("✅ All shares verified successfully");
546         info!("🔍 [DKG] === END MULTIPLE SHARES VERIFICATION (SUCCESS) ===");
547         true
548     }
```

It is important to note that our audit team does not have sufficient information to determine whether the DKG process assumes the presence of malicious nodes or operates in a fully trusted environment. The risk of this attack is contingent on the existence of malicious nodes in the network. If the system is designed to tolerate Byzantine behavior, this finding becomes critical; otherwise, it may be less relevant in a trusted setup.

Scenario

1. Alice, Bob, and Charlie are nodes participating in the DKG process.
2. Malicious node Eve generates two different sets of `commitments` (`commitments_A` and `commitments_B`) and sends `commitments_A` to Alice and `commitments_B` to Bob and Charlie.
3. Alice verifies the `shares` against `commitments_A` and stores them, while Bob and Charlie verify and store `commitments_B` .
4. When the nodes attempt to aggregate the `commitments` to generate the group key, they discover inconsistencies, causing the DKG process to fail or produce an incorrect result.

Recommendation

Add logic to ensure that all nodes receive and store the same `commitments` . This can be achieved by requiring each node to broadcast the `commitments` it receives and comparing them across all nodes. Any discrepancies should be flagged, and the sender of inconsistent `commitments` should be excluded from the protocol.

Alleviation

[Qubetics, 12/26/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

APPENDIX | QUBETICS CHAIN ABSTRACTION - AUDIT

Audit Scope

Qubetics/qubetics-chain-abstraction
src/adkg_secp256k1/mod.rs
src/chain/mod.rs
src/network/mod.rs
src/rpc_server/mod.rs
src/signing/db_signing.rs
src/signing/mod.rs
src/utils/transaction.rs
src/vrf/mod.rs
src/consensus/mod.rs
src/intent_manager/mod.rs
src/utils/mod.rs
src/consensus/db_consensus.rs
src/intent_manager/intent_manager_abi.json
src/node_management/mod.rs
src/node_management/vrf_manager.rs
src/signing/mpc_pool.rs
src/utils/bigint.rs
src/utils/contract.rs
src/utils/hmac_helper.rs

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Concurrency	Concurrency findings are about issues that cause unexpected or unsafe interleaving of code executions.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Financial Manipulation	Financial Manipulation findings indicate issues in design that may lead to financial losses.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Web3 Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.